

深度学习

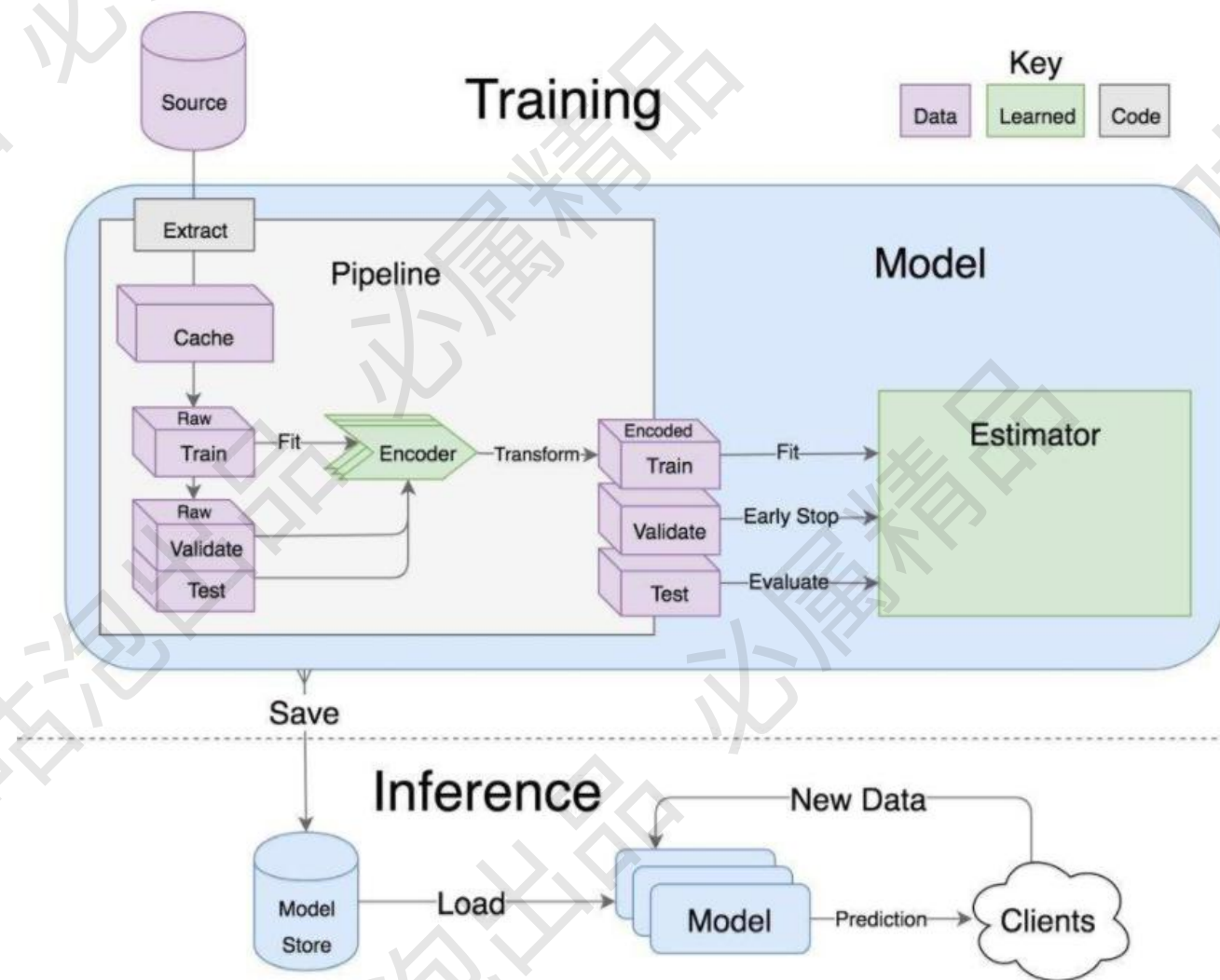
✓ 机器学习流程：

✎ 数据获取

✎ 特征工程

✎ 建立模型

✎ 评估与应用



深度学习

✓ 特征工程的作用：

✎ 数据特征决定了模型的上限

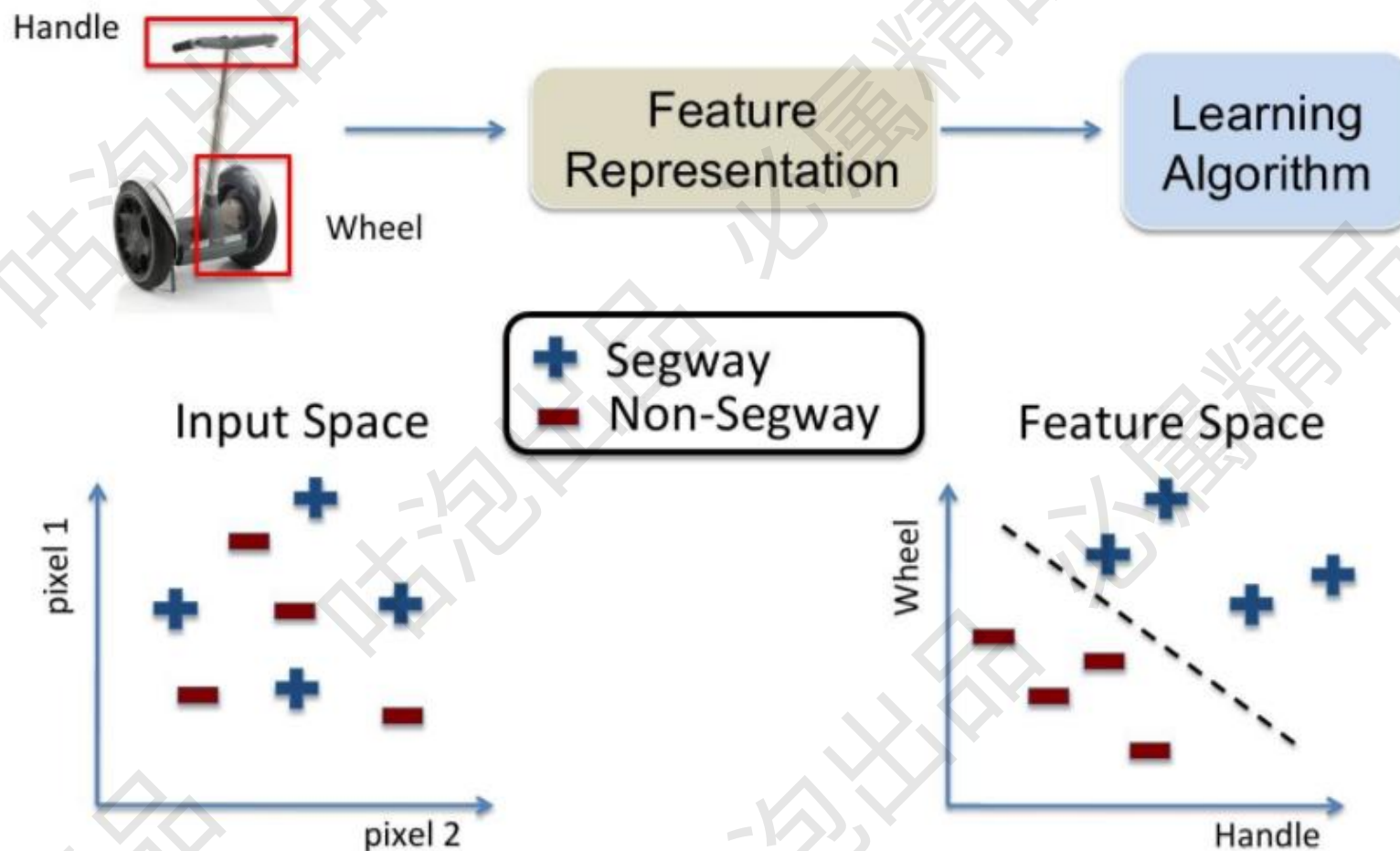
✎ 预处理和特征提取是最核心的

✎ 算法与参数选择决定了如何逼近这个上限



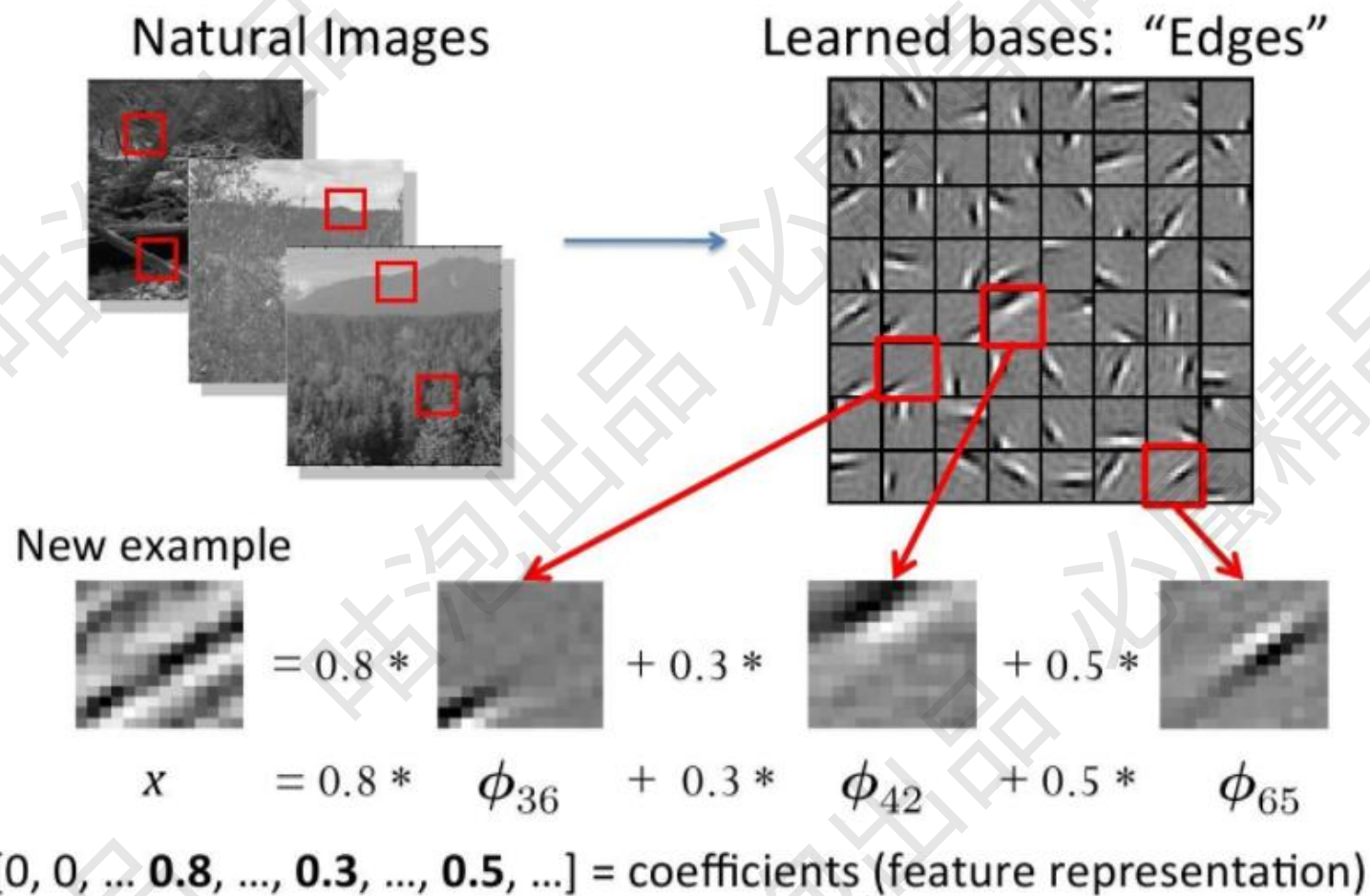
深度学习

✓ 特征如何提取：



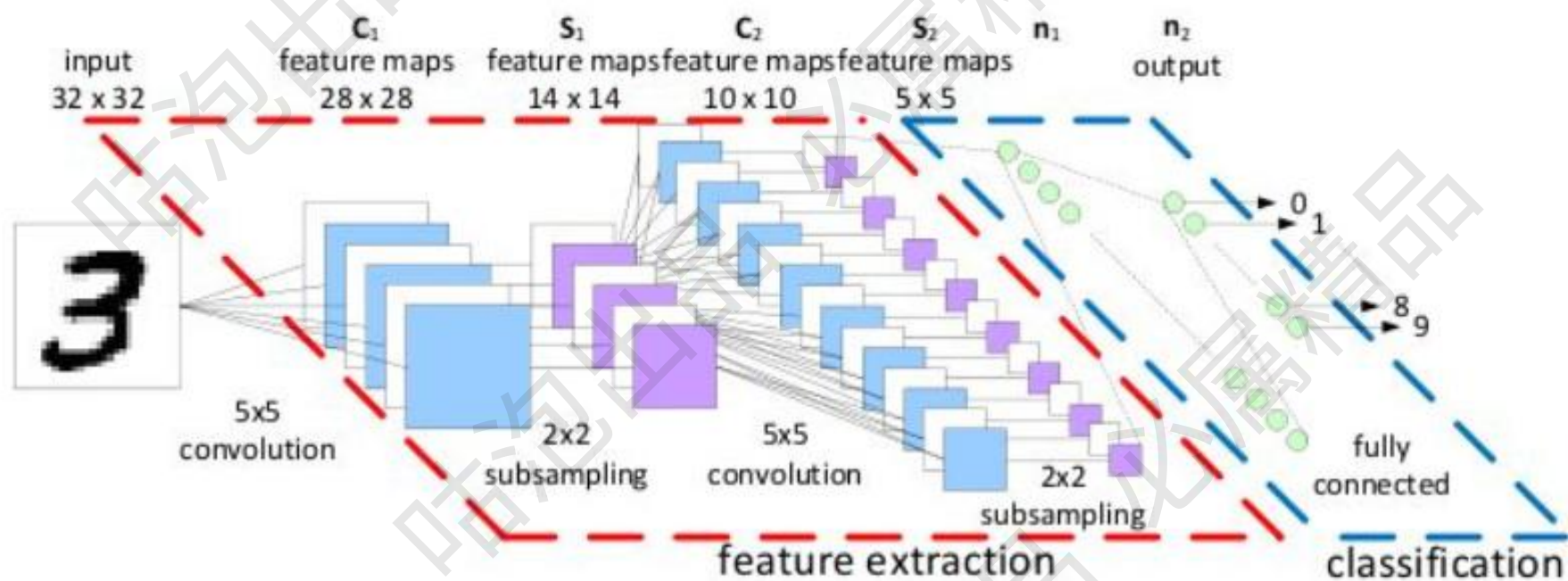
深度学习

✓ 传统特征提取方法:



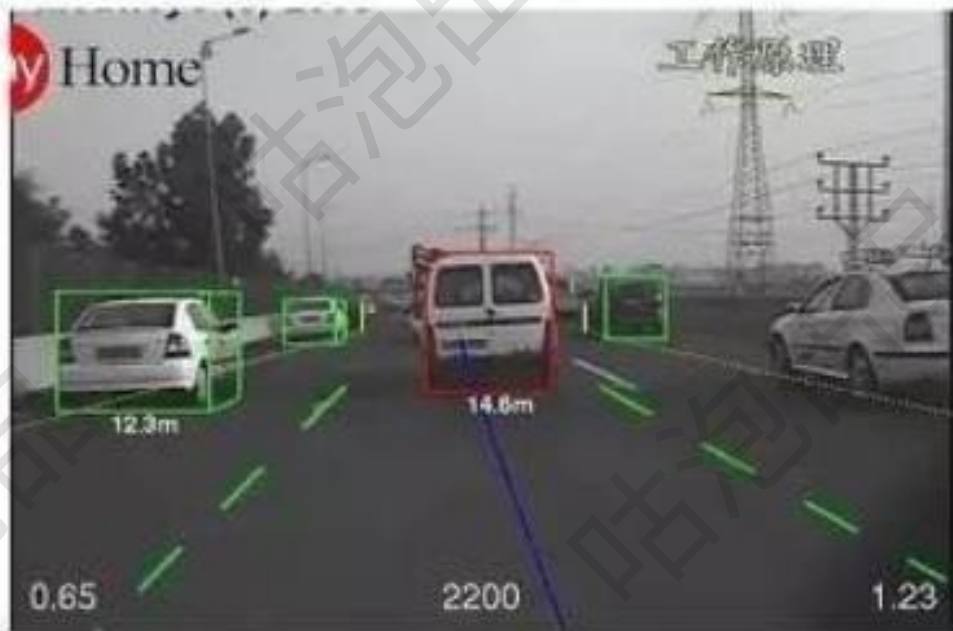
深度学习

✓ 为什么需要深度学习：



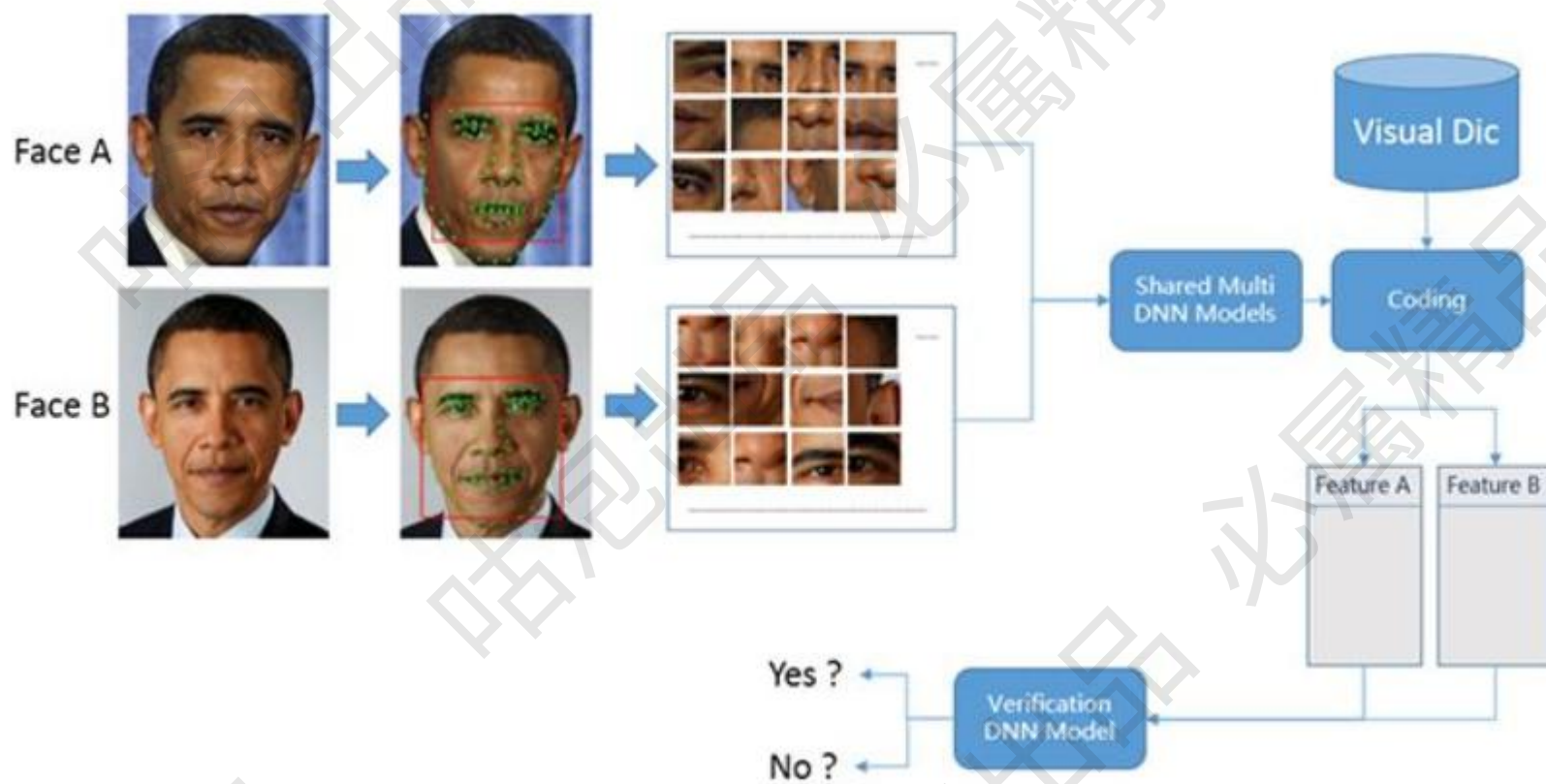
深度学习

✓ 深度学习应用：



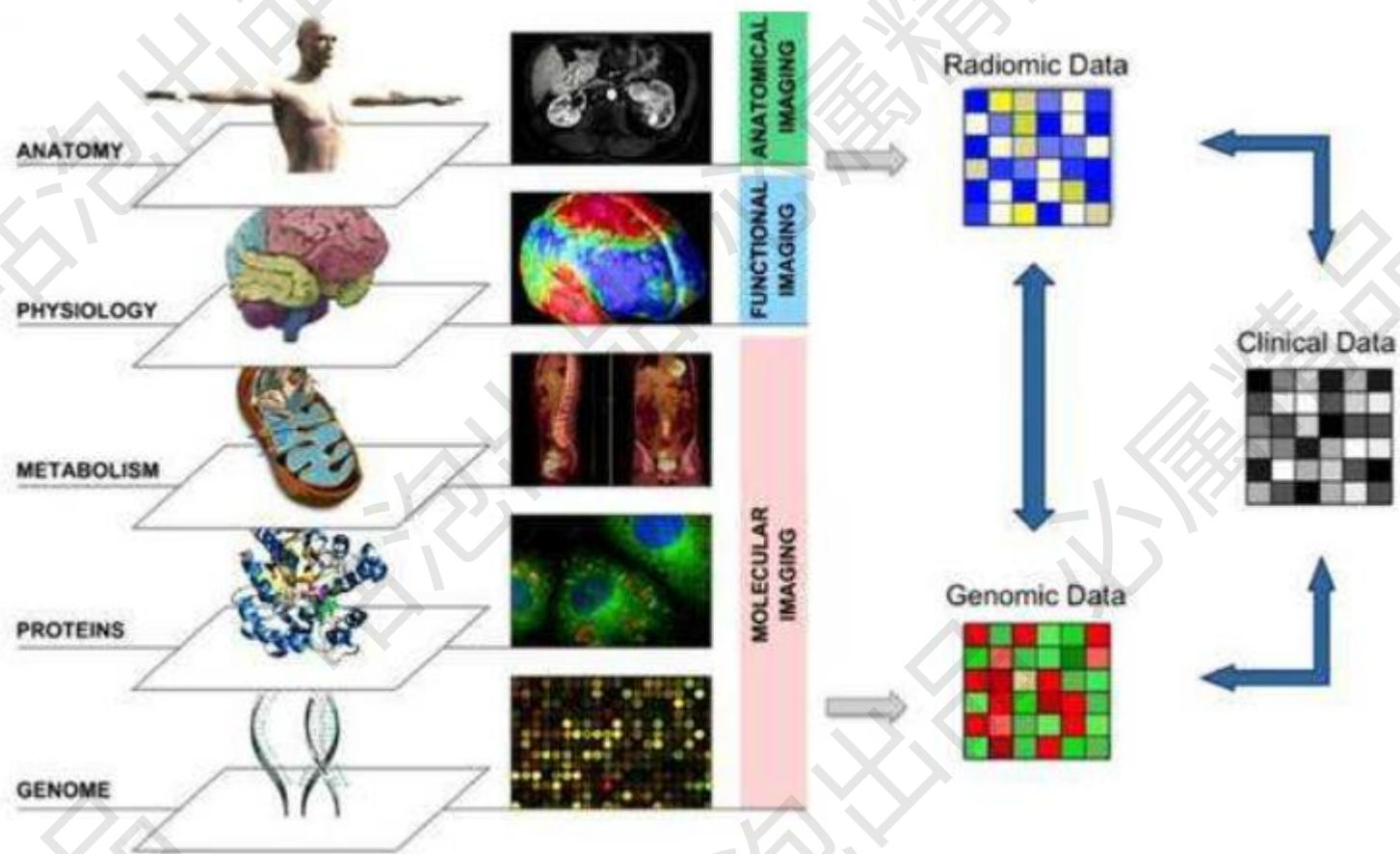
深度学习

✓ 深度学习应用：



深度学习

✓ 深度学习应用:



深度学习

✓ 深度学习应用：



深度学习

✓ 深度学习应用：

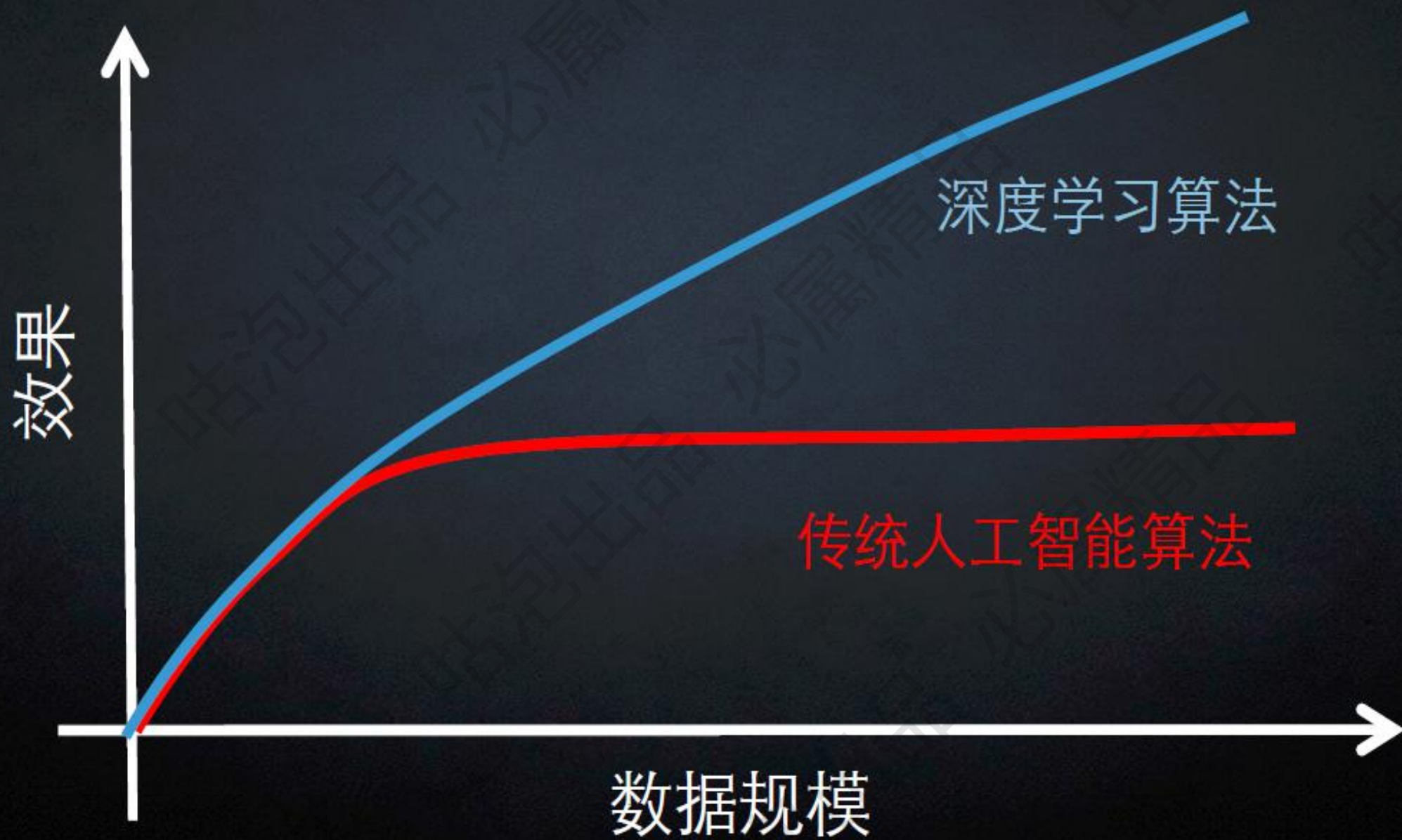




www.image-net.org

22K categories and **14M** images

- Animals
 - Bird
 - Fish
 - Mammal
 - Invertebrate
- Plants
 - Tree
 - Flower
- Food
- Materials
- Structures
 - Artifact
 - Tools
 - Appliances
 - Structures
- Person
 - Scenes
 - Indoor
 - Geological Formation
 - Sport Activities



图像分类

✓ 计算机视觉:

📎 图像分类任务



(假设我们有一系列的标签: 狗, 猫, 汽车, 飞机。。。)

————→ 猫

图像分类

✓ 计算机视觉:

✎ 图像表示: 计算机眼中的图像

✎ 一张图片被表示成三维数组的形式, 每个像素的值从0到255

例如: $300 \times 100 \times 3$



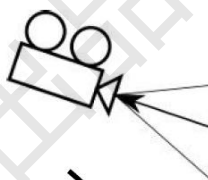
08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	88
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	42	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	55	88	30	03	49	13	36	65
52	70	95	23	04	60	11	42	69	27	68	56	01	32	56	71	37	02	36	91
22	31	16	71	51	63	05	89	41	92	36	54	22	40	40	28	66	33	13	80
24	47	15	00	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	94	39	63	08	40	91	66	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	05	03	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
15	16	88	67	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	69
04	42	16	73	35	05	39	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	05	89	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	48	84	81	16	23	57	05	54
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	87	13	67	48

What the computer sees

图像分类

✓ 计算机视觉面临的挑战:

✎ 照射角度:



18	02	22	97	38	15	00	40	00	75	04	05	07	78	82	32	55	77		
49	49	99	40	17	81	18	57	60	87	17	40	96	43	65	14	49	56	42	00
51	49	31	73	55	79	14	29	93	71	40	50	47	50	05	49	13	94	68	
52	70	95	23	04	60	11	42	68	74	10	54	05	32	34	71	37	02	96	91
22	31	16	71	51	00	09	41	92	36	54	22	40	40	28	66	35	55	85	
04	37	00	99	03	45	02	44	75	33	53	75	94	84	20	35	27	12	50	
44	95	01	39	44	23	47	10	26	37	60	47	59	54	70	64	18	39	64	70
67	26	20	68	02	42	12	20	95	68	94	39	63	08	40	91	64	49	94	21
24	55	58	05	46	73	99	24	97	37	75	78	94	83	18	89	34	89	43	72
21	34	23	09	70	00	76	44	20	49	33	14	00	41	33	97	24	31	33	95
78	17	53	28	22	75	51	47	15	94	05	80	04	62	14	14	09	53	56	92
16	39	05	42	94	35	11	47	55	58	88	24	00	17	34	24	34	29	85	57
86	56	00	48	35	71	88	07	05	44	94	37	49	60	21	58	51	54	17	58
19	80	81	48	05	94	47	69	25	75	92	13	84	52	17	77	04	89	55	40
04	52	08	93	97	35	99	14	07	97	57	32	14	24	24	79	33	27	98	64
74	42	07	57	62	20	72	03	46	33	47	44	55	12	32	43	93	53	49	
04	42	14	78	00	11	24	94	70	10	05	44	29	32	40	42	74	34		
20	69	96	41	72	59	28	57	71	48	32	49	82	67	59	85	74	04	34	14
20	73	35	29	78	31	80	51	74	35	49	73	87	24	11	16	23	57	05	54
01	70	04	71	03	51	54	49	14	92	35	48	41	43	52	01	87	94	56	41

✎ 形状改变:



图像分类

✓ 计算机视觉面临的挑战:

📎 部分遮蔽:



📎 背景混入:



图像分类

✓ 机器学习常规套路:

- 1. 收集数据并给定标签
- 2. 训练一个分类器
- 3. 测试, 评估

```
def train(train_images, train_labels):  
    # build a model for images -> labels...  
    return model  
  
def predict(model, test_images):  
    # predict test_labels using the model...  
    return test_labels
```

Example training set



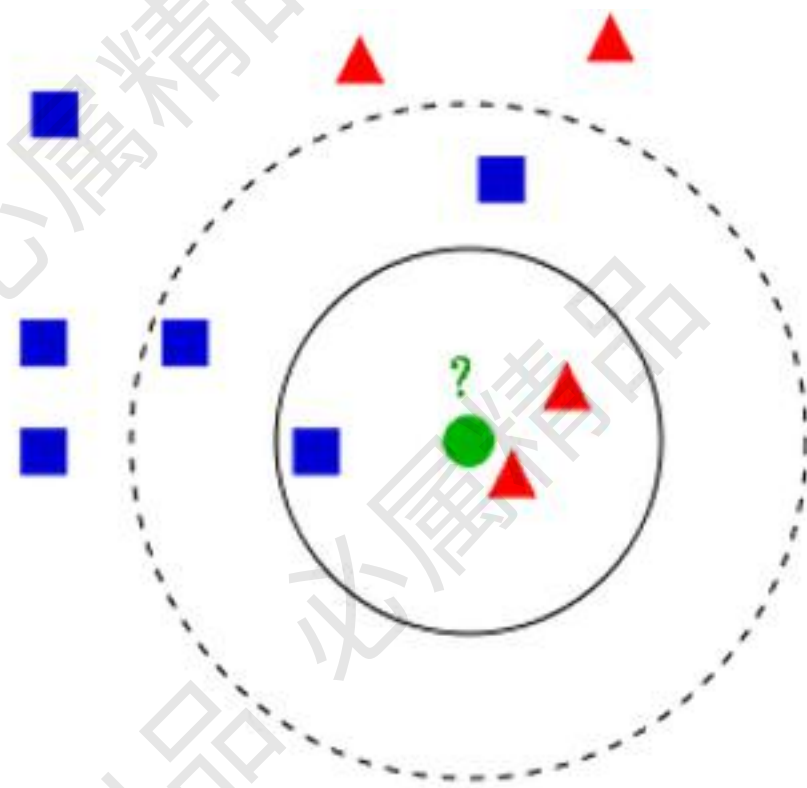
K近邻

✓ K近邻算法：

✎ 数据：两类点方块和三角

✎ 绿色的点属于方块还是三角呢？

✎ $K=3$ 还是 $K=5$ ？ 结果一样吗？



K近邻

✓ K近邻计算流程：

- ✎ 1. 计算已知类别数据集中的点与当前点的距离
2. 按照距离依次排序
3. 选取与当前点距离最小的K个点
4. 确定前K个点所在类别的出现概率
5. 返回前K个点出现频率最高的类别作为当前点预测分类

K近邻

✓ K近邻分析

✎ KNN 算法本身简单有效，它是一种 lazy-learning 算法。

✎ 分类器不需要使用训练集进行训练，训练时间复杂度为0。

✎ KNN 分类的计算复杂度和训练集中的文档数目成正比，也就是说，如果训练集中文档总数为 n ，那么 KNN 的分类时间复杂度为 $O(n)$ 。

✎ K 值的选择，距离度量和分类决策规则是该算法的三个基本要素

数据库样例: CIFAR-10

✓ 数据库简介:

📎 10类标签

50000个训练数据

10000个测试数据

大小均为 32×32

airplane



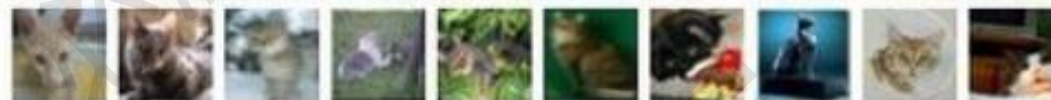
automobile



bird



cat



deer



dog



frog



horse



ship



truck



图像分类

✓ 距离的选择: **L1 distance:** $d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$

✎ 图像距离计算方式:

test image					training image					pixel-wise absolute value differences				
56	32	10	18		10	20	24	17		46	12	14	1	
90	23	128	133		8	10	89	100		82	13	39	33	
24	26	178	200	-	12	16	178	170	=	12	10	0	30	
2	0	255	220		4	32	233	112		2	32	22	108	add → 456

图像分类:

✓ 测试结果:

✎ 部分结果还可以的

✎ 没有分类对的图像,

问题出在哪里呢?



图像分类：

✓ 参数会对结果有影吗？

✎ 距离的定义： L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

✎ 对于K近邻的K该如何选择？

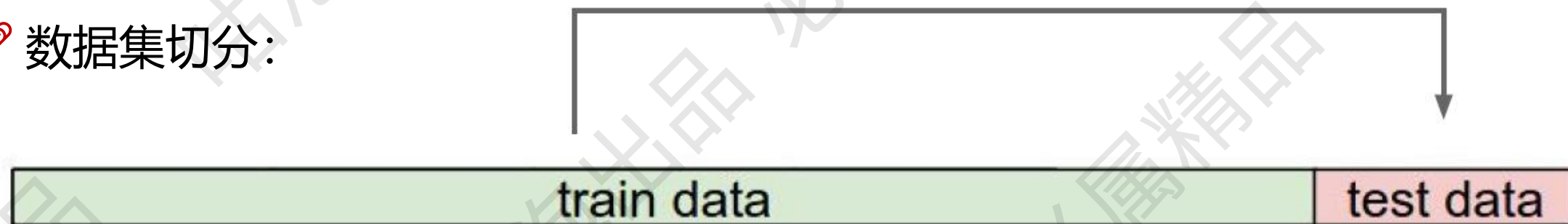
✎ 如果有的话，其它的超参数该怎么设定呢？

图像分类:

✓ 选择最好的参数

✎ 交叉验证: 实践来检验真理

✎ 数据集切分:

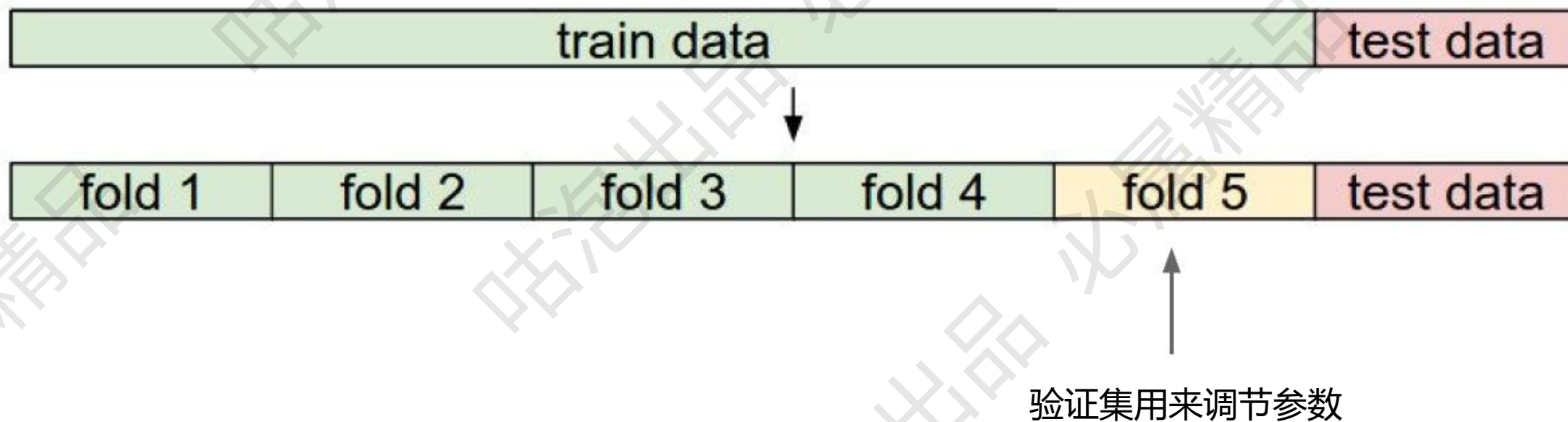


✎ 用测试集来选择最好的参数吗?

图像分类:

✓ 选择最好的参数

✎ 交叉验证: 得到更可靠的验证结果

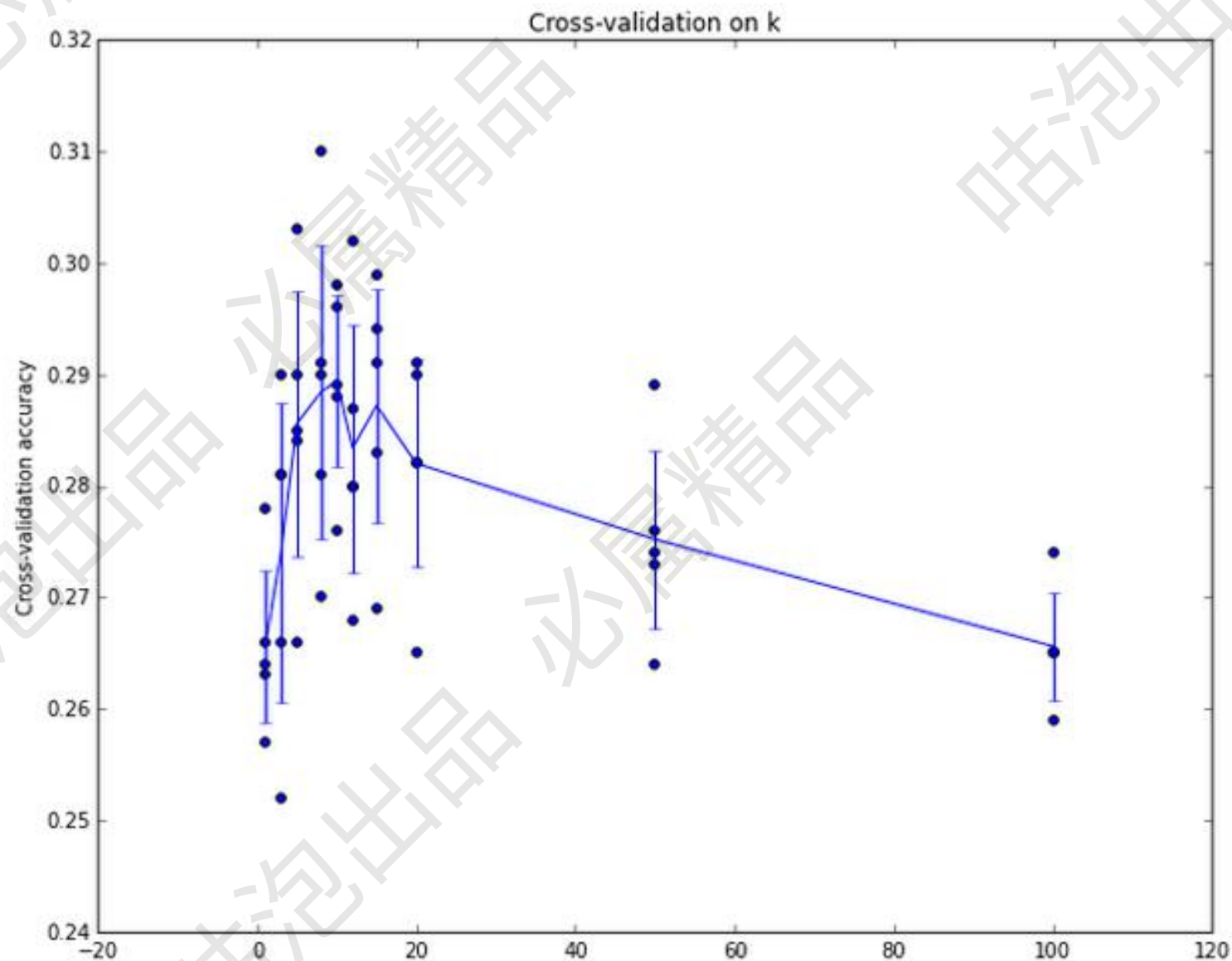


图像分类：

✓ K近邻交叉验证结果

✎ 参数对结果的影响有多大？

✎ 交叉验证是最合适的参数选择方法，比经验值靠谱的多！



图像分类：

✓ 为什么K近邻不能用来图像分类？

✎ 背景主导是一个最大的问题，我们关注的却是主体（主要成分）

✎ 如何才能让机器学习到哪些是重要的成分呢？



神经网络基础

✓ 线性函数

✎ 从输入-->输出的映射



(32x32x3)

image parameters

$f(\mathbf{x}, \mathbf{W})$

→ 每个类别的得分

神经网络基础

✓ 线性函数

📎 数学表示



[32x32x3]

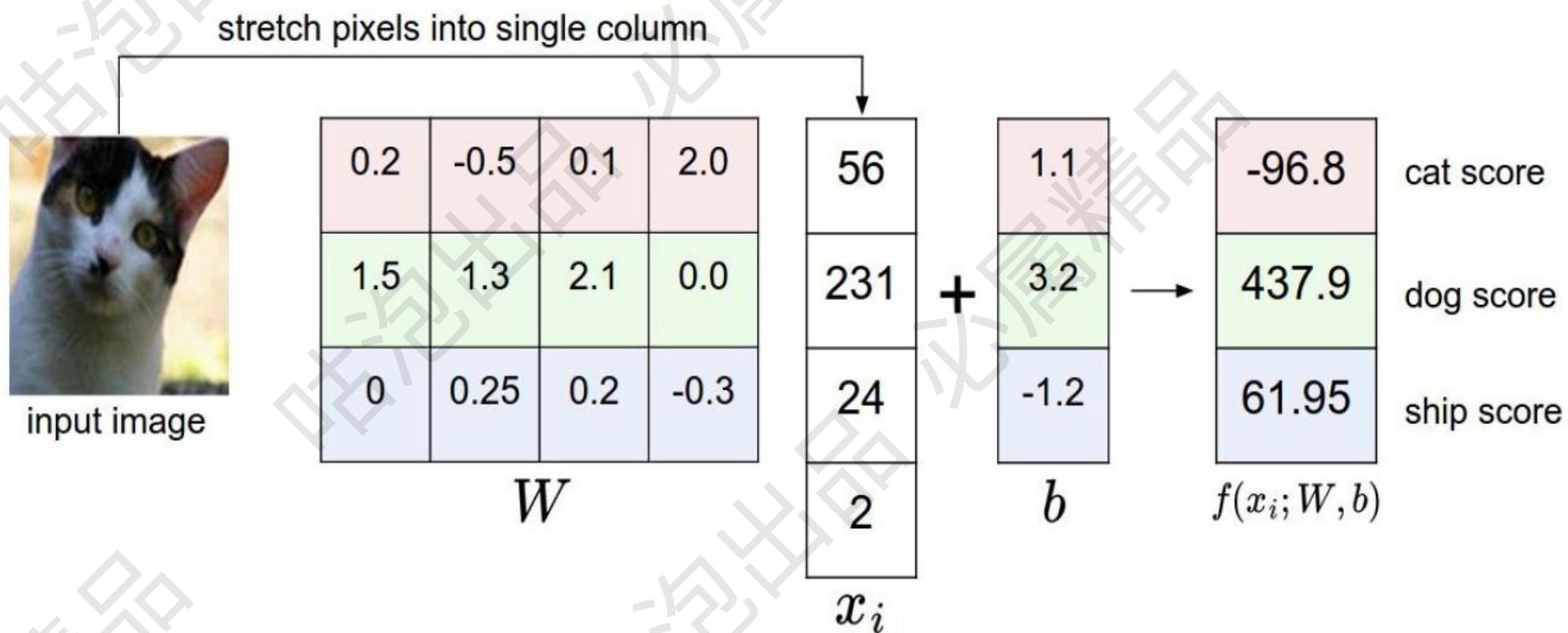
$$\boxed{f(x, W)}_{10 \times 1} = \boxed{W}_{10 \times 3072} \boxed{x}_{3072 \times 1} \boxed{(+b)}_{10 \times 1}$$

→ 每个类别的得分

神经网络基础

✓ 线性函数

✎ 计算方法

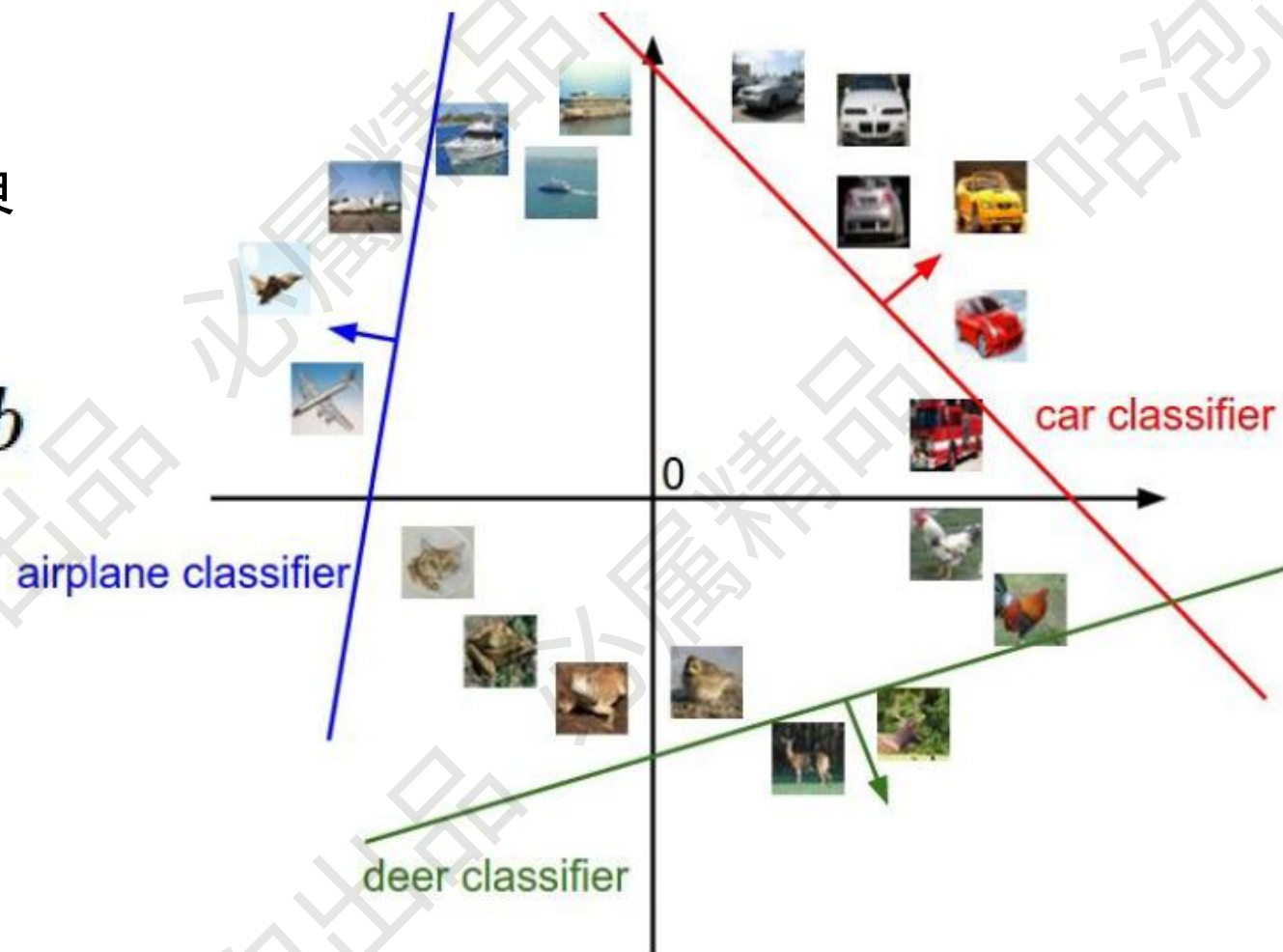


神经网络基础

✓ 线性函数

✎ 多组权重参数构成了决策边界

$$f(x_i, W, b) = Wx_i + b$$



神经网络基础

✓ 损失函数

✎ 如何衡量分类的结果呢？

✎ 结果的得分值有着明显的差异，
我们需要明确的指导模型的当前
效果，有多好或是多差！



cat

3.2

1.3

2.2

car

5.1

4.9

2.5

frog

-1.7

2.0

-3.1

神经网络基础

✓ 损失函数 $L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$

✎ 损失函数其实有很多种，我们来实验一个



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

$$\begin{aligned} &= \max(0, 5.1 - 3.2 + 1) \\ &\quad + \max(0, -1.7 - 3.2 + 1) \\ &= \max(0, 2.9) + \max(0, -3.9) \\ &= 2.9 + 0 \\ &= 2.9 \end{aligned}$$

$$\begin{aligned} &= \max(0, 1.3 - 4.9 + 1) \\ &\quad + \max(0, 2.0 - 4.9 + 1) \\ &= \max(0, -2.6) + \max(0, -1.9) \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

$$\begin{aligned} &= \max(0, 2.2 - (-3.1) + 1) \\ &\quad + \max(0, 2.5 - (-3.1) + 1) \\ &= \max(0, 5.3) + \max(0, 5.6) \\ &= 5.3 + 5.6 \\ &= 10.9 \end{aligned}$$

神经网络基础

✓ 损失函数

✎ 如何损失函数的值相同，那么意味着两个模型一样吗？

$$f(x, W) = Wx$$

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1)$$

✎ 输入数据: $x = [1, 1, 1, 1]$

模型A:

$$w_1 = [1, 0, 0, 0]$$

模型B:

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

神经网络基础

✓ 损失函数

✎ 损失函数 = 数据损失 + 正则化惩罚项

正则化惩罚项

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \lambda R(W)$$

✎ 正则化惩罚项: $R(W) = \sum_k \sum_l W_{k,l}^2$

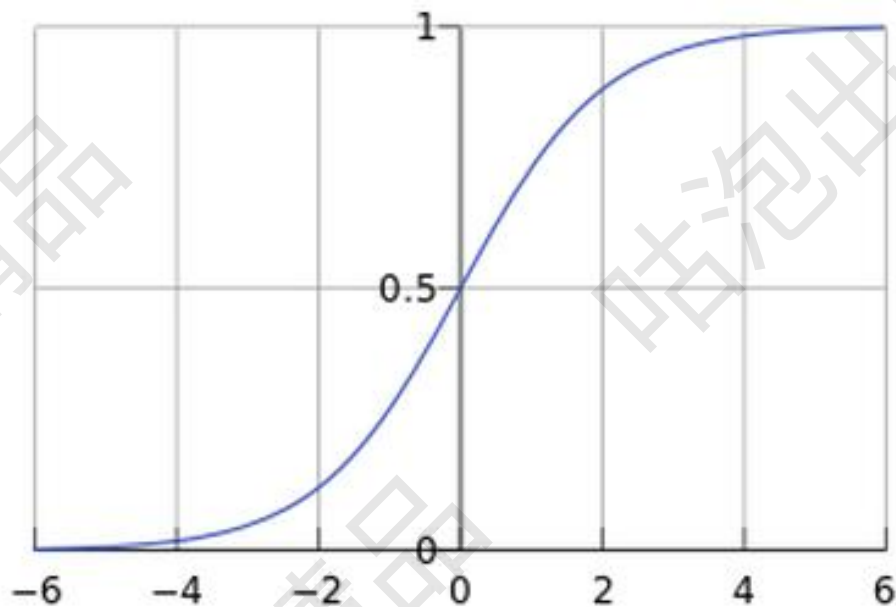
(我们总是希望模型不要太复杂, 过拟合的模型是没用的)

神经网络基础

✓ Softmax分类器

✎ 现在我们得到的是一个输入的得分值，但如果给我一个概率值岂不更好！

✎ 如何把一个得分值转换成一个概率值呢？



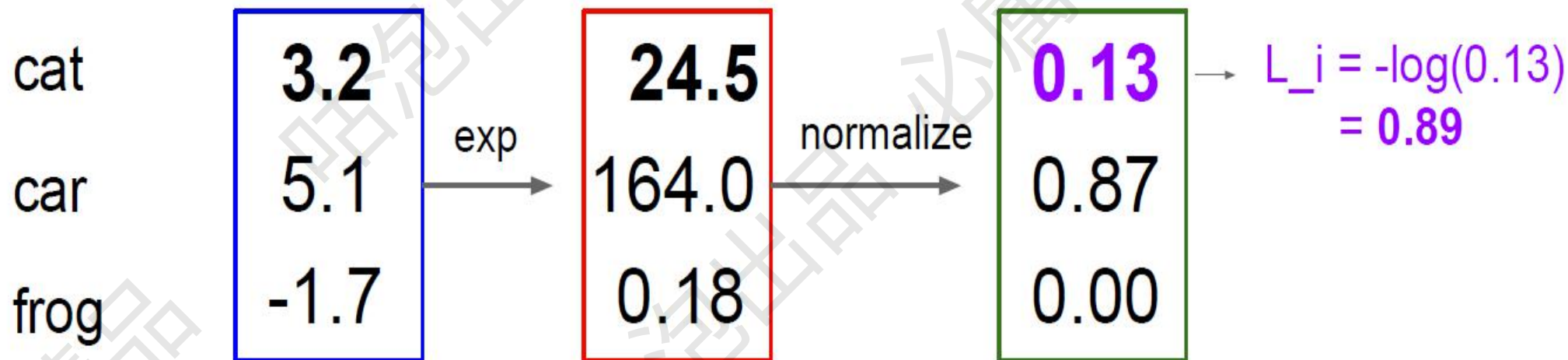
$$g(z) = \frac{1}{1 + e^{-z}}$$

神经网络基础

✓ Softmax分类器

✎ 归一化: $P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$ where $s = f(x_i; W)$

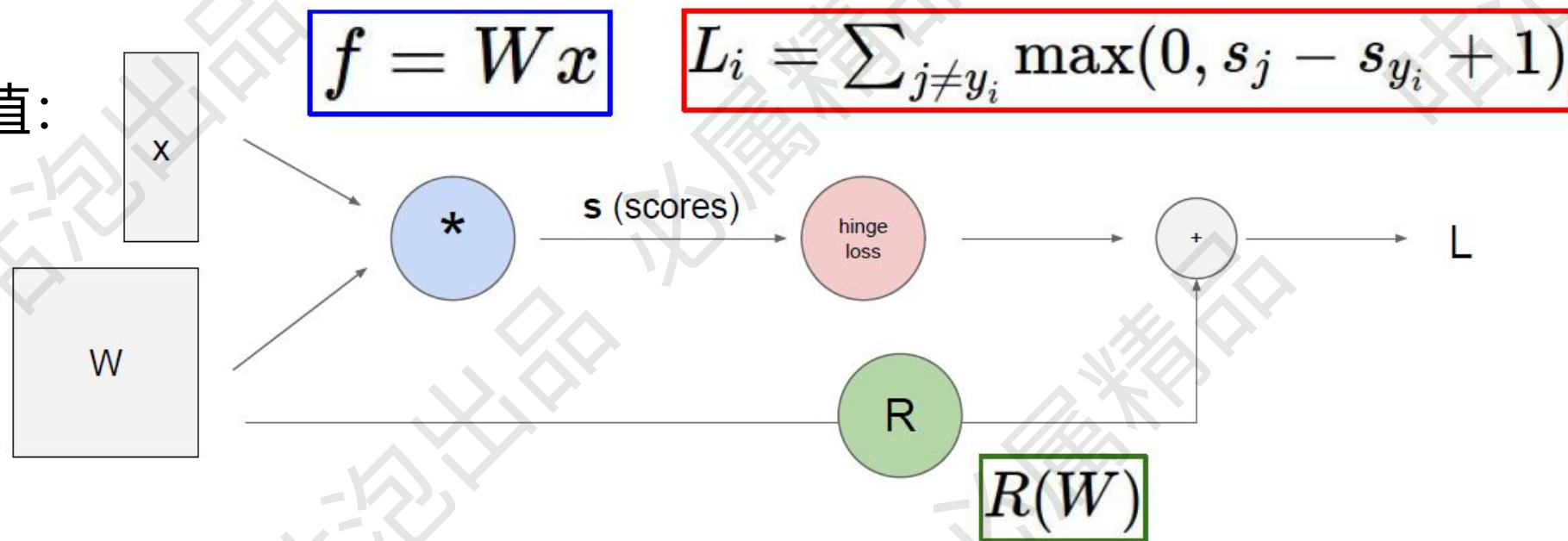
✎ 计算损失值: $L_i = -\log P(Y = y_i|X = x_i)$



神经网络基础

✓ 前向传播

✎ 得出损失值:



✎ 如何更新模型呢? 这个就交给反向传播了 (梯度下降)

线性回归

✓ 梯度下降

✎ 引入：当我们得到了一个目标函数后，如何进行求解？
直接求解？（并不一定可解，线性回归可以当做是一个特例）

✎ 常规套路：机器学习的套路就是我交给机器一堆数据，然后告诉它什么样的学习方式是对的（目标函数），然后让它朝着这个方向去做

✎ 如何优化：一口吃不成个胖子，我们要静悄悄的一步步的完成迭代
（每次优化一点点，累积起来就是个大成绩了）

线性回归

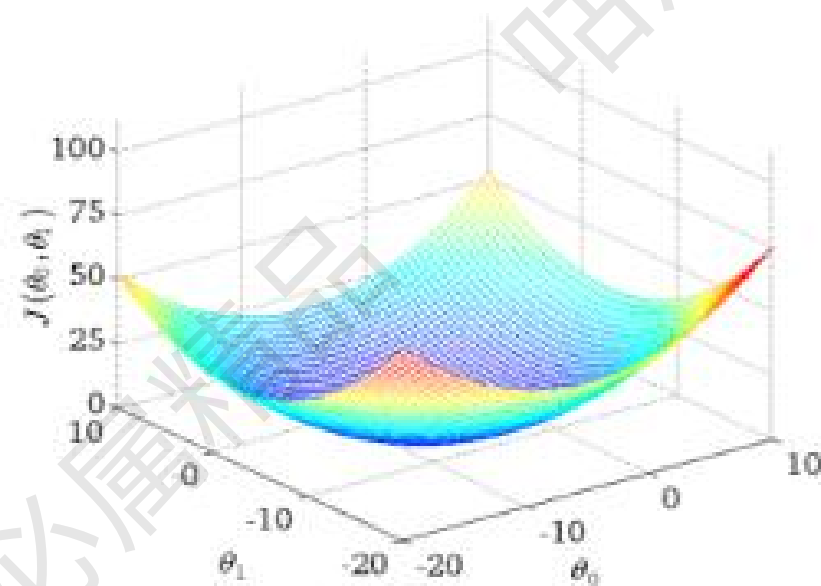
✓ 梯度下降

📌 目标函数: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

📌 寻找山谷的最低点，也就是我们的目标函数终点
(什么样的参数能使得目标函数达到极值点)

📌 下山分几步走呢？（更新参数）

- (1) : 找到当前最合适的方向
- (2) : 走那么一小步，走快了该“跌倒”了
- (3) : 按照方向与步伐去更新我们的参数



线性回归

✓ 梯度下降，目标函数： $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (y^i - h_{\theta}(x^i))^2$

✎ 批量梯度下降： $\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m (y^i - h_{\theta}(x^i))x_j^i$ $\theta_j' = \theta_j + \frac{1}{m} \sum_{i=1}^m (y^i - h_{\theta}(x^i))x_j^i$

(容易得到最优解，但是由于每次考虑所有样本，速度很慢)

✎ 随机梯度下降： $\theta_j' = \theta_j + (y^i - h_{\theta}(x^i))x_j^i$

(每次找一个样本，迭代速度快，但不一定每次都朝着收敛的方向)

✎ 小批量梯度下降法： $\theta_j := \theta_j - \alpha \frac{1}{10} \sum_{k=i}^{i+9} (h_{\theta}(x^{(k)}) - y^{(k)})x_j^{(k)}$

(每次更新选择一小部分数据来算，实用！)

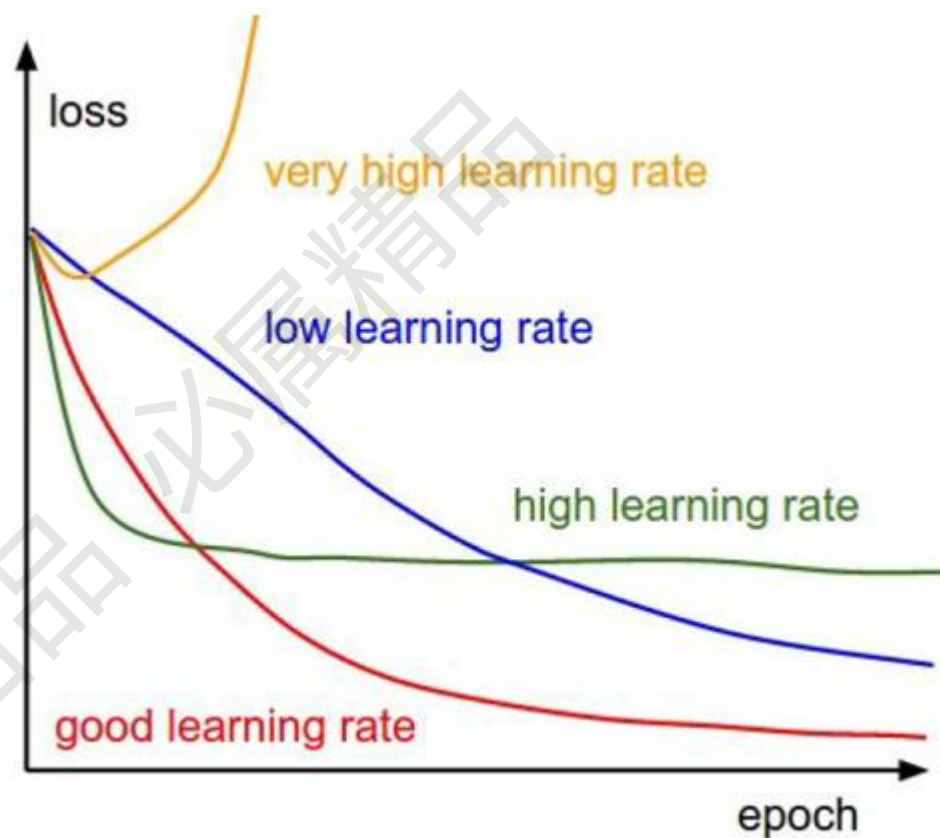
线性回归

✓ 梯度下降

✎ 学习率（步长）：对结果会产生巨大的影响，一般小一些

✎ 如何选择：从小的时候，不行再小

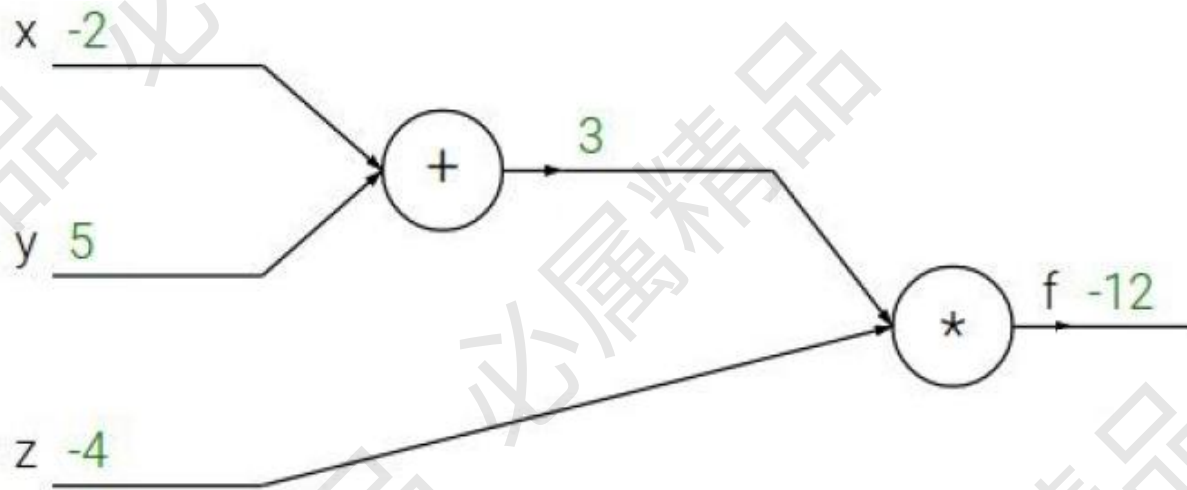
✎ 批处理数量：32, 64, 128都可以，很多时候还得考虑内存和效率



神经网络基础

✓ 反向传播

📎 简单的栗子：



$$f(x, y, z) = (x + y)z$$

$$\text{e.g. } x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

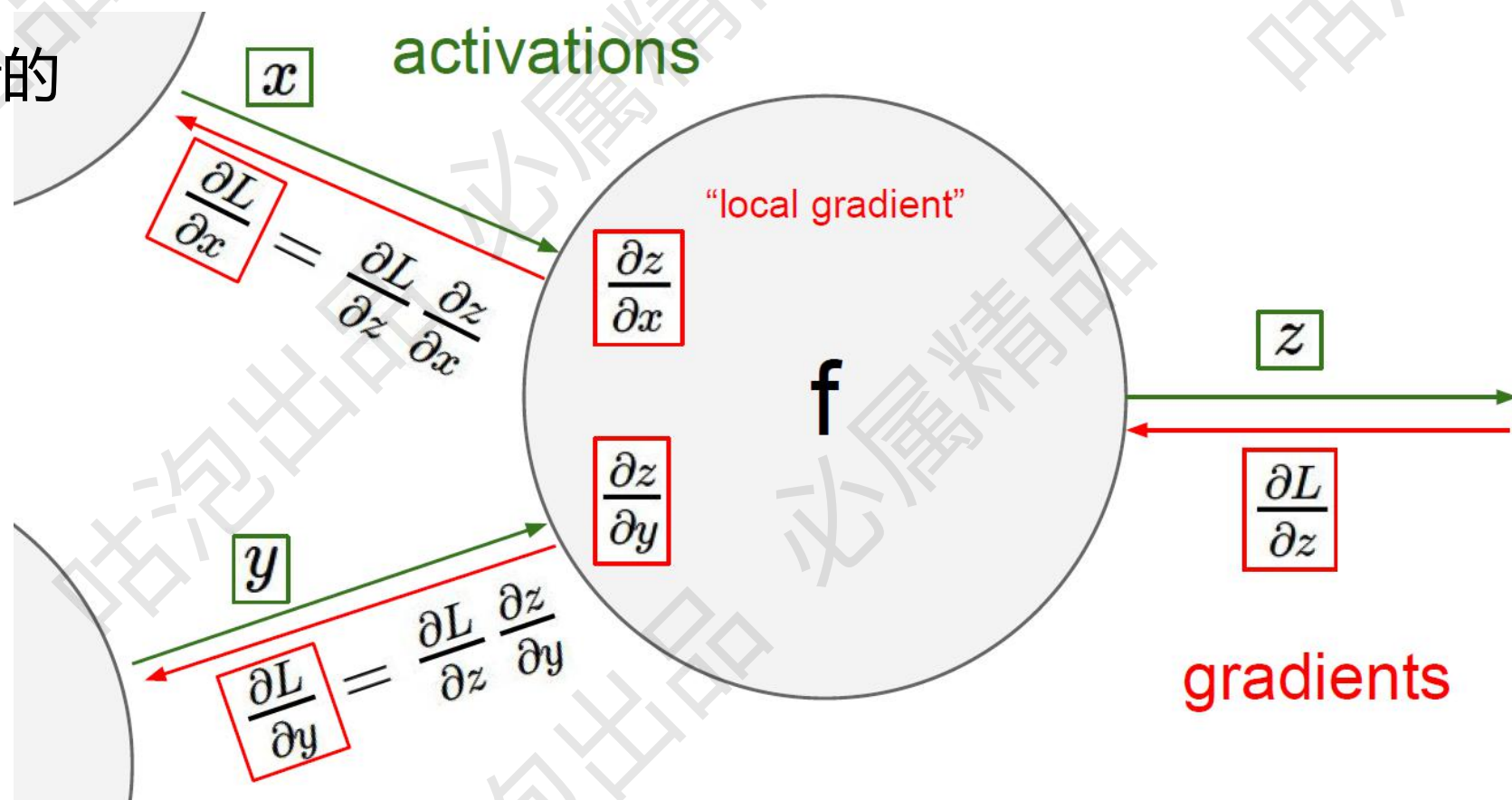
$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

神经网络基础

✓ 链式法则

✎ 梯度是一步一步传的



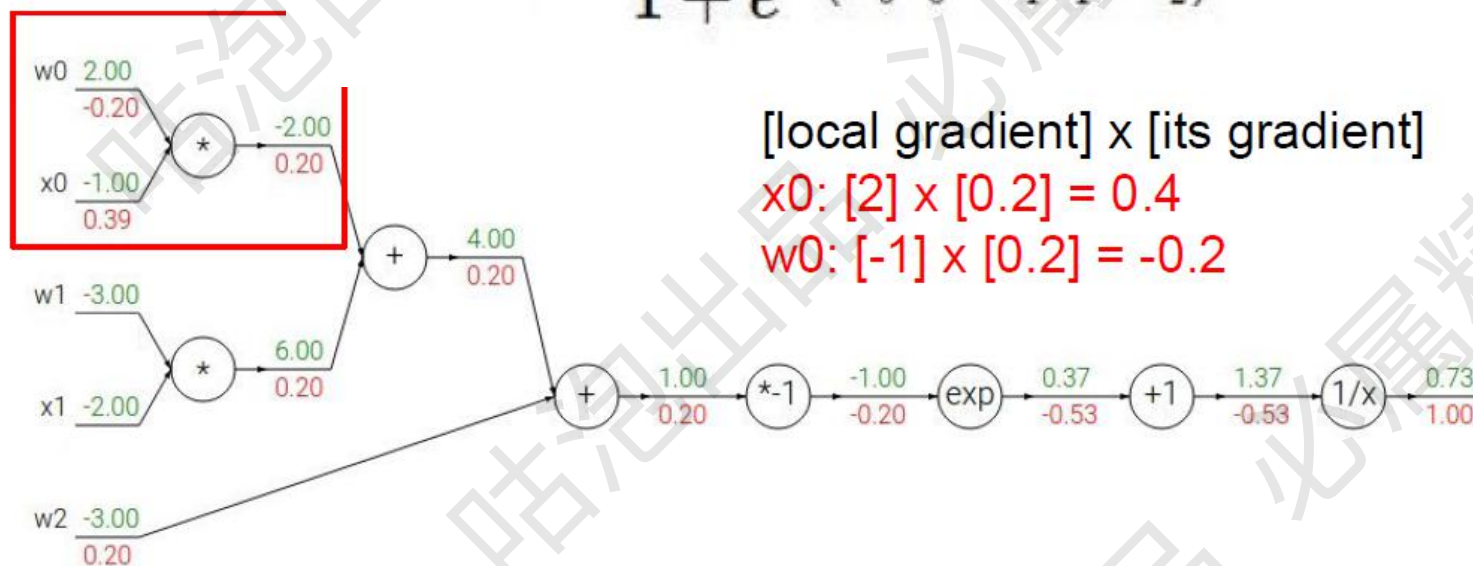
神经网络基础

✓ 反向传播



复杂的栗子：

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x$$

→

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

→

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

→

$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

→

$$\frac{df}{dx} = 1$$

神经网络基础

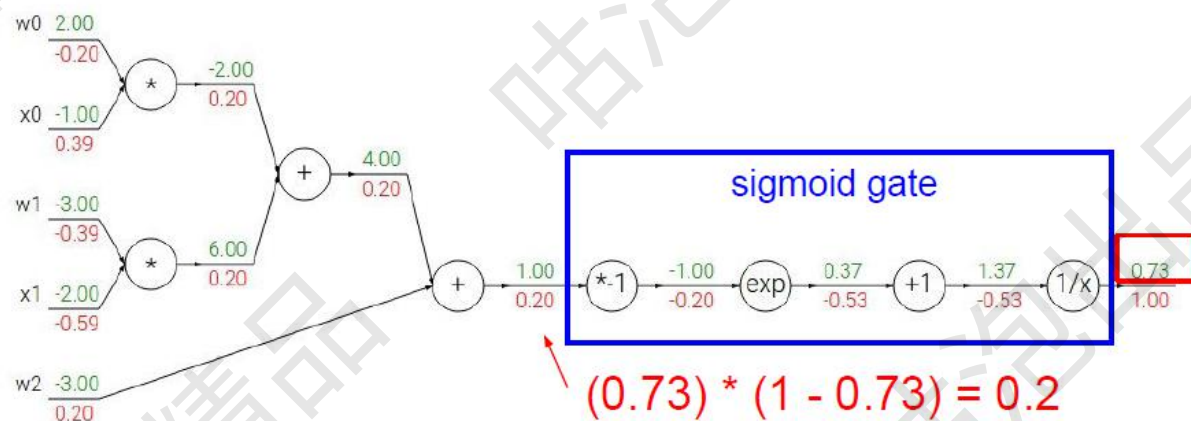
✓ 反向传播

✎ 可以一大块一大块的计算吗？

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{sigmoid function}$$

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$



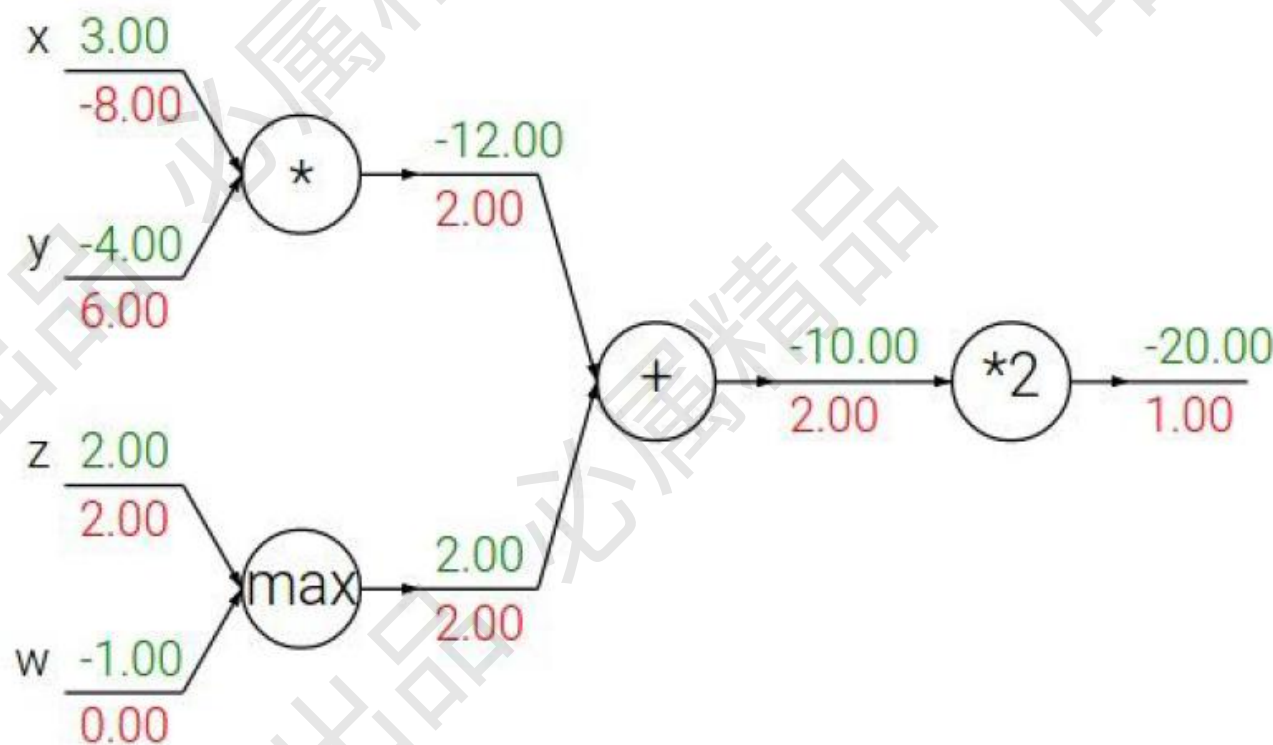
神经网络基础

✓ 反向传播

✎ 加法门单元：均等分配

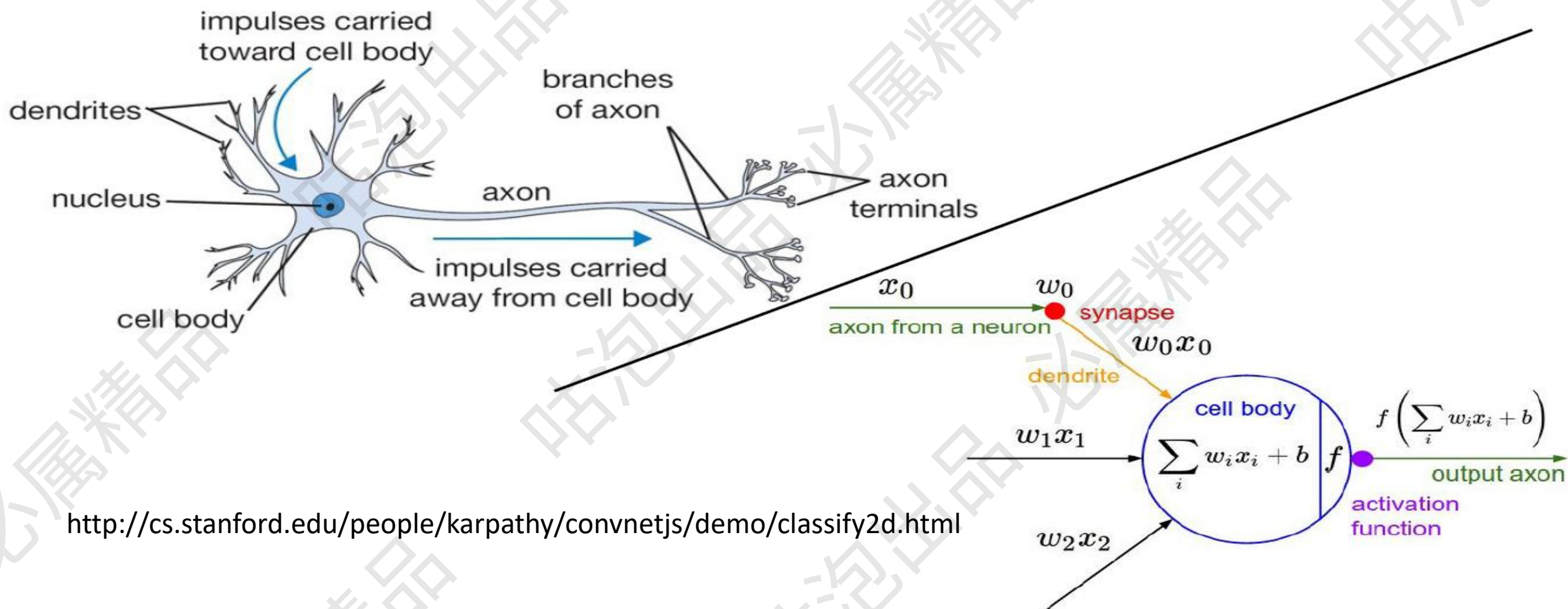
MAX门单元：给最大的

乘法门单元：互换的感觉



神经网络

✓ 整体架构



神经网络

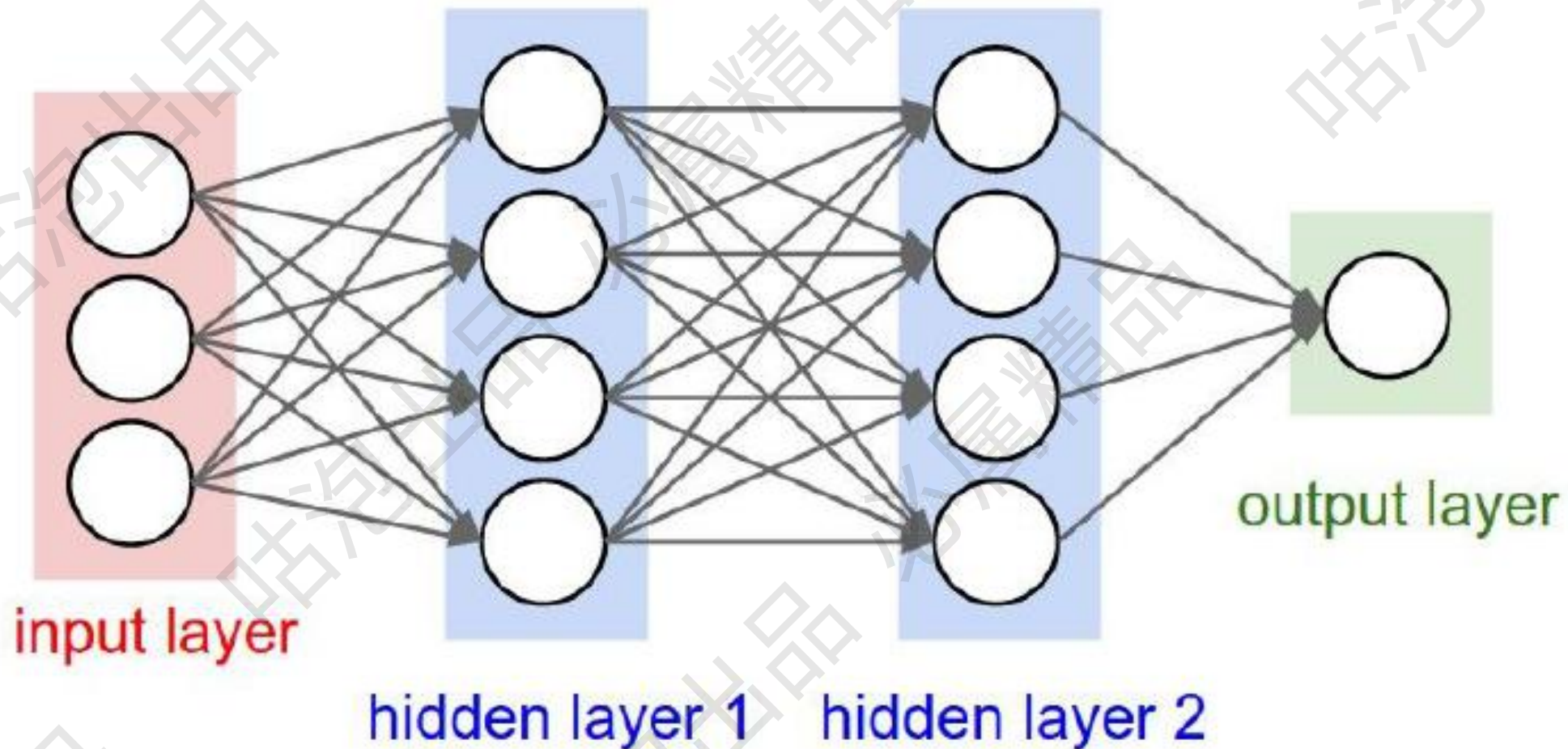
✓ 整体架构

📎 层次结构

📎 神经元

📎 全连接

📎 非线性



神经网络

✓ 整体架构

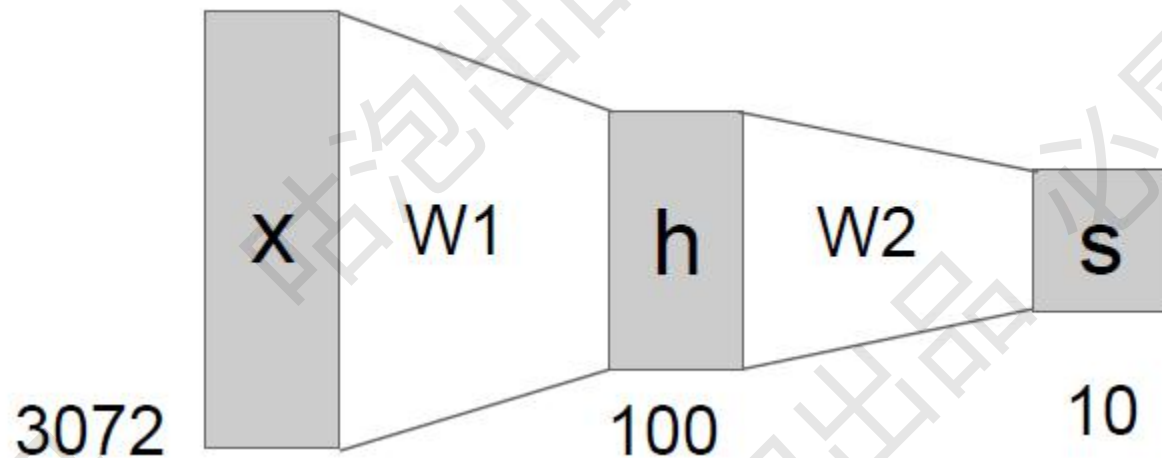
✎ 线性方程：

$$f = Wx$$

✎ 非线性方程：

$$f = W_2 \max(0, W_1 x)$$

✎ 计算结果：



神经网络

✓ 整体架构

✎ 基本机构:

$$f = W_2 \max(0, W_1 x)$$

✎ 继续堆叠一层:

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

✎ 神经网络的强大之处在于, 用更多的参数来拟合复杂的数据

(参数多到多少呢? 百万级别都是小儿科了, 但是参数越多越好吗?)

神经网络

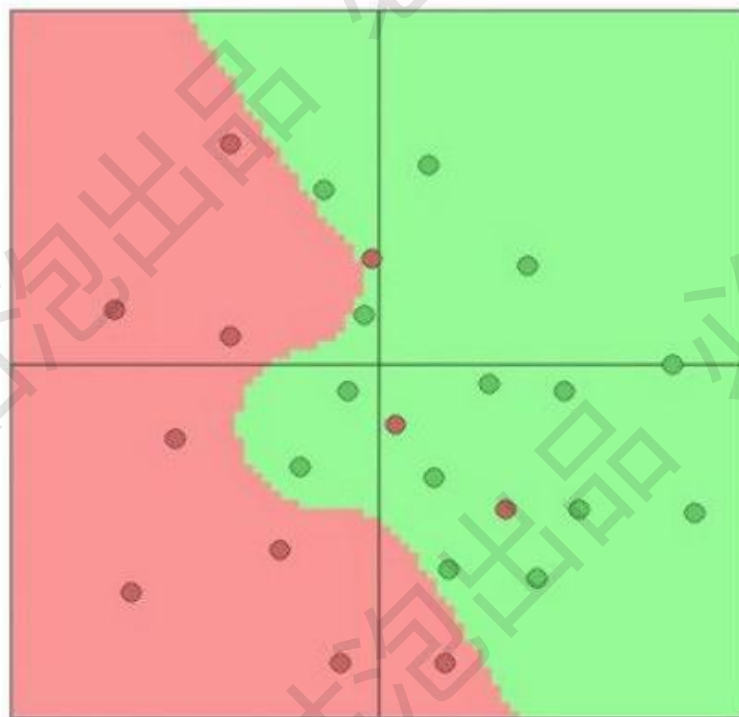
✓ 正则化的作用

📎 惩罚力度对结果的影响：

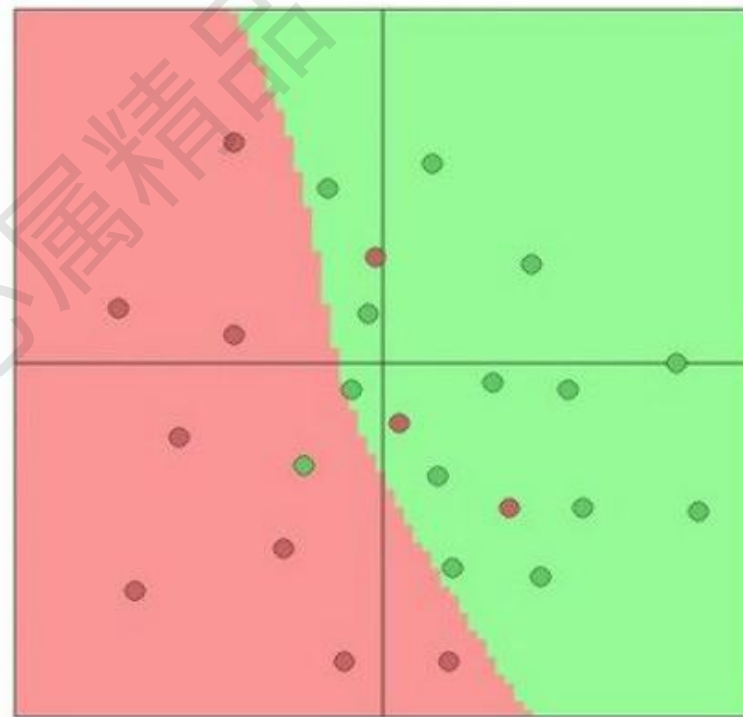
$\lambda = 0.001$



$\lambda = 0.01$



$\lambda = 0.1$

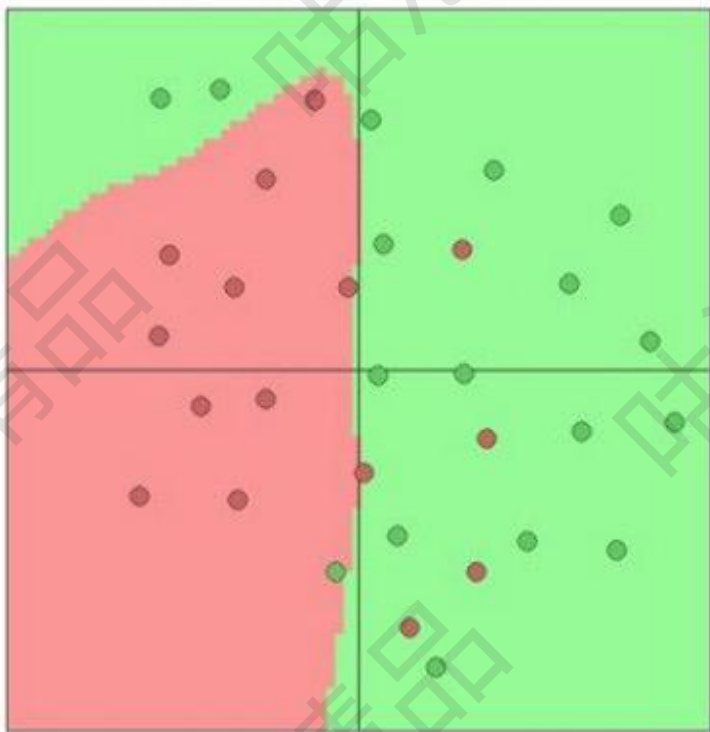


神经网络

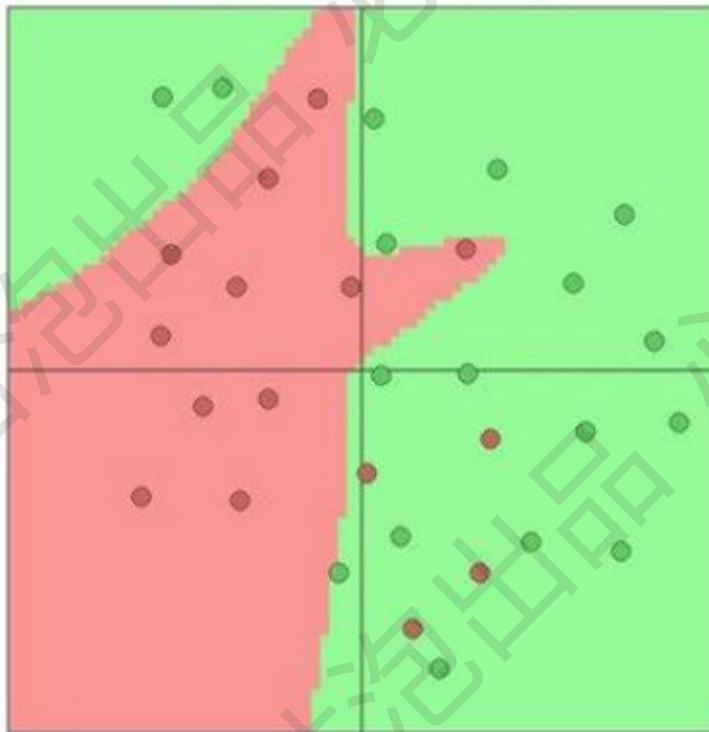
✓ 神经元

✎ 参数个数对结果的影响：

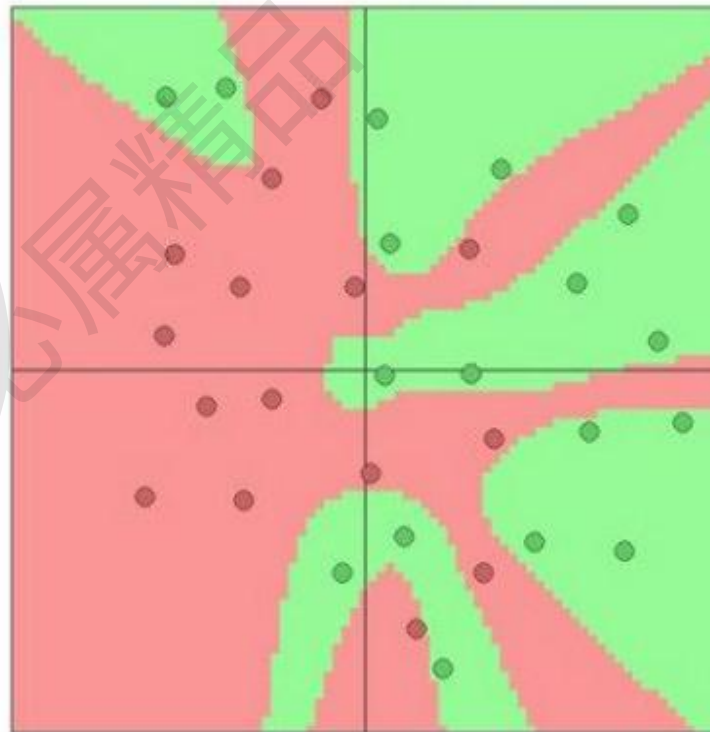
3 hidden neurons



6 hidden neurons



20 hidden neurons



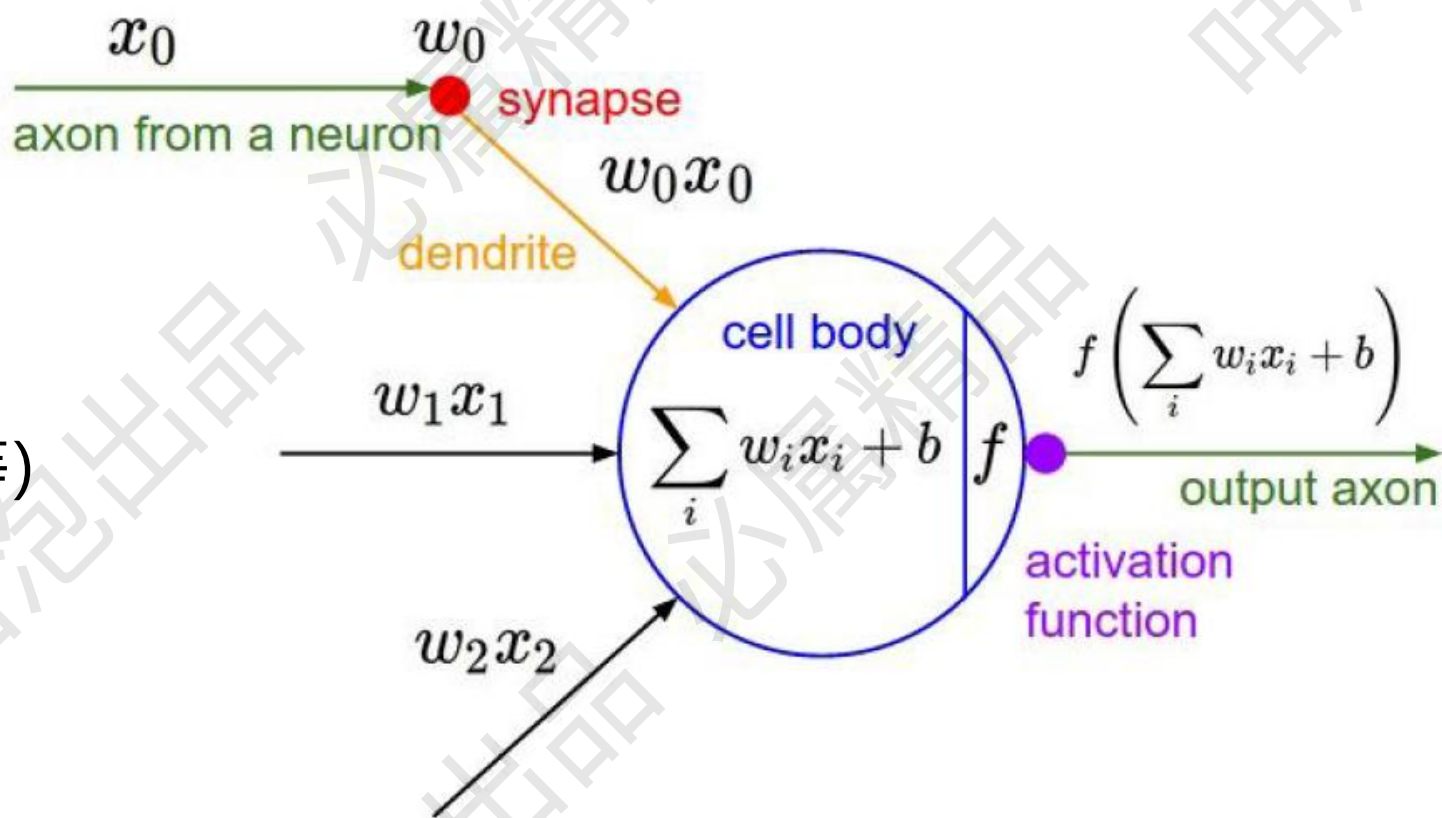
神经网络

✓ 激活函数

✎ 非常重要的一部分

✎ 常用的激活函数

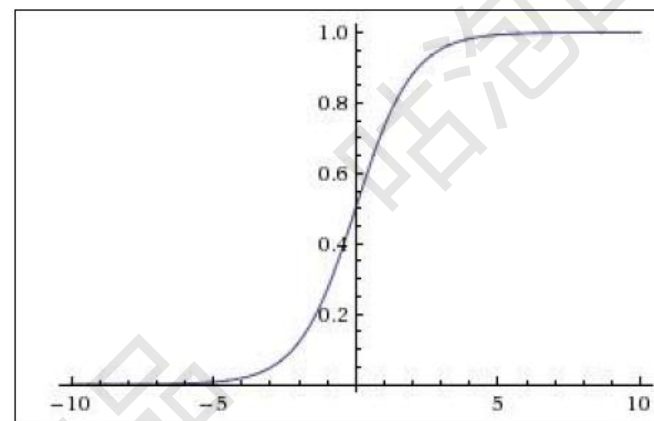
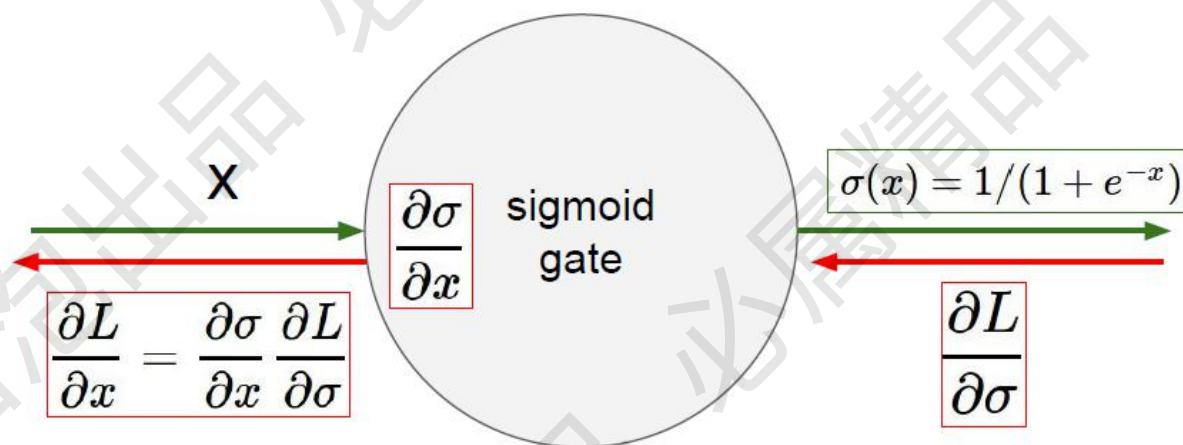
(Sigmoid, Relu, Tanh 等)



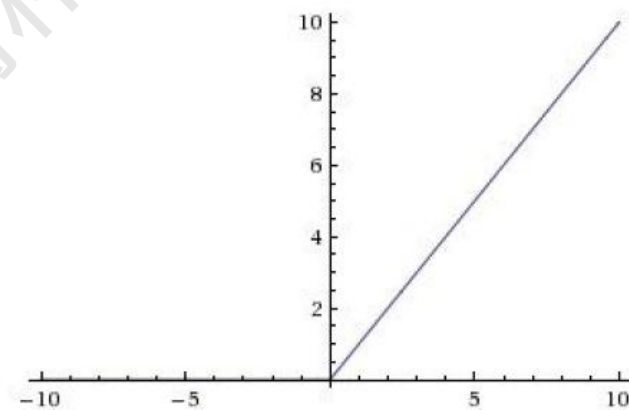
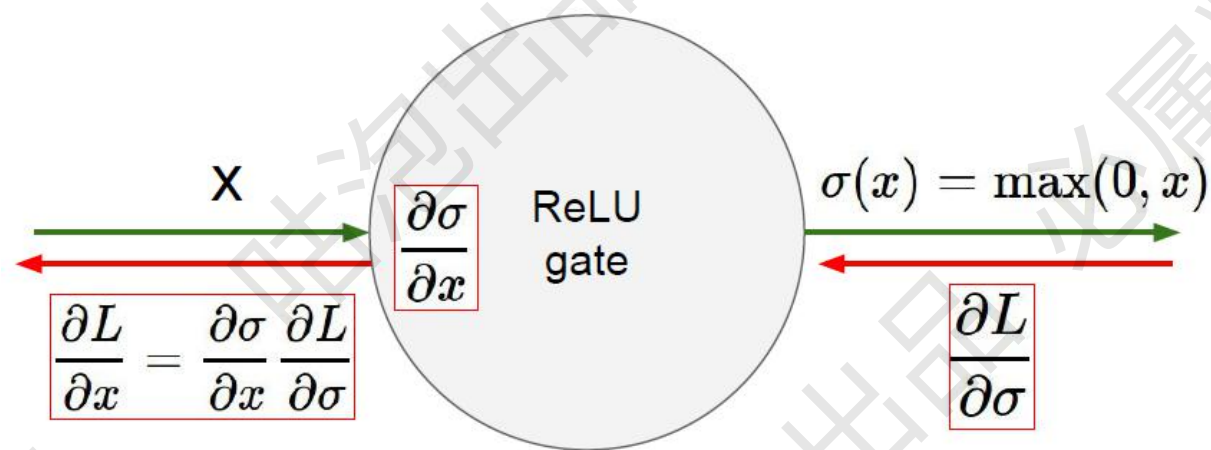
神经网络

✓ 激活函数对比

✎ Sigmoid:



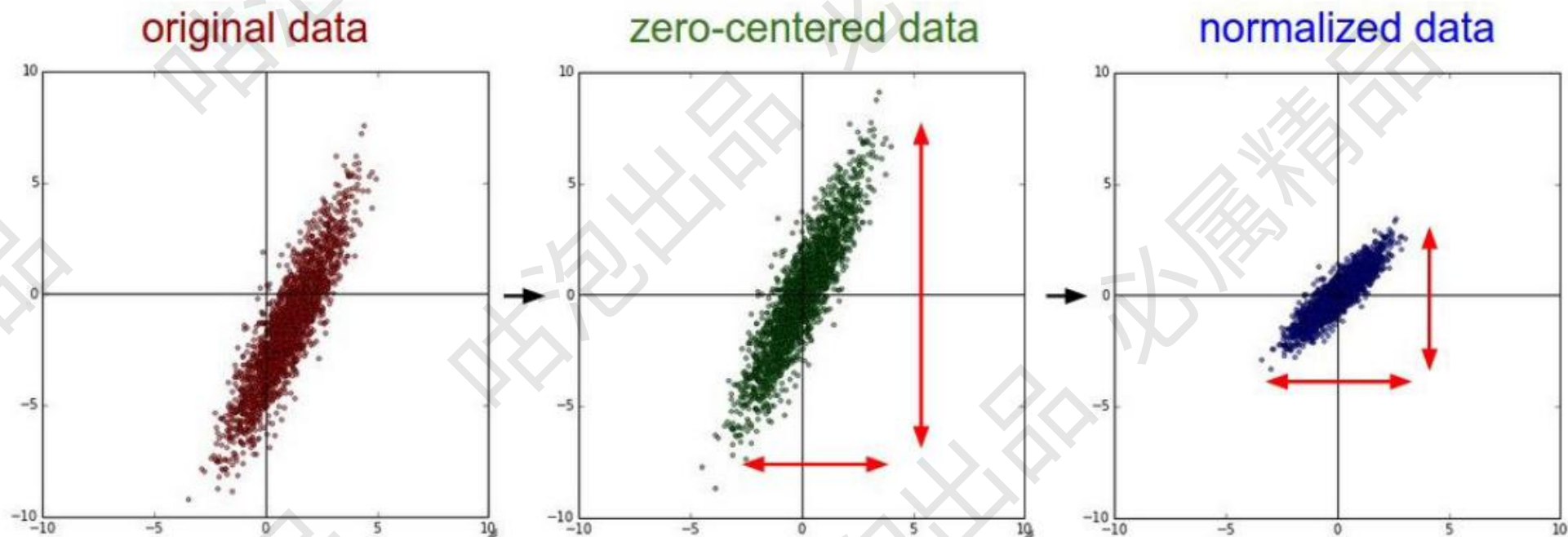
✎ Relu:



神经网络

✓ 数据预处理

✎ 不同的预处理结果会使得模型的效果发生很大的差异!



```
X -= np.mean(X, axis = 0)
```

```
X /= np.std(X, axis = 0)
```

神经网络

✓ 参数初始化

✎ 参数初始化同样非常重要！

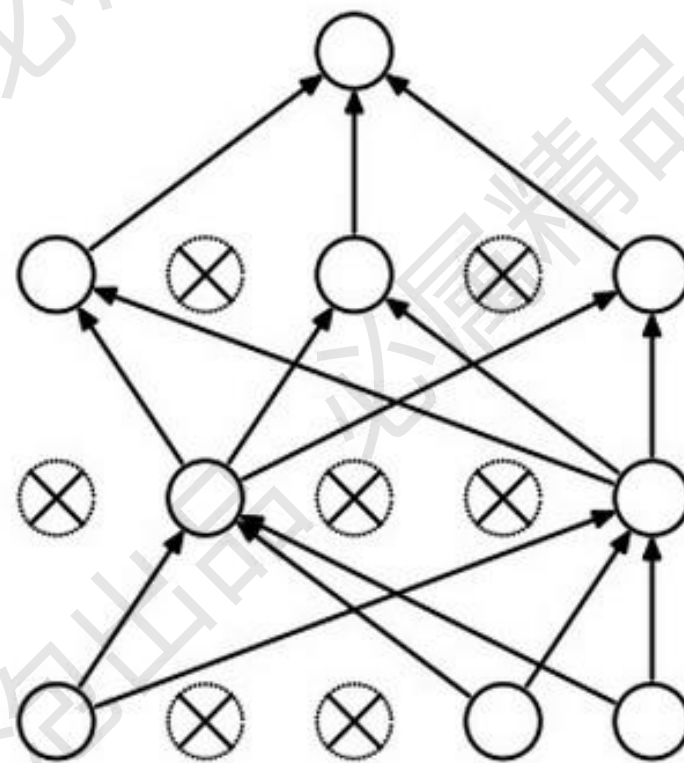
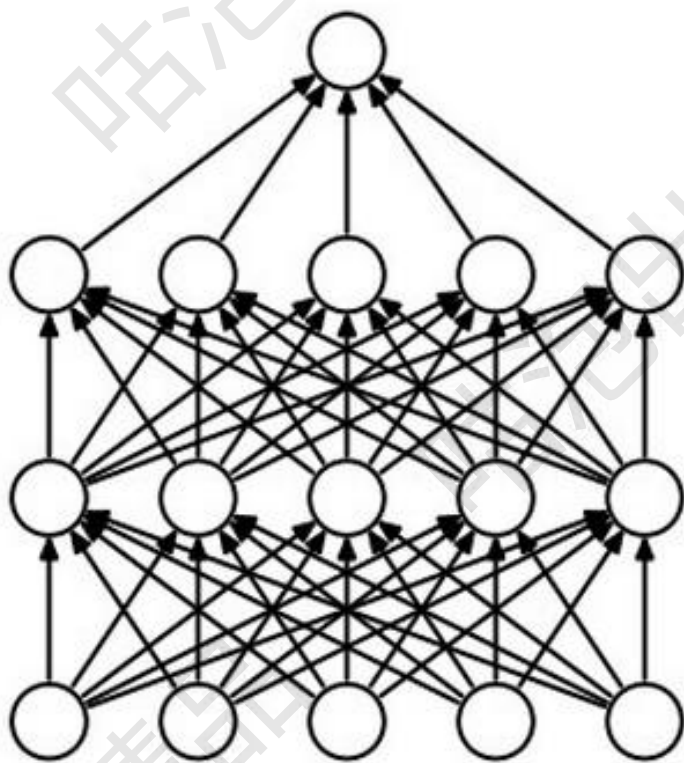
✎ 通常我们都使用随机策略来进行参数初始化

```
W = 0.01 * np.random.randn(D, H)
```

神经网络

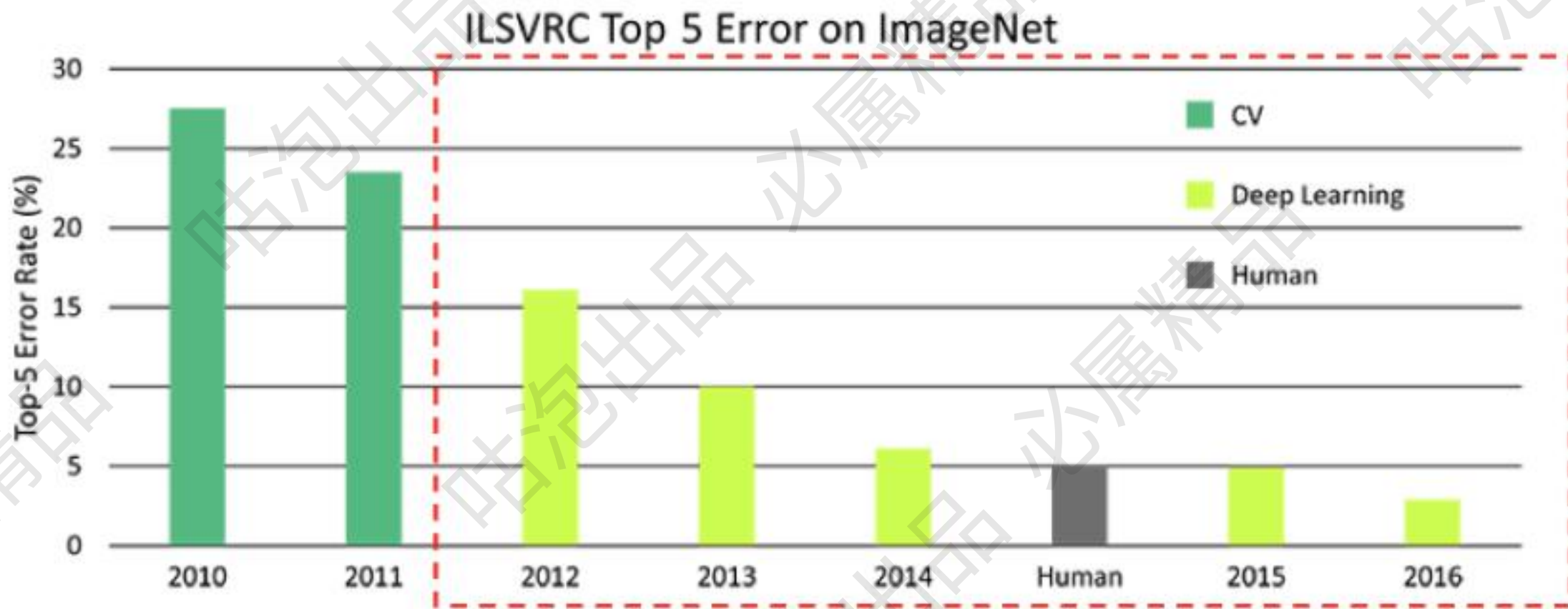
✓ DROP-OUT (传说中的七伤拳)

✎ 过拟合是神经网络非常头疼的一个大问题!



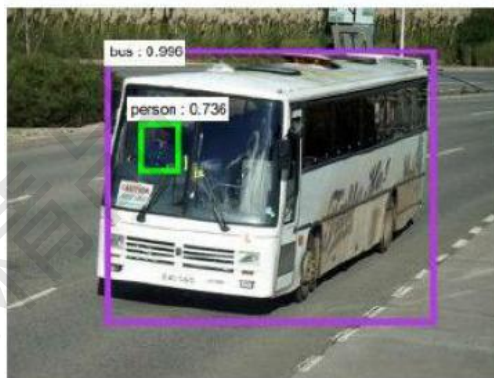
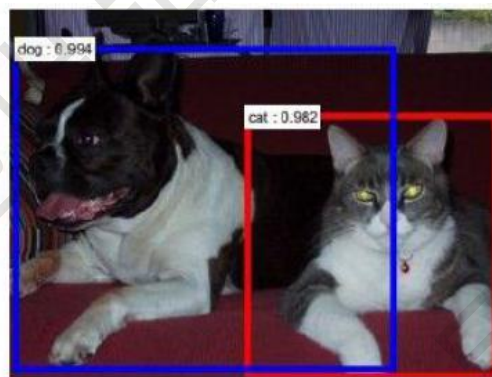
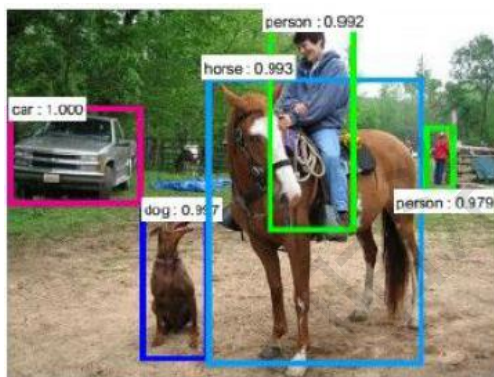
计算机视觉

✓ CV领域发展:



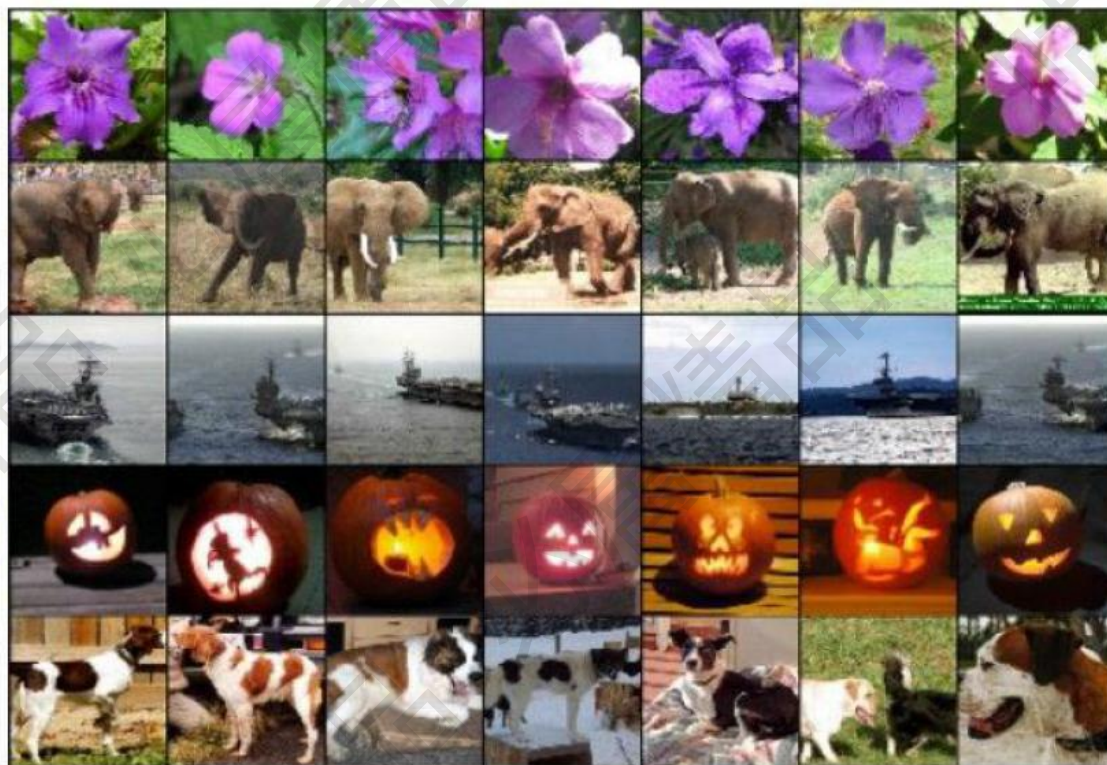
卷积神经网络:

✓ 检测任务:



卷积神经网络:

✓ 分类与检索:



卷积神经网络:

✓ 超分辨率重构:



Inputs



Our result



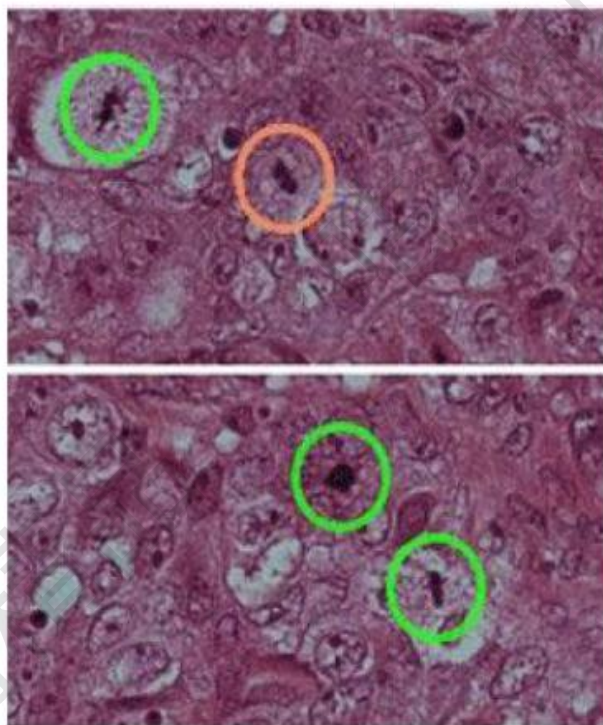
SRGAN result



Original

卷积神经网络：

✓ 医学任务等：



卷积神经网络:

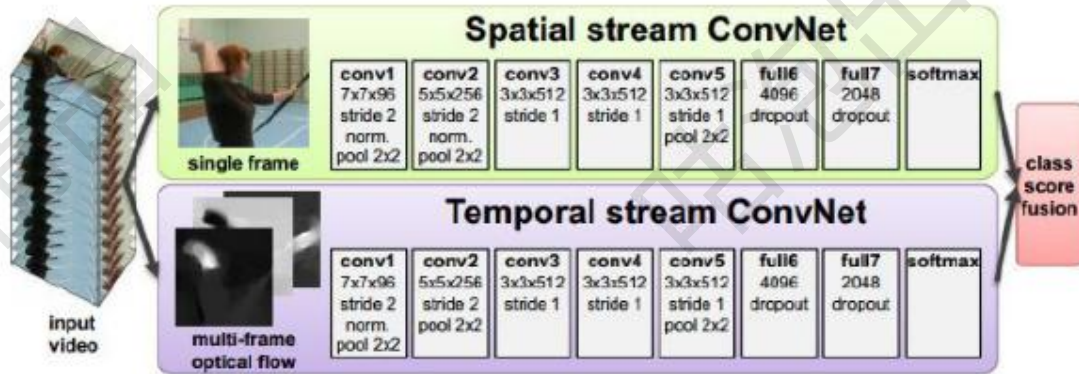
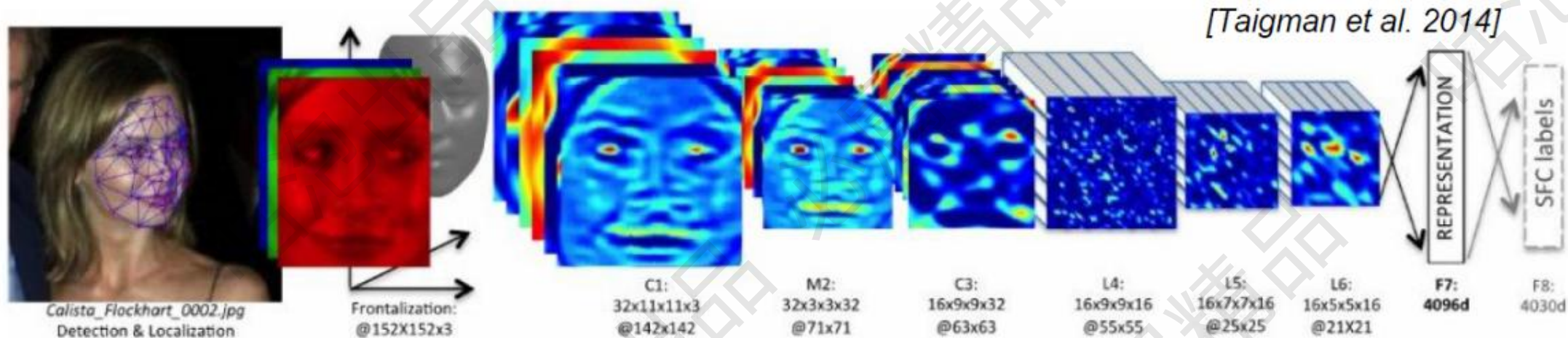
✓ 无人驾驶:



NVIDIA Tegra X1

卷积神经网络:

✓ 人脸识别:



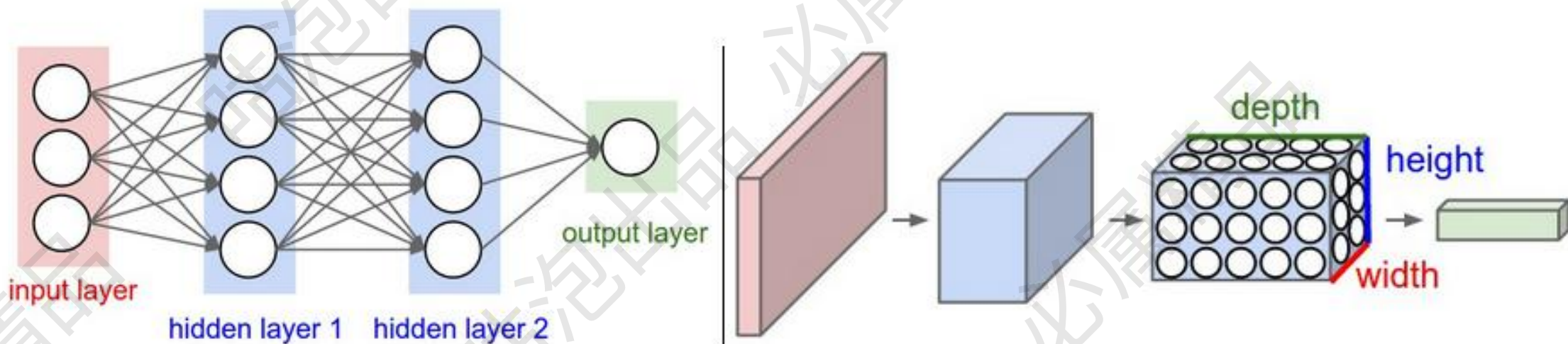
[Simonyan et al. 2014]



[Goodfellow 2014]

卷积神经网络：

✓ 卷积网络与传统网络的区别：



卷积神经网络：

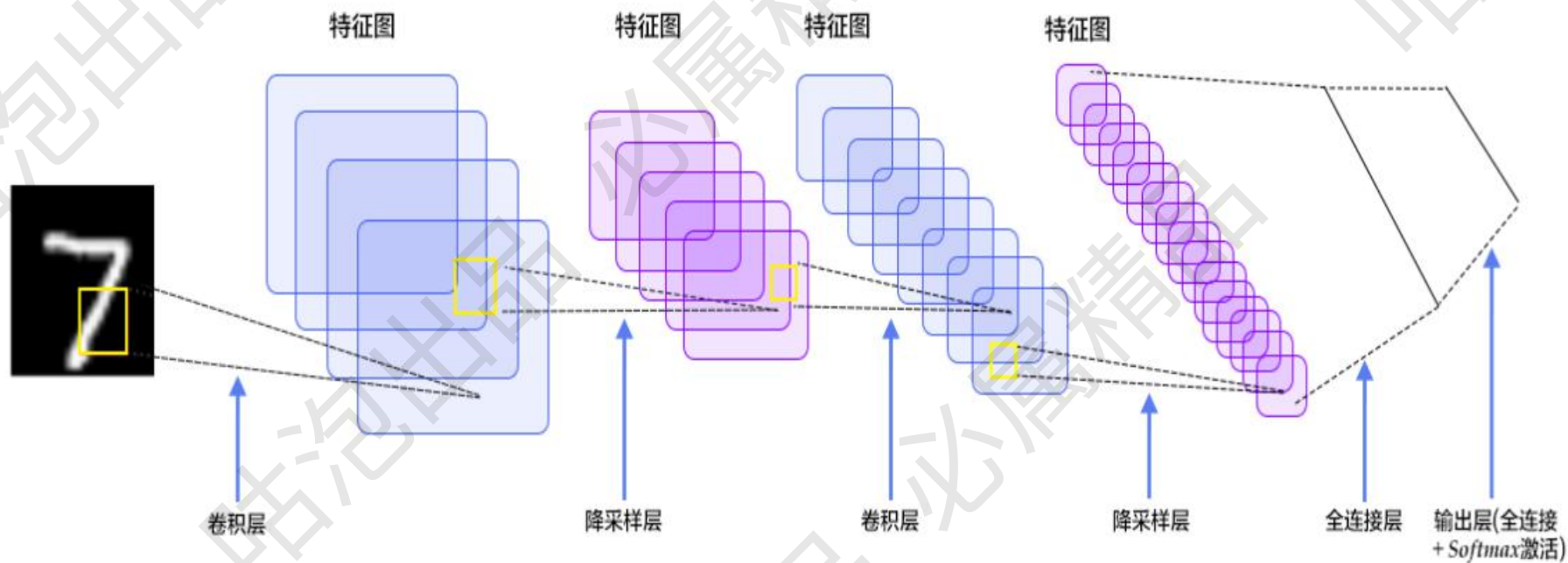
✓ 整体架构：

✎ 输入层

✎ 卷积层

✎ 池化层

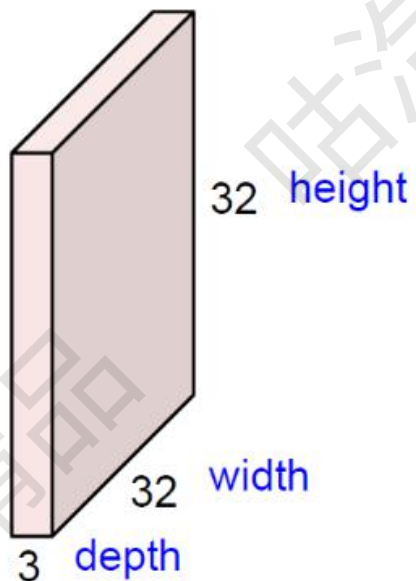
✎ 全连接层



卷积神经网络:

✓ 卷积做了一件什么事?

32x32x3 image



5x5x3



3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

卷积神经网络:

✓ 图像颜色通道



Original image (RGB)



R channel



G channel

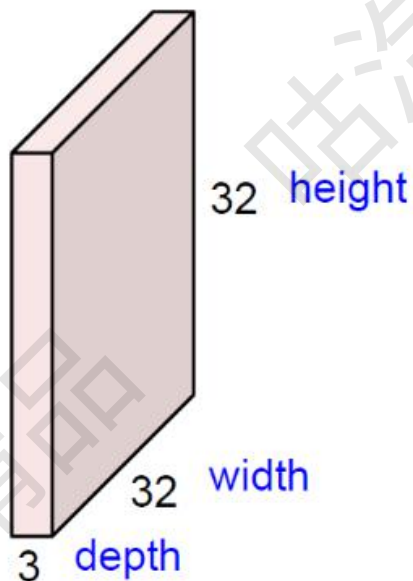


B channel

卷积神经网络:

✓ 卷积做了一件什么事?

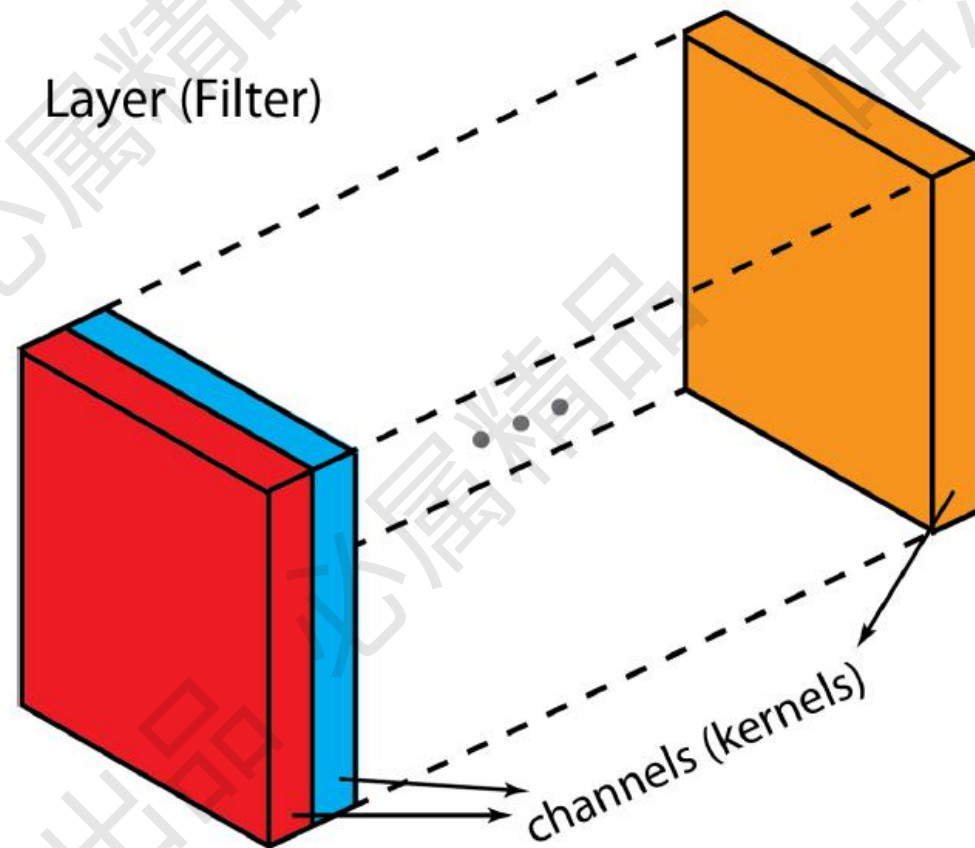
32x32x3 image



5x5x3



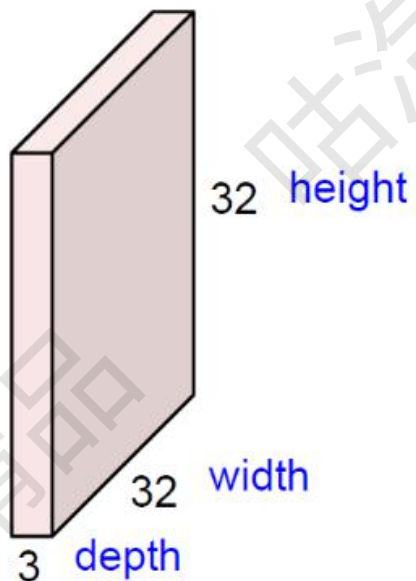
Layer (Filter)



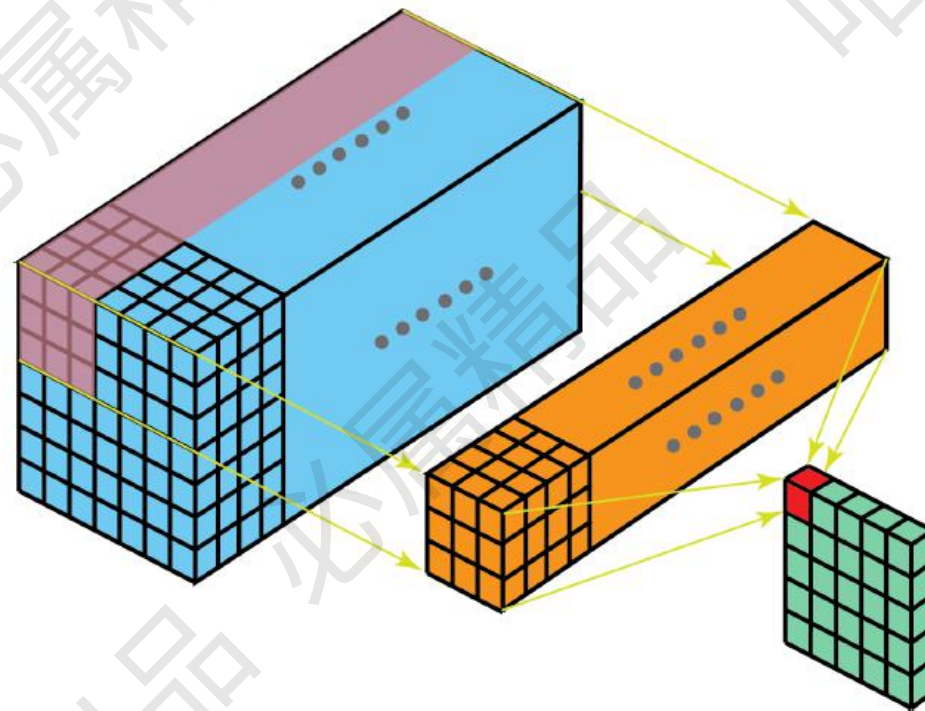
卷积神经网络:

✓ 卷积做了一件什么事?

32x32x3 image

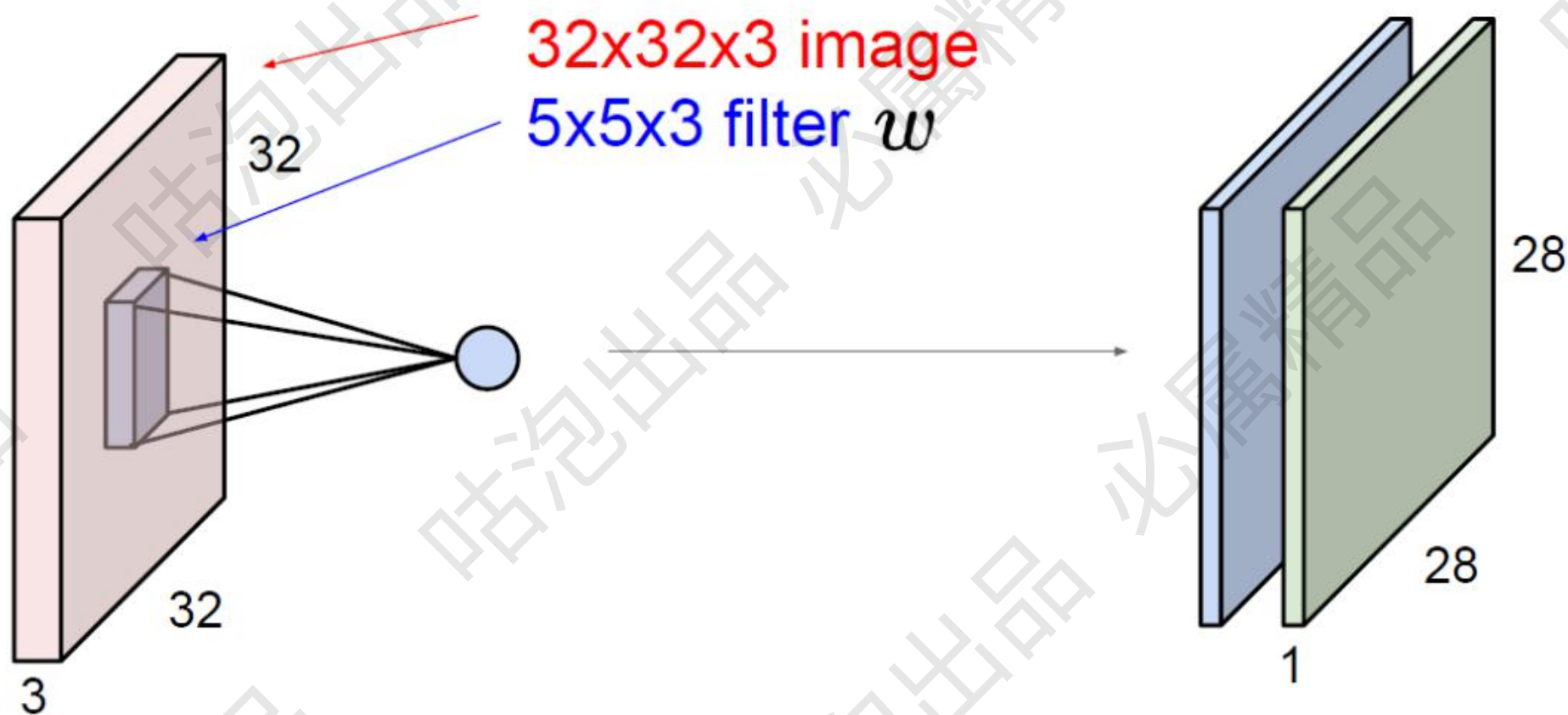


5x5x3



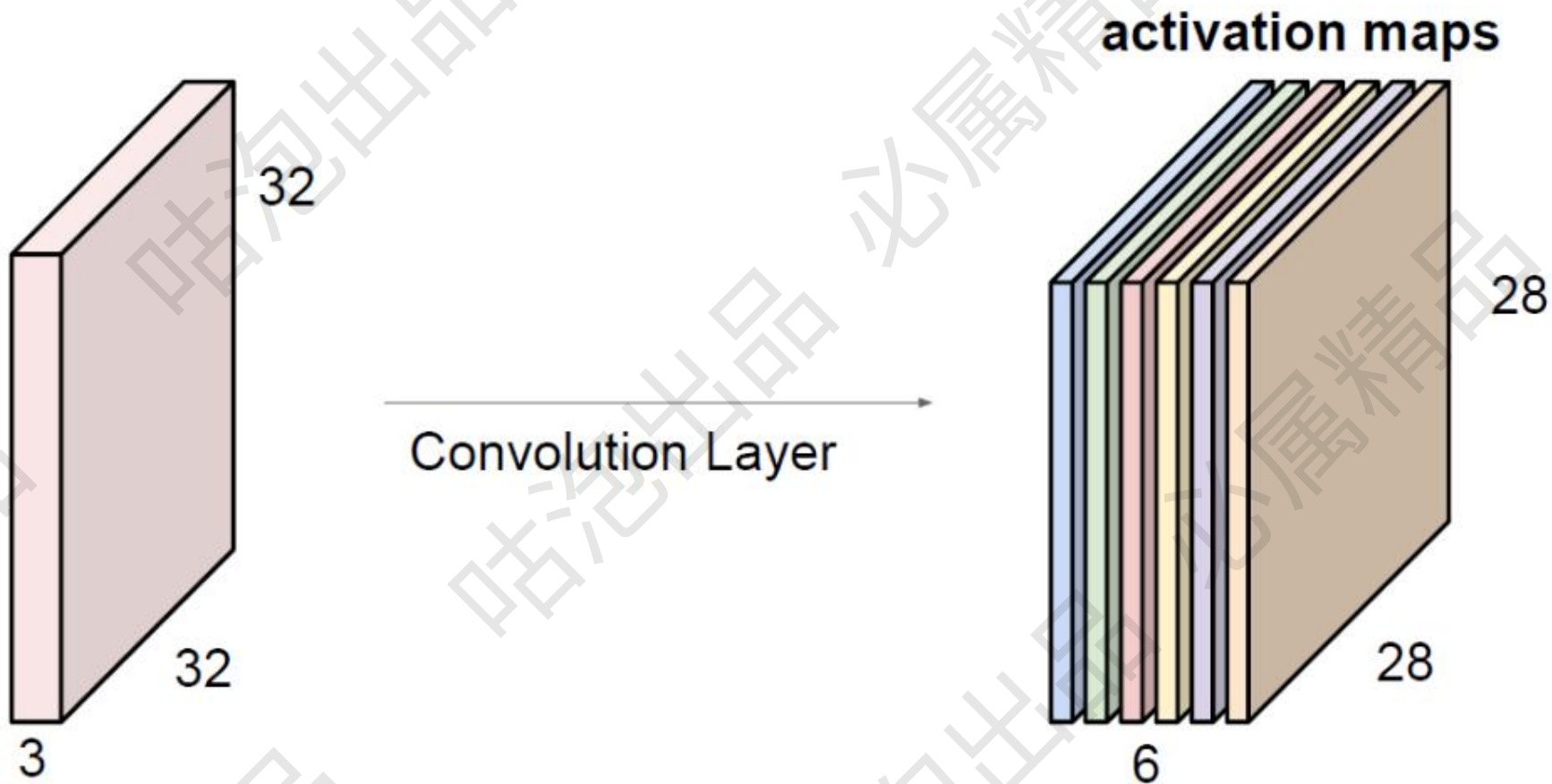
卷积神经网络:

✓ 特征图个数



卷积神经网络:

✓ 特征图个数



Input Volume (+pad 1) (7x7x3)

x[:, :, 0]

0	0	0	0	0	0	0
0	1	1	2	2	1	0
0	1	1	1	2	1	0
0	2	1	1	0	2	0
0	2	1	0	1	2	0
0	2	1	2	2	2	0
0	0	0	0	0	0	0

x[:, :, 1]

0	0	0	0	0	0	0
0	0	1	2	0	1	0
0	2	2	1	1	0	0
0	2	1	0	0	2	0
0	1	0	0	0	2	0
0	0	1	0	1	2	0
0	0	0	0	0	0	0

x[:, :, 2]

0	0	0	0	0	0	0
0	2	2	0	1	2	0
0	0	0	2	1	2	0
0	2	1	0	2	1	0
0	1	1	0	0	0	0
0	0	0	1	1	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

w0[:, :, 0]

1	1	1
-1	-1	0
-1	1	0

w0[:, :, 1]

-1	-1	1
-1	1	0
-1	1	0

w0[:, :, 2]

1	0	-1
0	0	0
1	-1	-1

Bias b0 (1x1x1)

b0[:, :, 0]

1

Filter W1 (3x3x3)

w1[:, :, 0]

0	0	-1
-1	1	1
0	0	0

w1[:, :, 1]

0	0	1
1	0	1
0	-1	-1

w1[:, :, 2]

-1	1	1
0	1	1
1	-1	1

Bias b1 (1x1x1)

b1[:, :, 0]

0

Output Volume (3x3x2)

o[:, :, 0]

3	-5	-4
4	-1	7
0	0	-1

o[:, :, 1]

3	2	0
7	6	2
5	5	2

toggle movement

Input Volume (+pad 1) (7x7x3)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	1	1	2	2	1	0
0	1	1	1	2	1	0
0	2	1	1	0	2	0
0	2	1	0	1	2	0
0	2	1	2	2	2	0
0	0	0	0	0	0	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	0	1	2	0	1	0
0	2	2	1	1	0	0
0	2	1	0	0	2	0
0	1	0	0	0	2	0
0	0	1	0	1	2	0
0	0	0	0	0	0	0

 $x[:, :, 2]$

0	0	0	0	0	0	0
0	2	2	0	1	2	0
0	0	0	2	1	2	0
0	2	1	0	2	1	0
0	1	1	0	0	0	0
0	0	0	1	1	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

 $w0[:, :, 0]$

1	1	1
-1	-1	0
-1	1	0

 $w0[:, :, 1]$

-1	-1	1
-1	1	0
-1	1	0

 $w0[:, :, 2]$

1	0	-1
0	0	0
1	-1	-1

Bias b0 (1x1x1)

 $b0[:, :, 0]$

1

Filter W1 (3x3x3)

 $w1[:, :, 0]$

0	0	-1
-1	1	1
0	0	0

 $w1[:, :, 1]$

0	0	1
1	0	1
0	-1	-1

 $w1[:, :, 2]$

-1	1	1
0	1	1
1	-1	1

Bias b1 (1x1x1)

 $b1[:, :, 0]$

0

Output Volume (3x3x2)

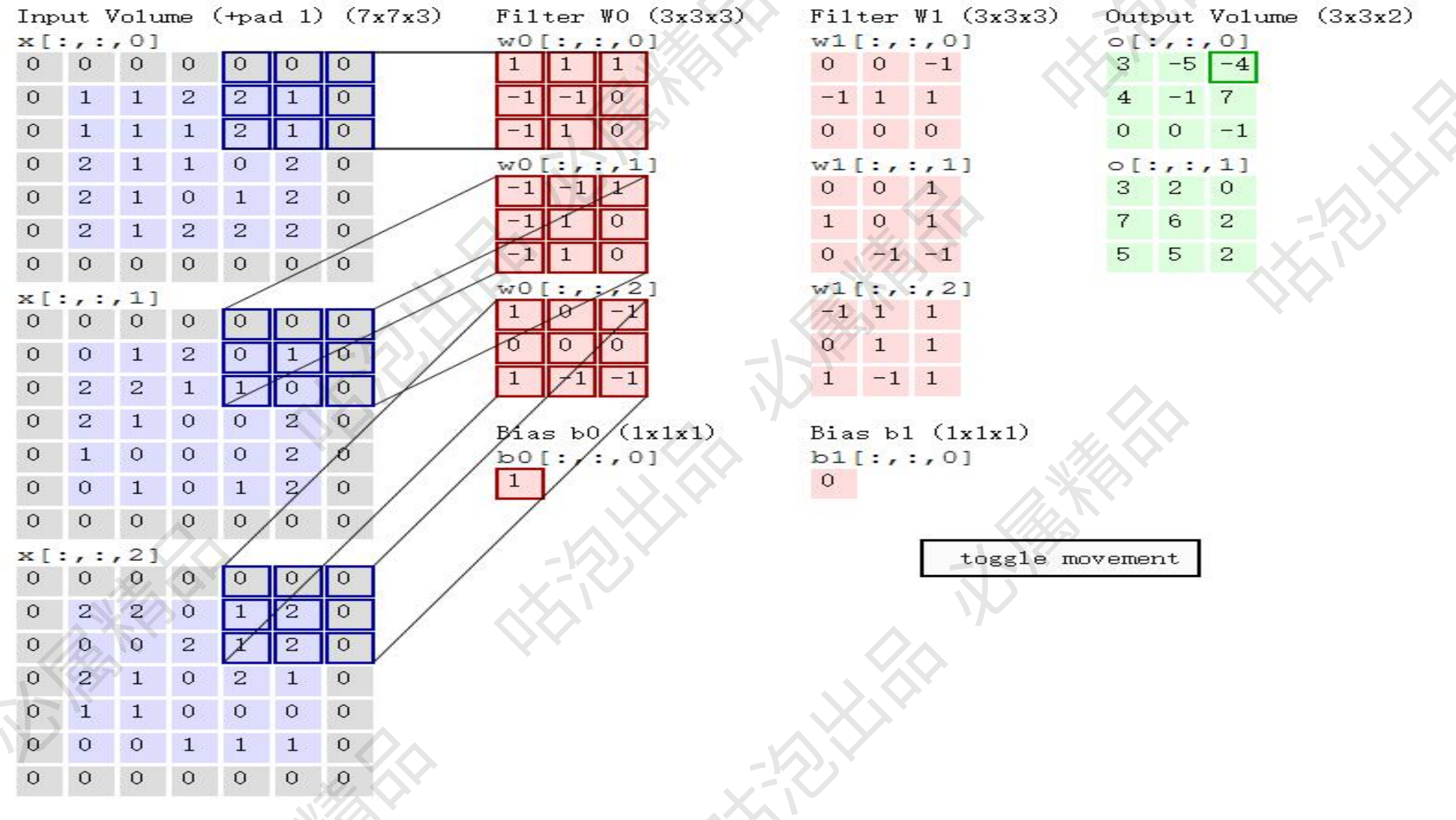
 $o[:, :, 0]$

3	-5	-4
4	-1	7
0	0	-1

 $o[:, :, 1]$

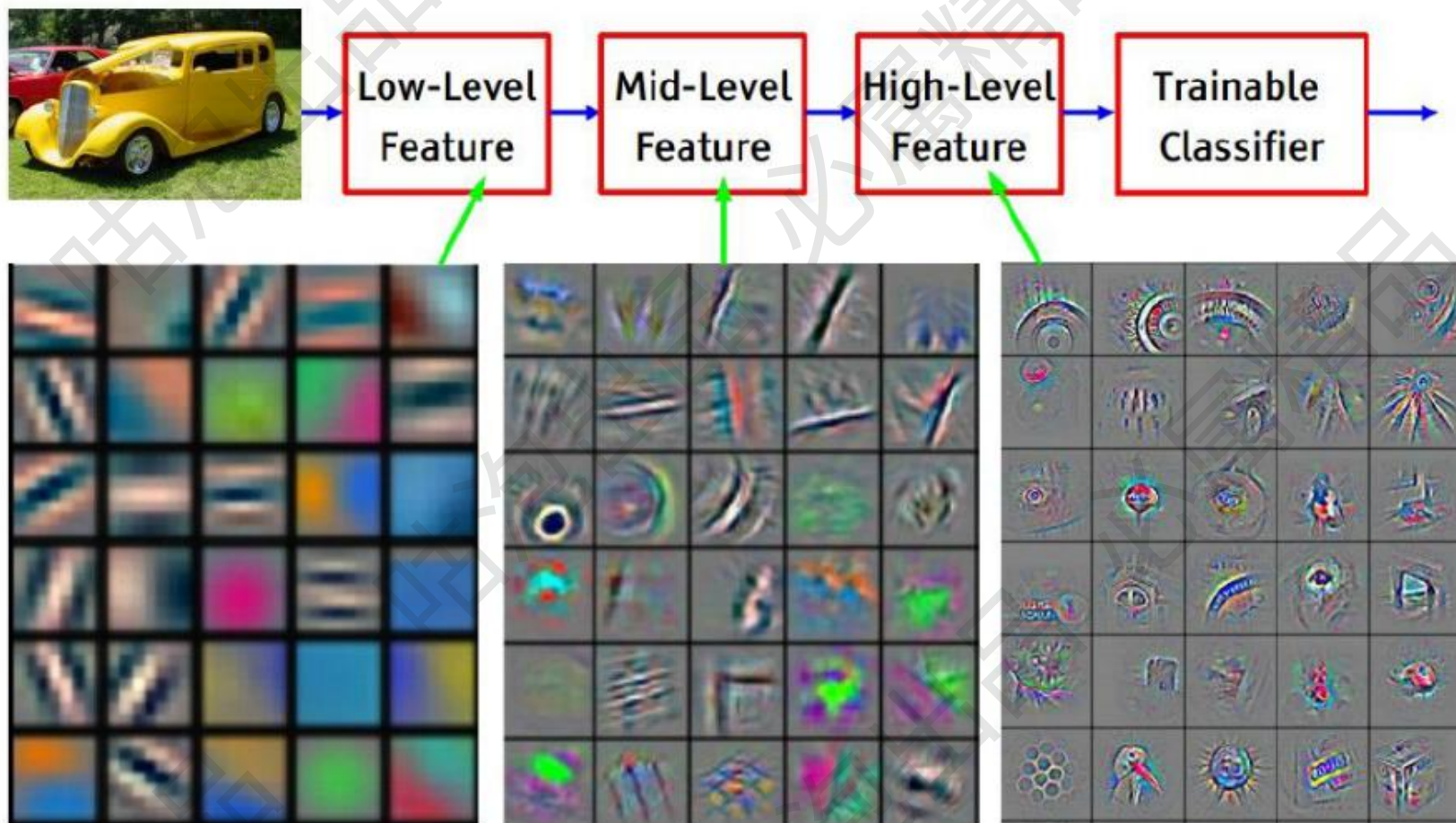
3	2	0
7	6	2
5	5	2

toggle movement



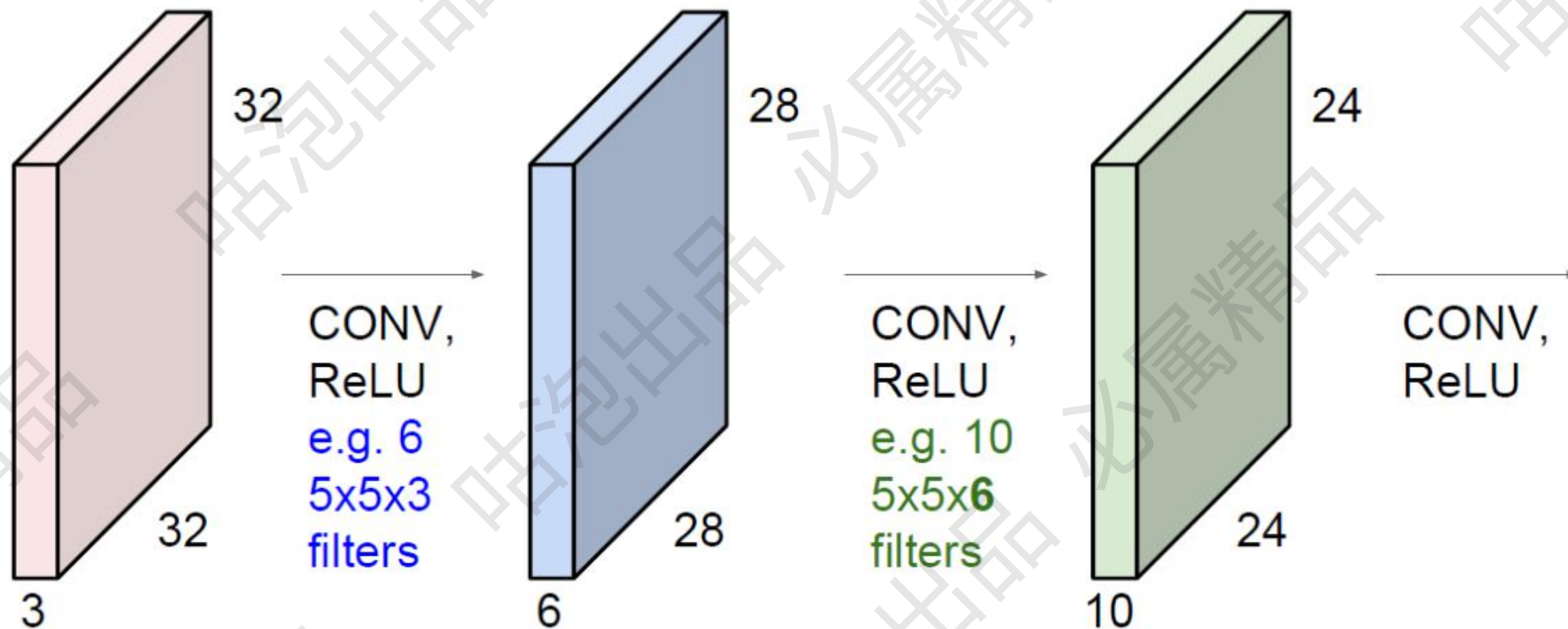
卷积神经网络:

✓ 只做一次卷积就可以了么?



卷积神经网络:

✓ 堆叠的卷积层



卷积神经网络：

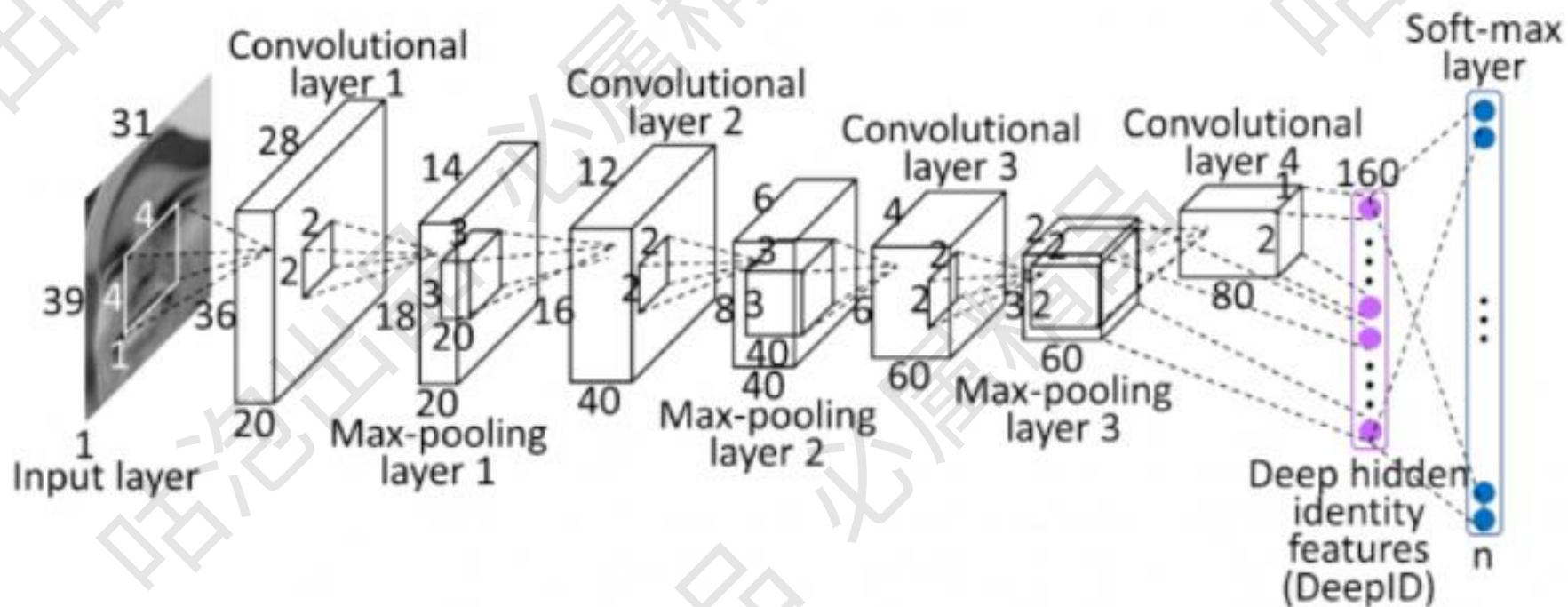
✓ 卷积层涉及参数：

✎ 滑动窗口步长

✎ 卷积核尺寸

✎ 边缘填充

✎ 卷积核个数



卷积神经网络:

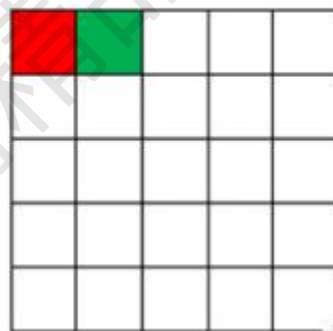
✓ 步长:

✎ 步长为1的卷积:

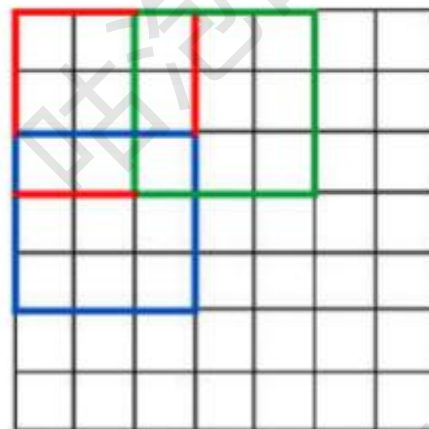
7 x 7 Input Volume



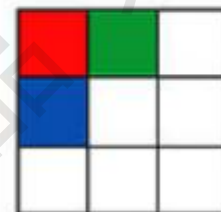
5 x 5 Output Volume



7 x 7 Input Volume



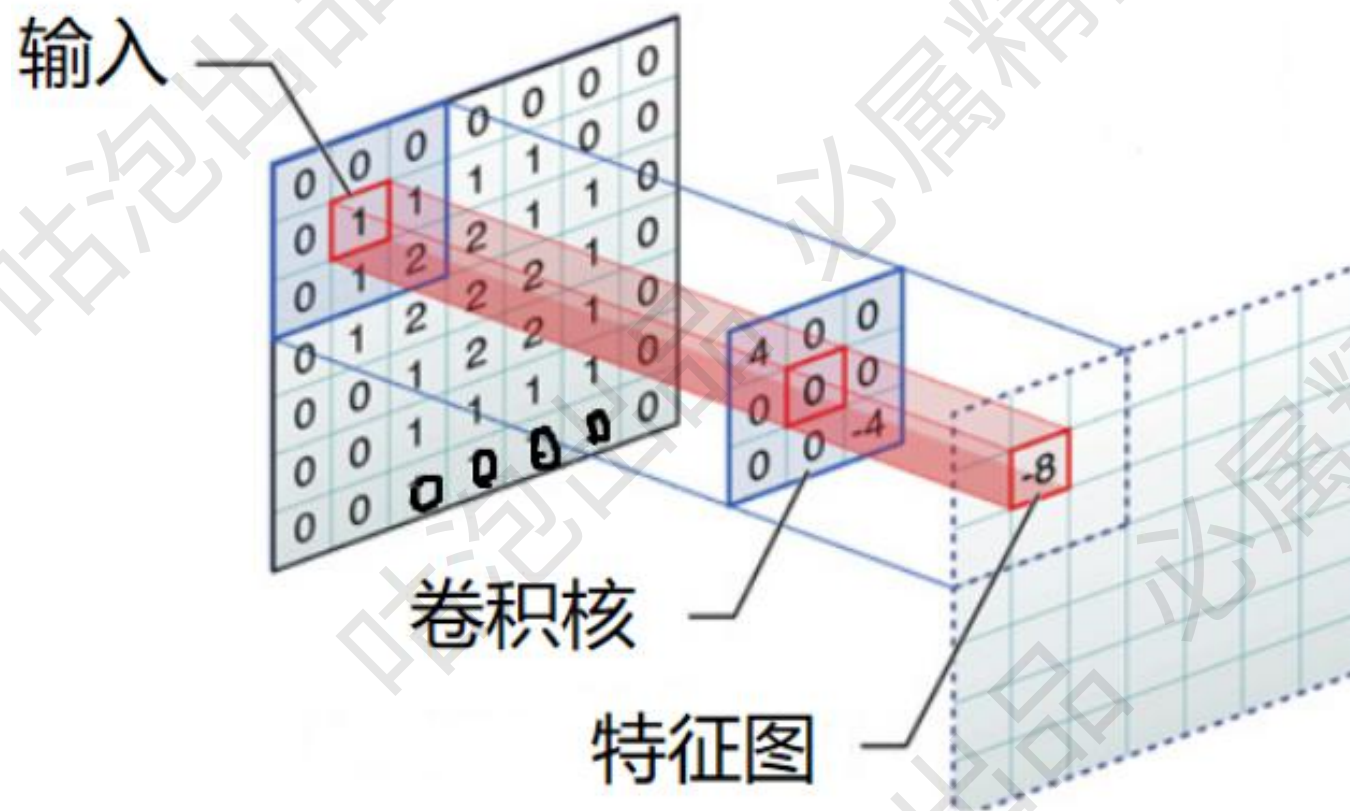
3 x 3 Output Volume



✎ 步长为2的卷积:

卷积神经网络:

✓ 边界填充:

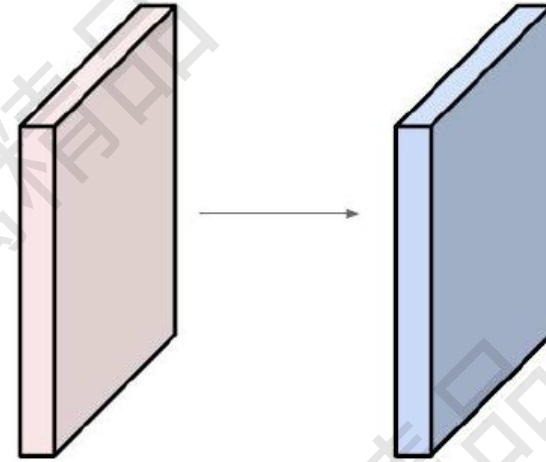


卷积神经网络：

✓ 卷积结果计算公式：

✎ 长度： $H_2 = \frac{H_1 - F_H + 2P}{S} + 1$

✎ 宽度： $W_2 = \frac{W_1 - F_W + 2P}{S} + 1$



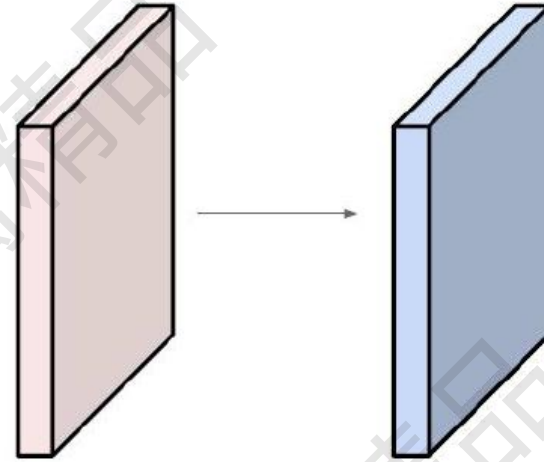
✎ 其中 W_1 、 H_1 表示输入的宽度、长度； W_2 、 H_2 表示输出特征图的宽度、长度； F 表示卷积核长和宽的大小； S 表示滑动窗口的步长； P 表示边界填充(加几圈0)。

卷积神经网络:

✓ 卷积结果计算公式:

✎ 长度: $H_2 = \frac{H_1 - F_H + 2P}{s} + 1$

✎ 宽度: $W_2 = \frac{W_1 - F_W + 2P}{s} + 1$

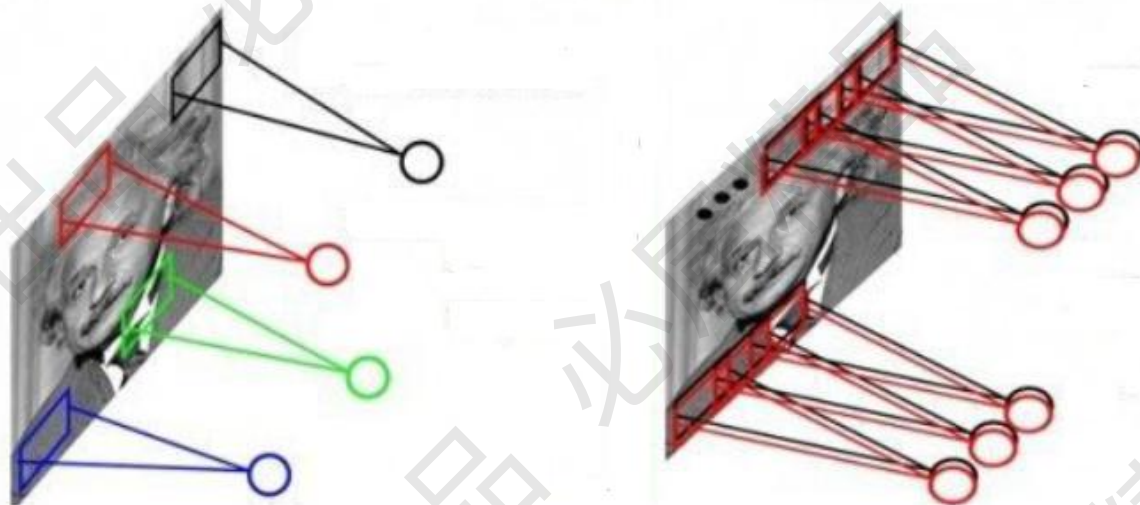


✎ 如果输入数据是 $32 \times 32 \times 3$ 的图像, 用10个 $5 \times 5 \times 3$ 的filter来进行卷积操作, 指定步长为1, 边界填充为2, 最终输出的规模为?

✎ $(32 - 5 + 2 \times 2) / 1 + 1 = 32$, 所以输出规模为 $32 \times 32 \times 10$, 经过卷积操作后也可以保持特征图长度、宽度不变。

卷积神经网络:

✓ 卷积参数共享:

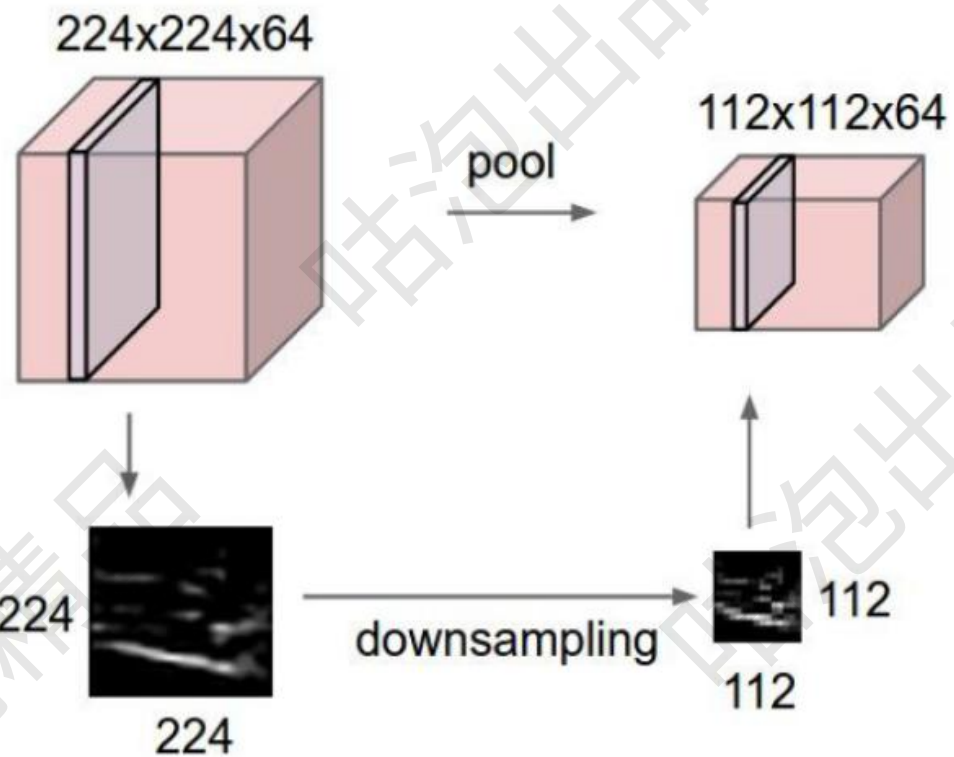


✎ 数据依旧是 $32 \times 32 \times 3$ 的图像，继续用10个 $5 \times 5 \times 3$ 的filter来进行卷积操作，所需的权重参数有多少个呢？

✎ $5 \times 5 \times 3 = 75$ ，表示每一个卷积核只需要75个参数，此时有10个不同的卷积核，就需要 $10 \times 75 = 750$ 个卷积核参数，不要忘记还有b参数，每个卷积核都有一个对应的偏置参数，最终只需要 $750 + 10 = 760$ 个权重参数。

卷积神经网络:

✓ 池化层:



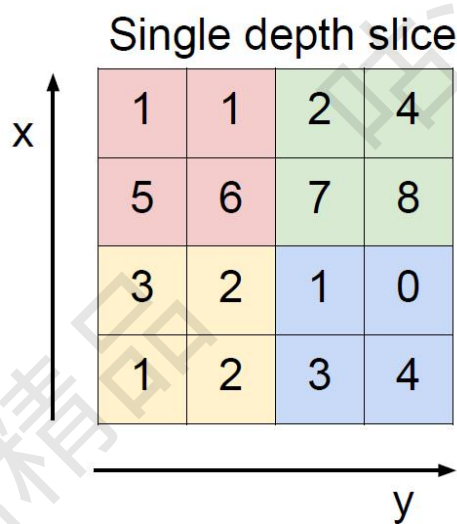
1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

7	9
8	

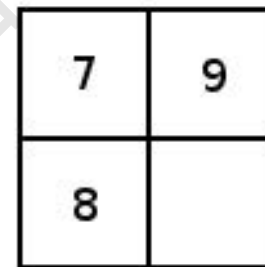
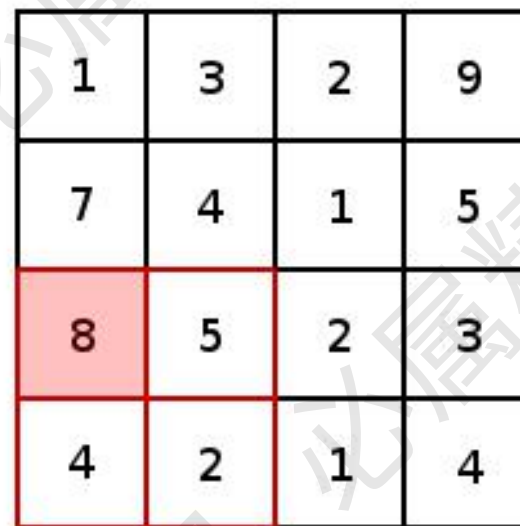
卷积神经网络:

✓ 最大池化:

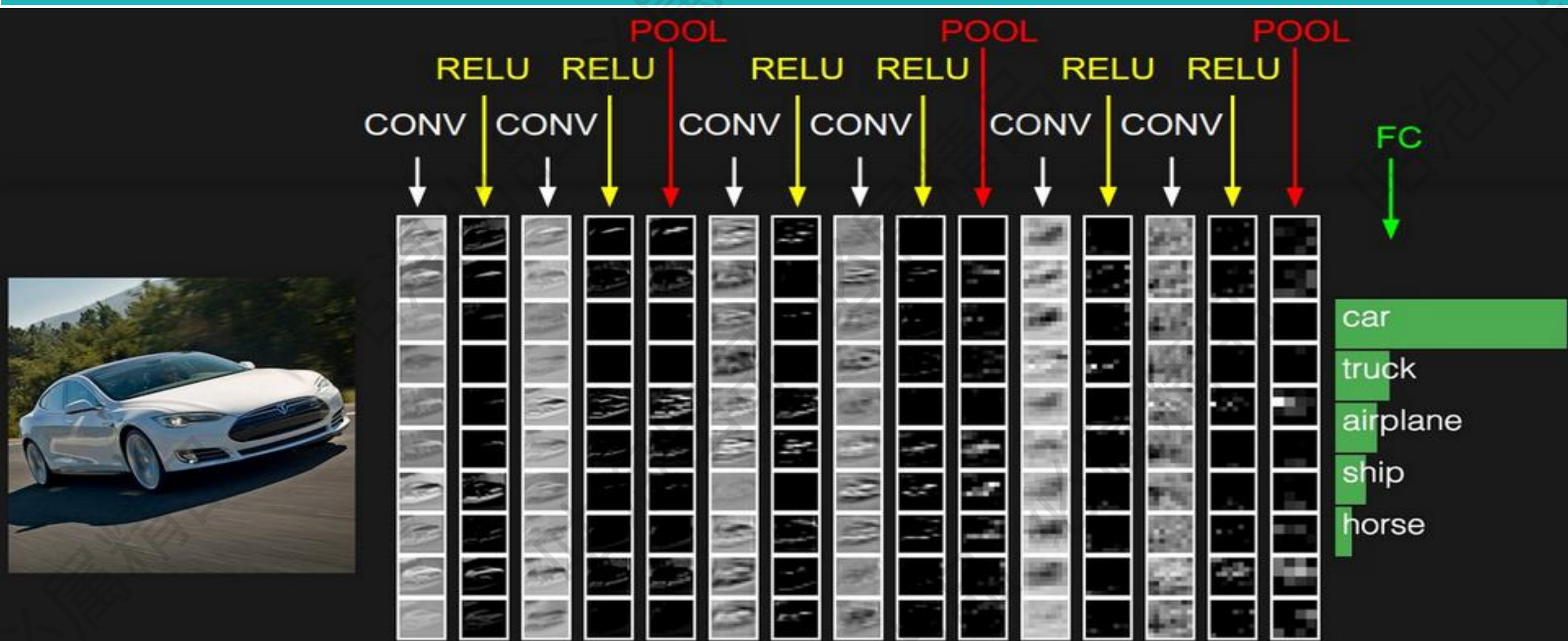
MAX POOLING



max pool with 2x2 filters
and stride 2

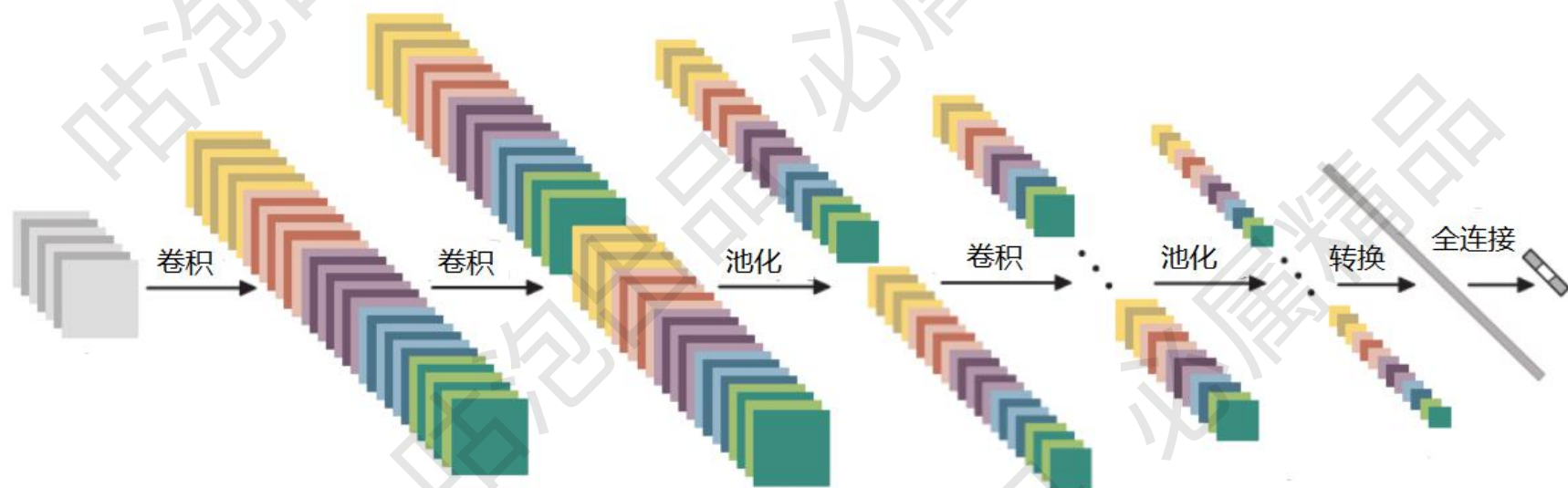


卷积神经网络:



卷积神经网络：

✓ 特征图变化：



卷积神经网络:

✓ 经典网络-Alexnet:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

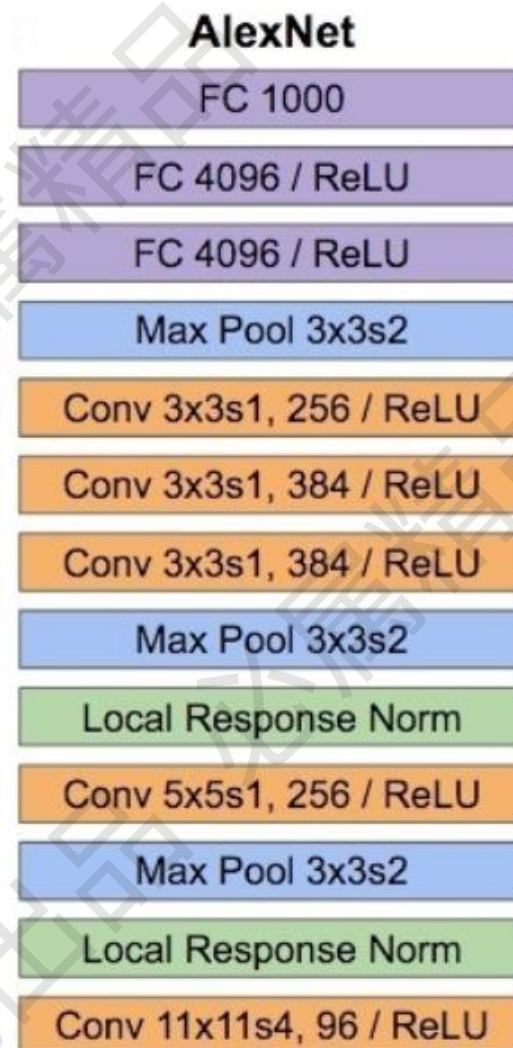
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

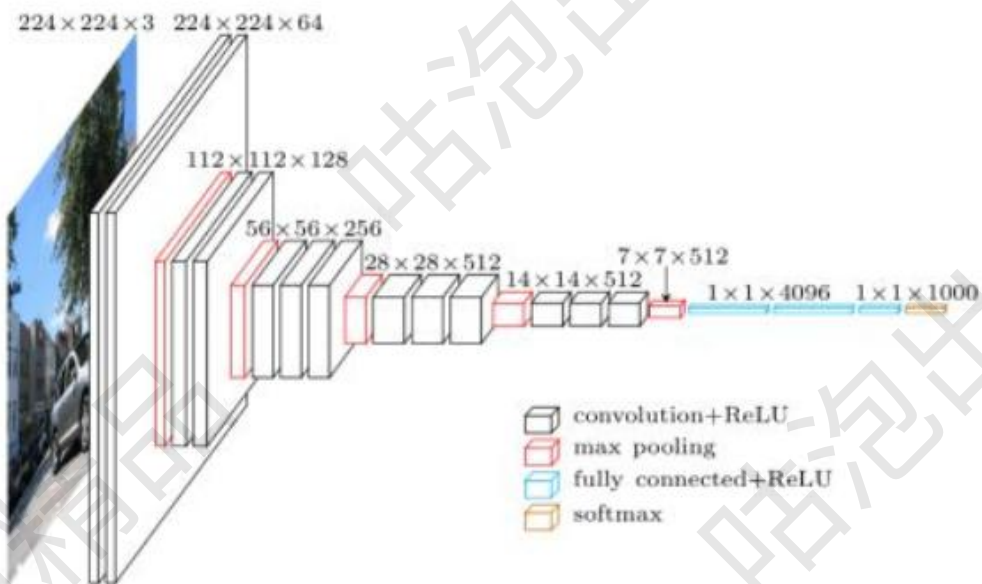
[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)



卷积神经网络:

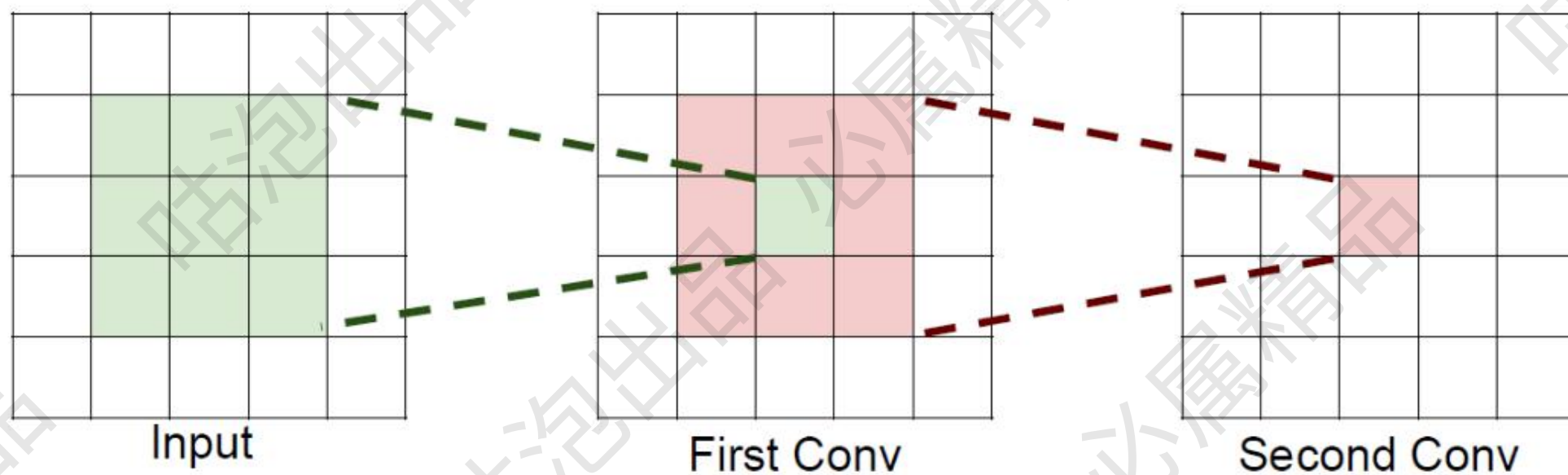
✓ 经典网络-Vgg:



ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

卷积神经网络：

✓ 感受野：



✎ 如果堆叠3个 3×3 的卷积层，并且保持滑动窗口步长为1，其感受野就是 7×7 的了，这跟一个使用 7×7 卷积核的结果是一样的，那为什么非要堆叠3个小卷积呢？

卷积神经网络：

✓ 感受野

✎ 假设输入大小都是 $h*w*c$ ，并且都使用 c 个卷积核(得到 c 个特征图)，可以用来计算一下其各自所需参数：

一个 $7*7$ 卷积核所需参数：

$$= C \times (7 \times 7 \times C) = 49 C^2$$

3个 $3*3$ 卷积核所需参数：

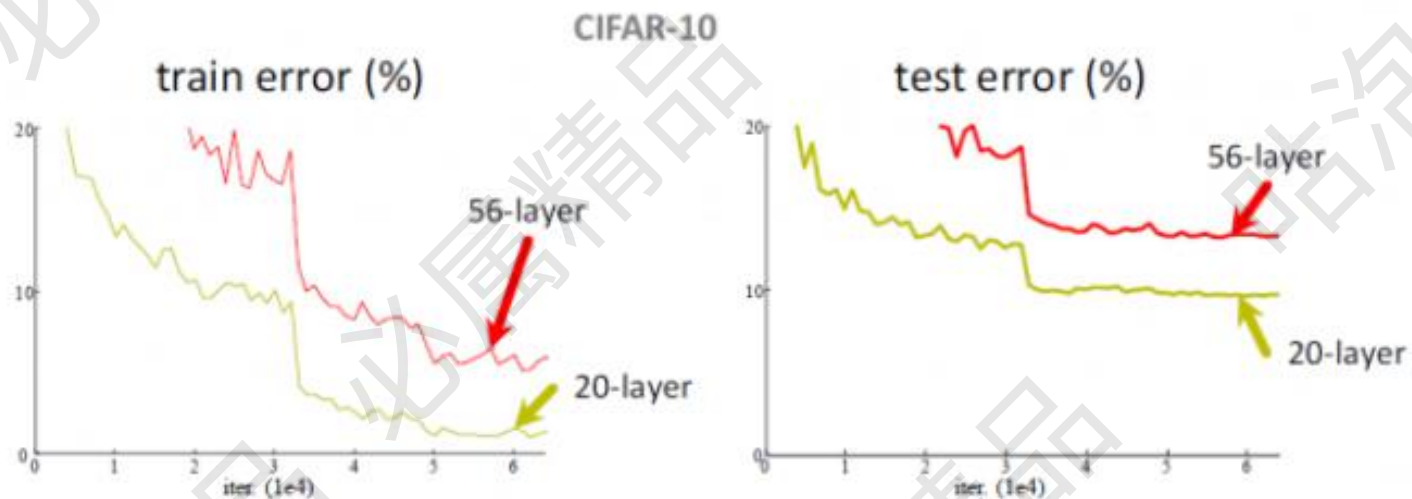
$$= 3 \times C \times (3 \times 3 \times C) = 27 C^2$$

✎ 很明显，堆叠小的卷积核所需的参数更少一些，并且卷积过程越多，特征提取也会越细致，加入的非线性变换也随着增多，还不会增大权重参数个数，这就是VGG网络的基本出发点，用小的卷积核来完成体特征提取操作。

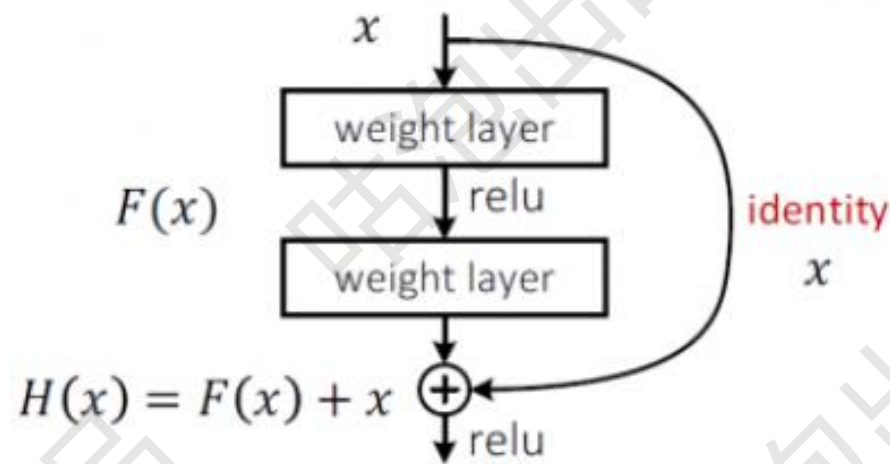
卷积神经网络：

✓ 经典网络-Resnet

✎ 深层网络遇到的问题：

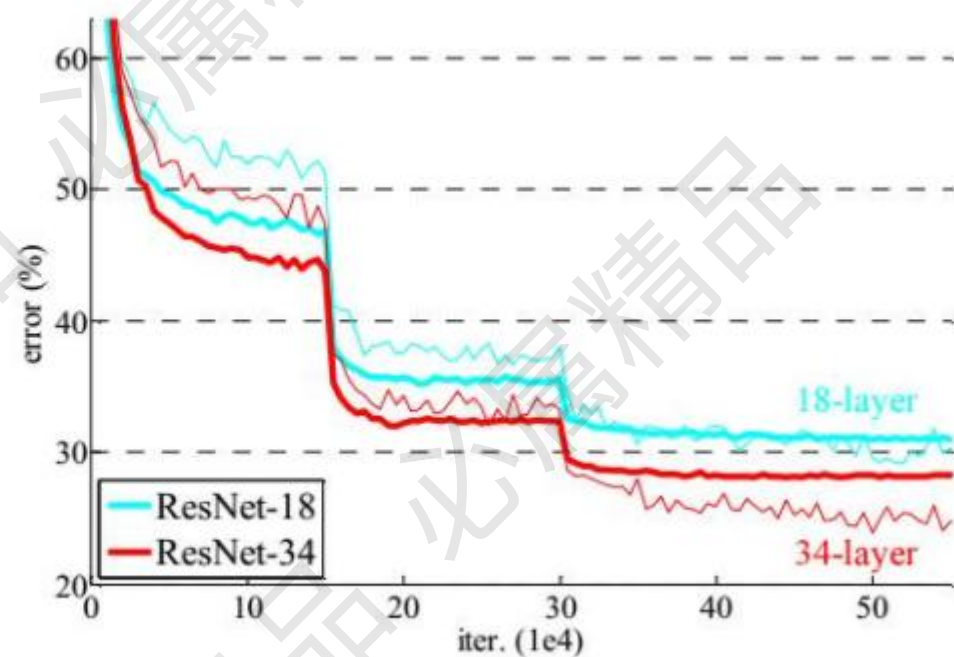
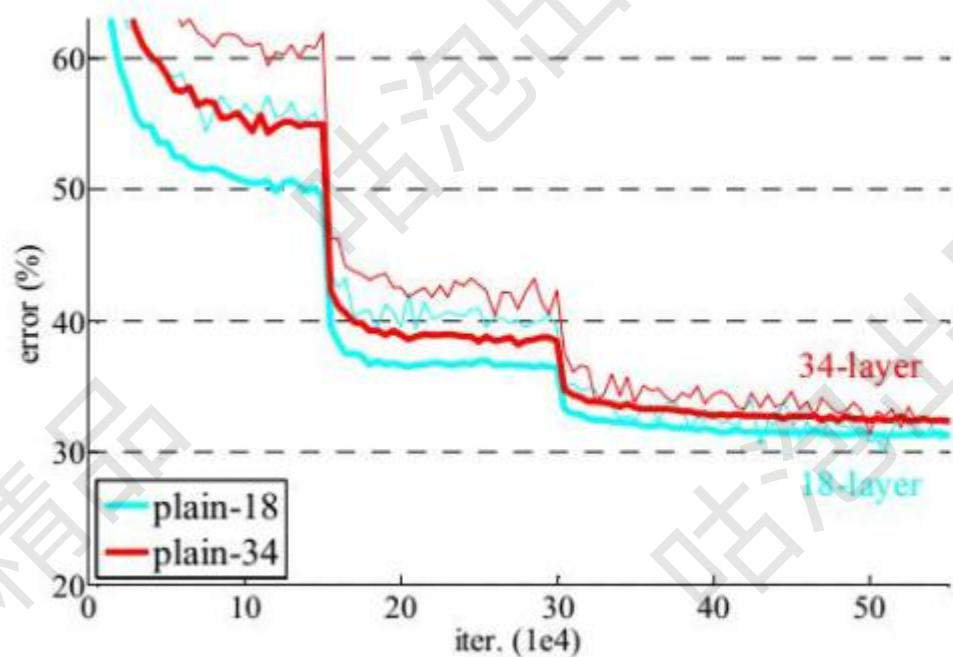


✎ 解决方案：



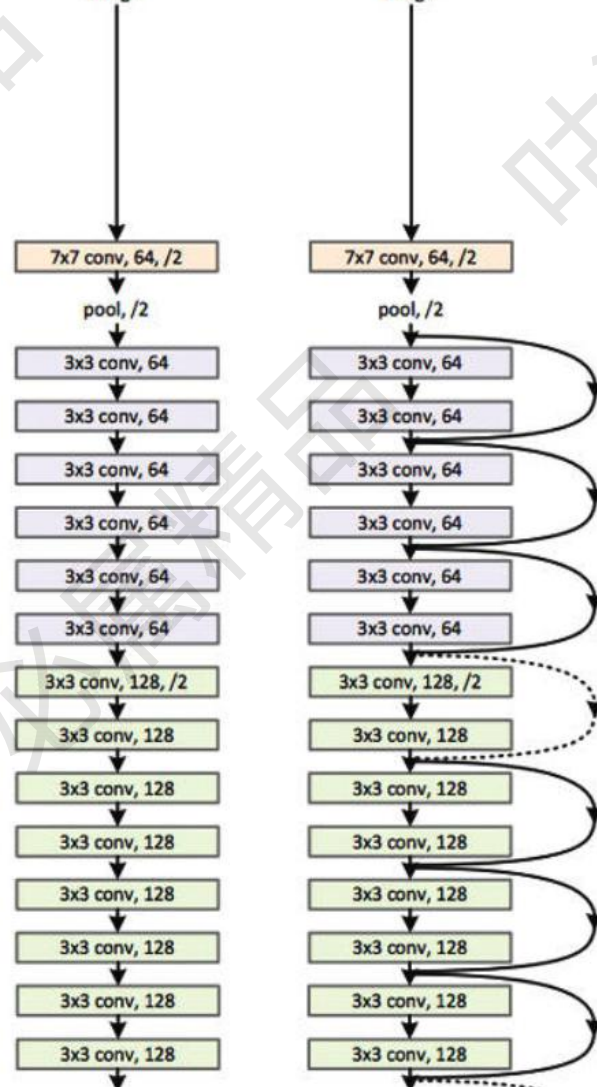
卷积神经网络：

✓ 经典网络-Resnet



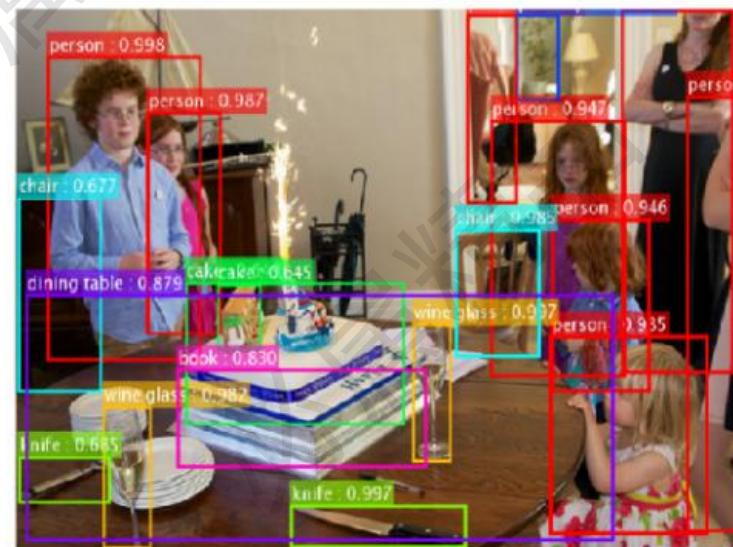
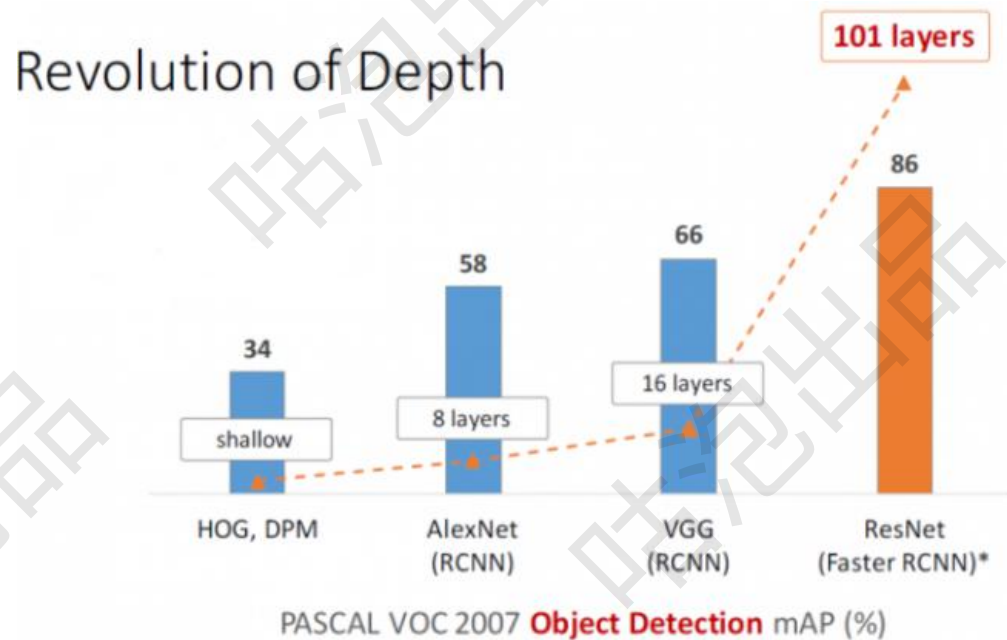
✓ 经典网络-Resnet

152 layers



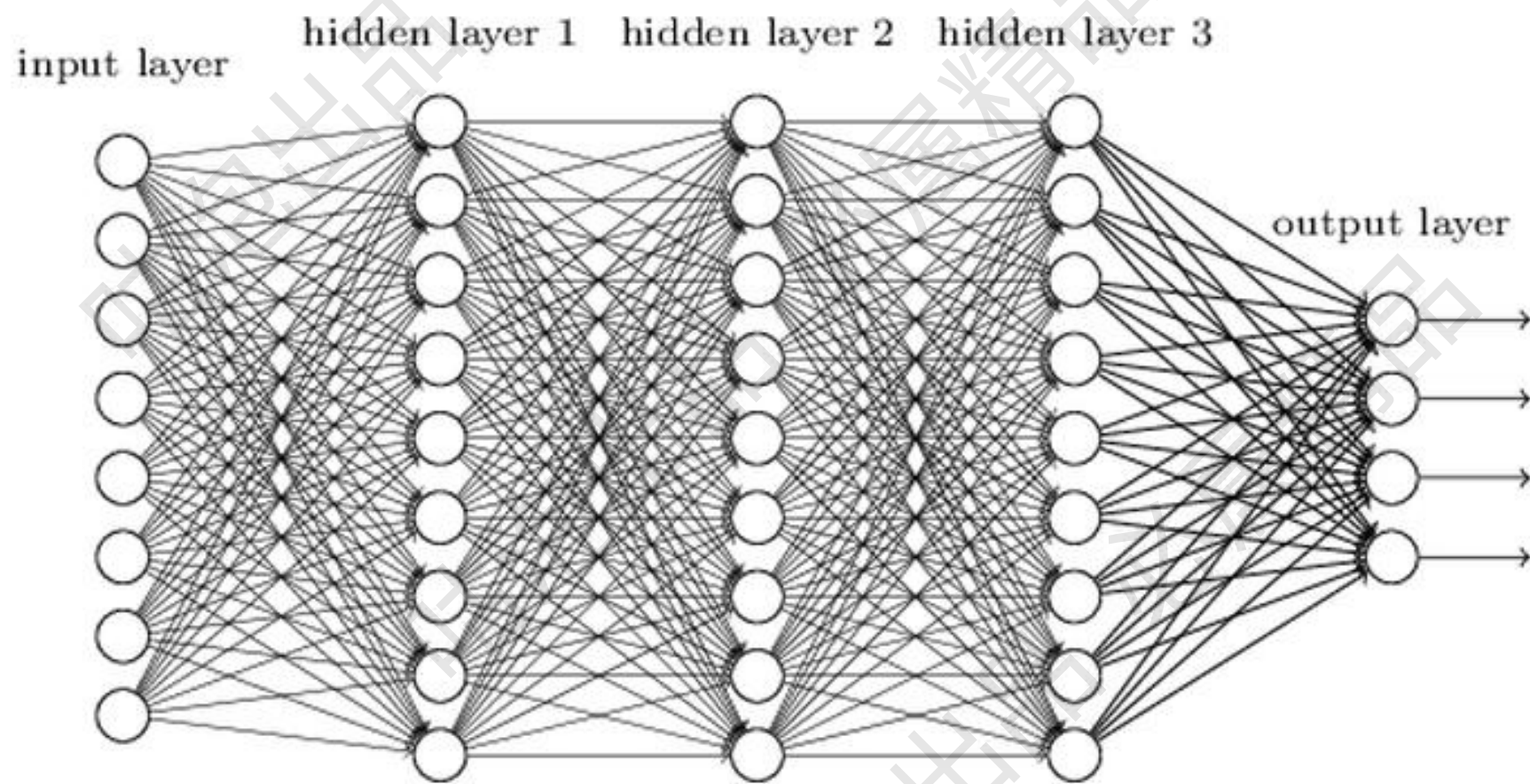
卷积神经网络:

✓ 经典网络-Resnet

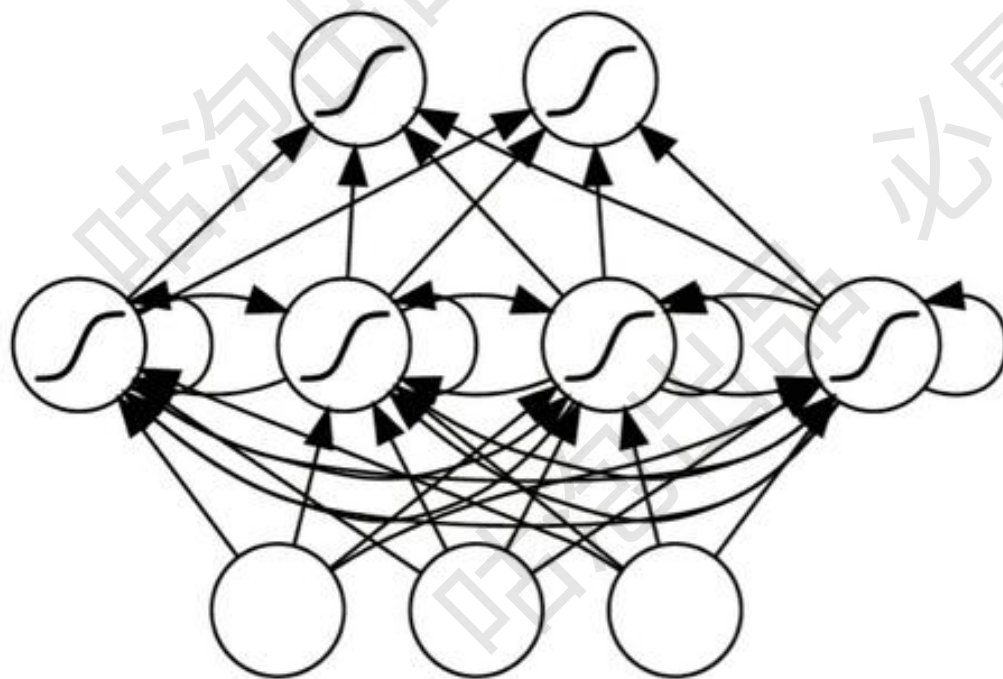


ResNet's object detection result on COCO

RNN



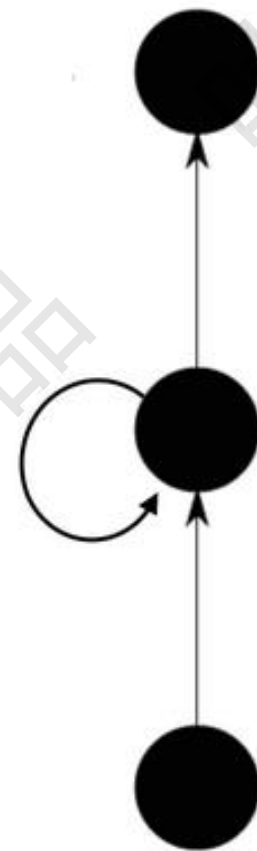
RNN



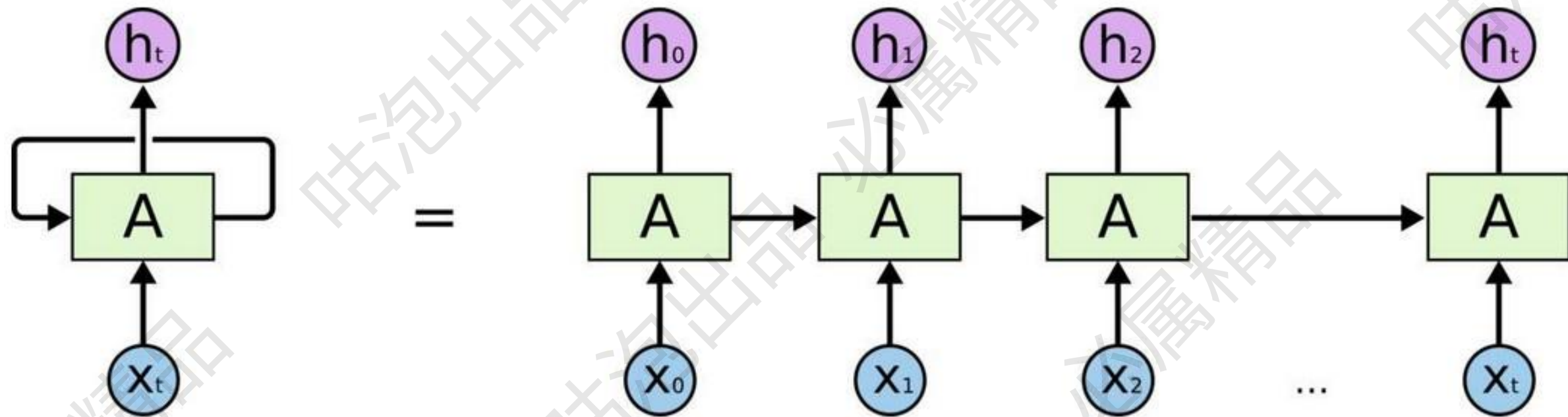
Output Layer

Hidden Layer

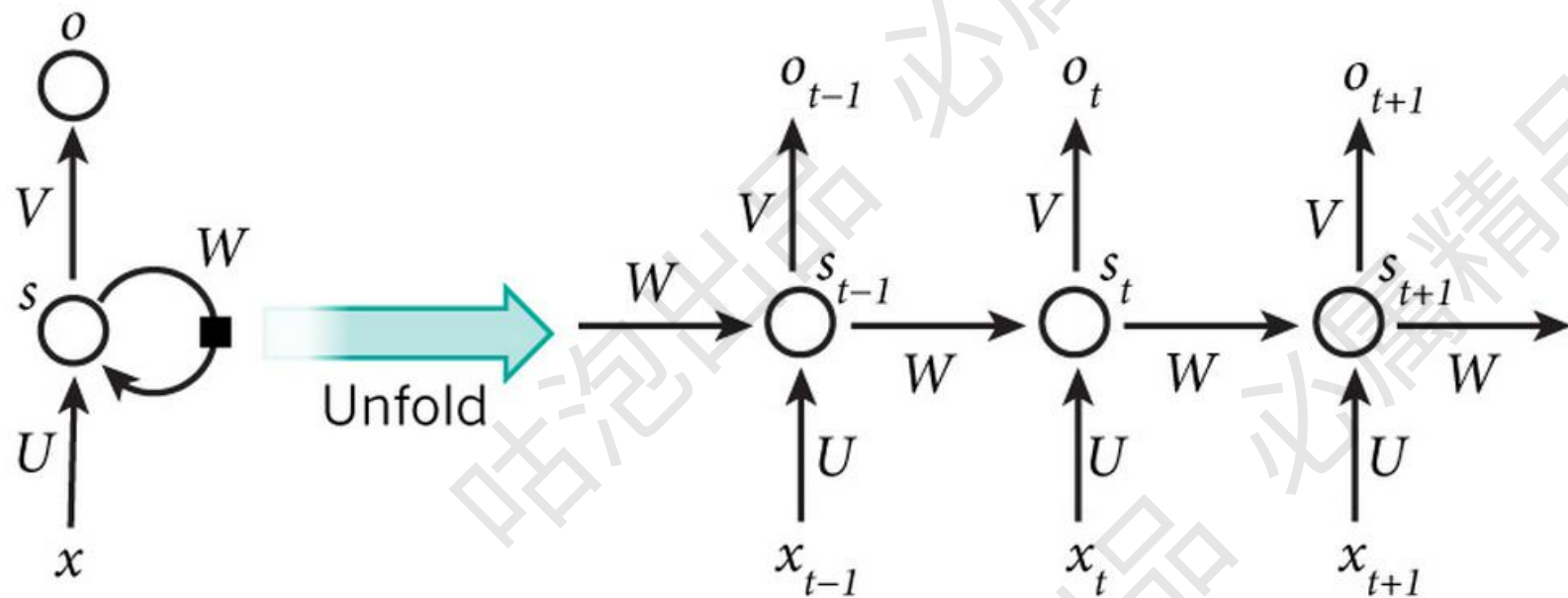
Input Layer



RNN



RNN



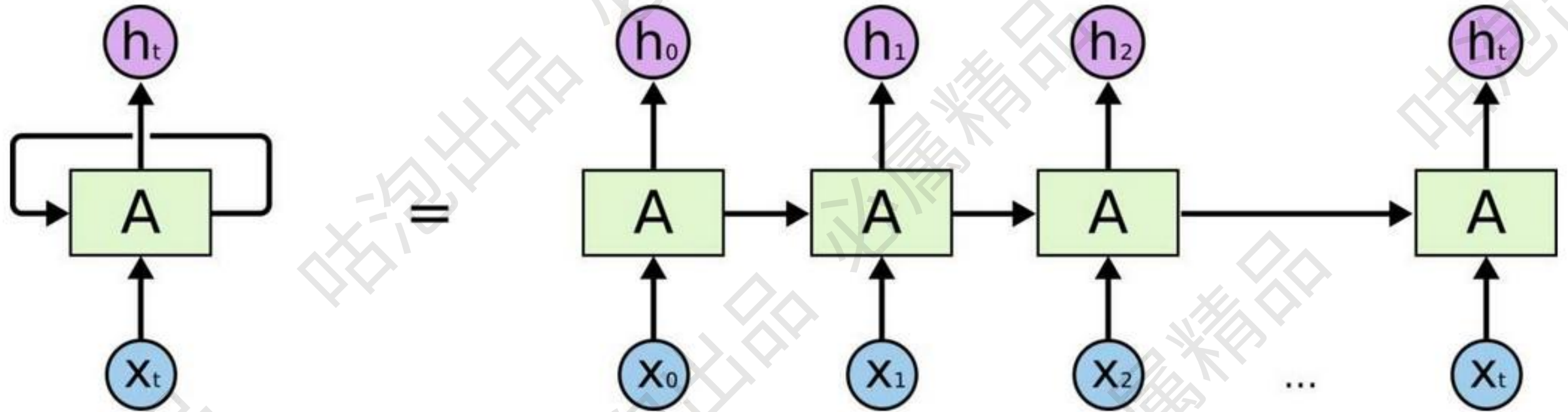
x_t 表示第 $t, t=1, 2, 3 \dots$ 步(step)的输入

s_t 为隐藏层的第 t 步的状态，它是网络的记忆单元。

$s_t = f(Ux_t + Ws_{t-1})$ ，其中 f 一般是非线性的激活函数

o_t 是第 t 步的输出，如下个单词的向量表示 $\text{softmax}(Vs_t)$

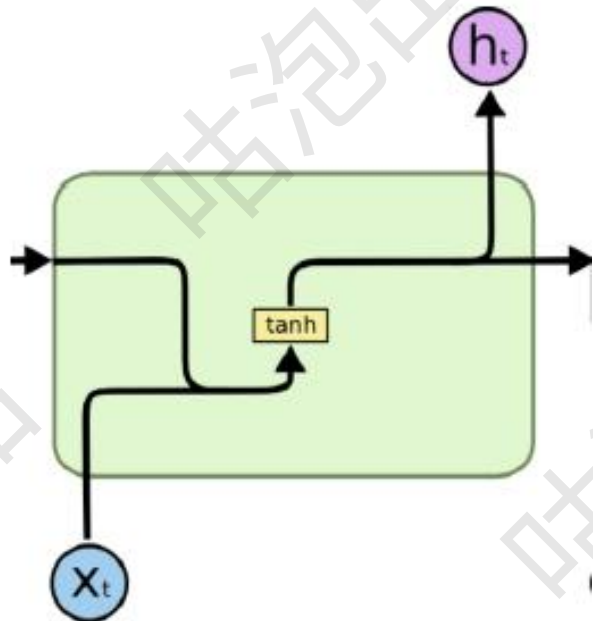
RNN



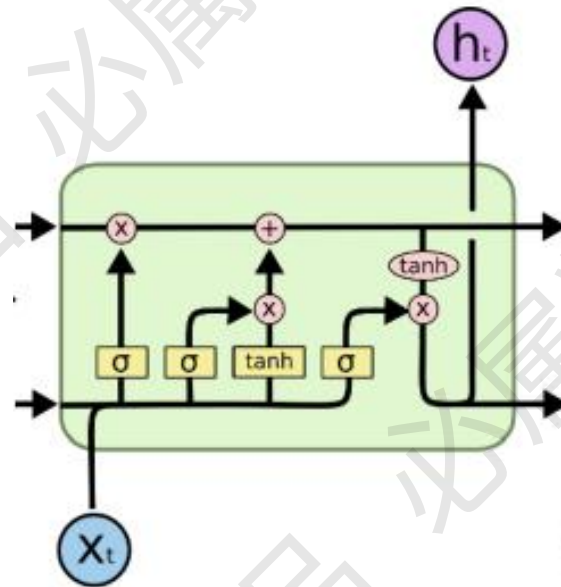
I am Chinese, I Love China

LSTM

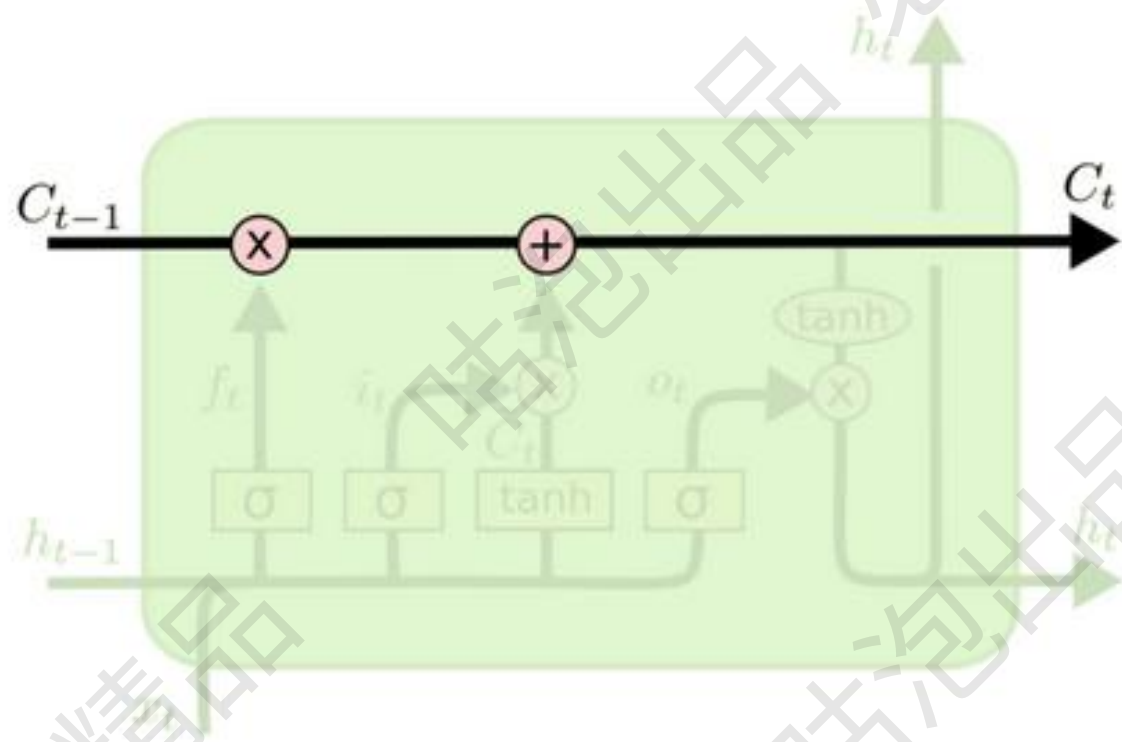
RNN



LSTM



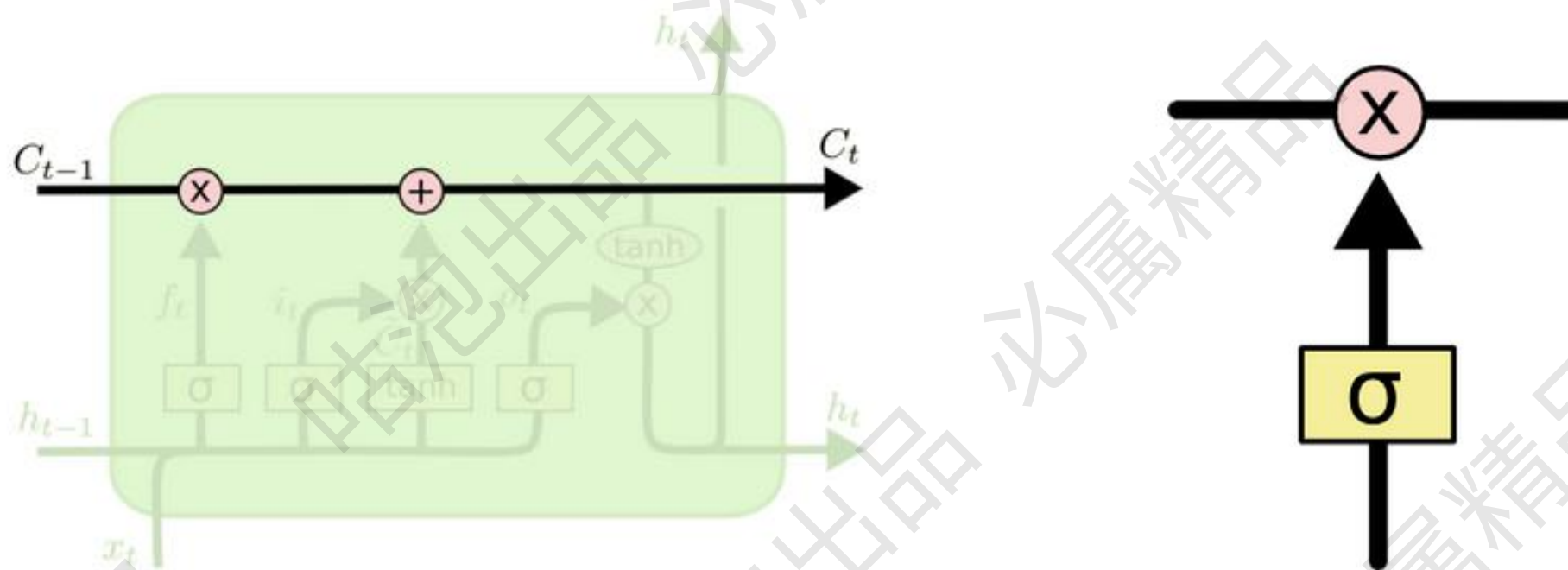
LSTM



C: 控制参数

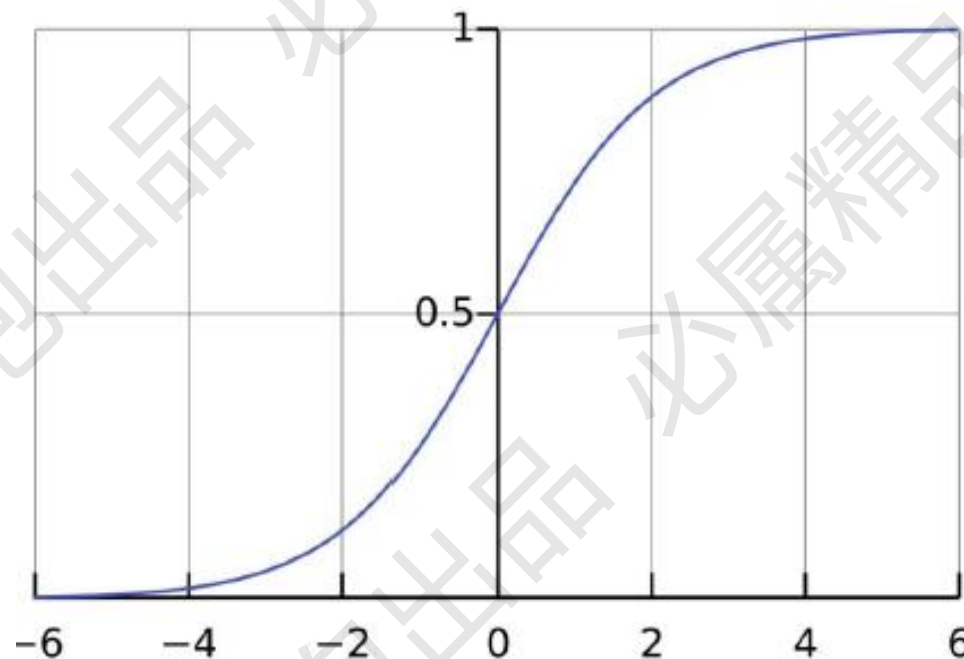
决定什么样的信息会被保留什么样的会被遗忘

LSTM



门是一种让信息选择式通过的方法
sigmoid 神经网络层和一乘法操作

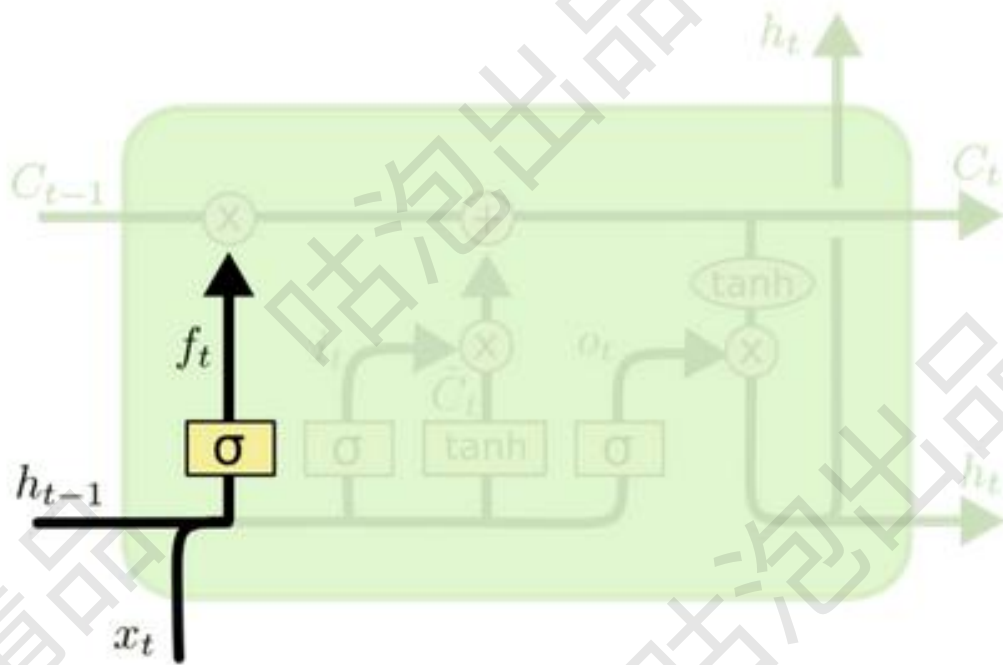
LSTM



Sigmoid函数

Sigmoid 层输出 0 到 1 之间的数值，描述每个部分有多少量可以通过。 0 代表“不许任何量通过”， 1 就指“允许任意量通过”！

LSTM

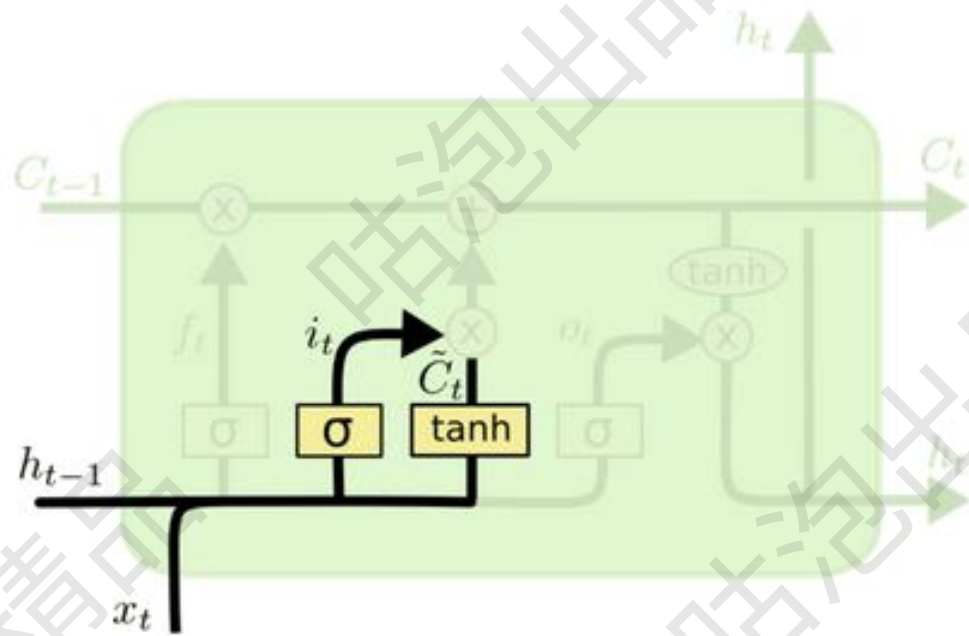


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

f_t 与 C_{t-1} 计算决定丢弃什么信息

决定丢弃信息

LSTM



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

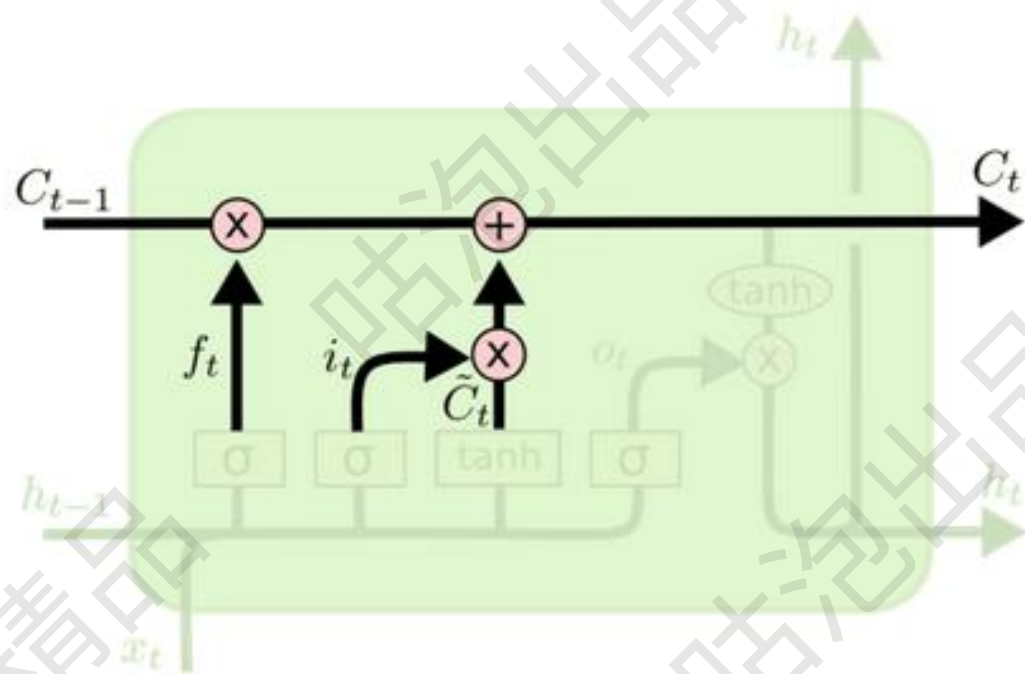
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

i_t 要保留下来的新信息

C_t 新数据形成的控制参数

确定更新的信息

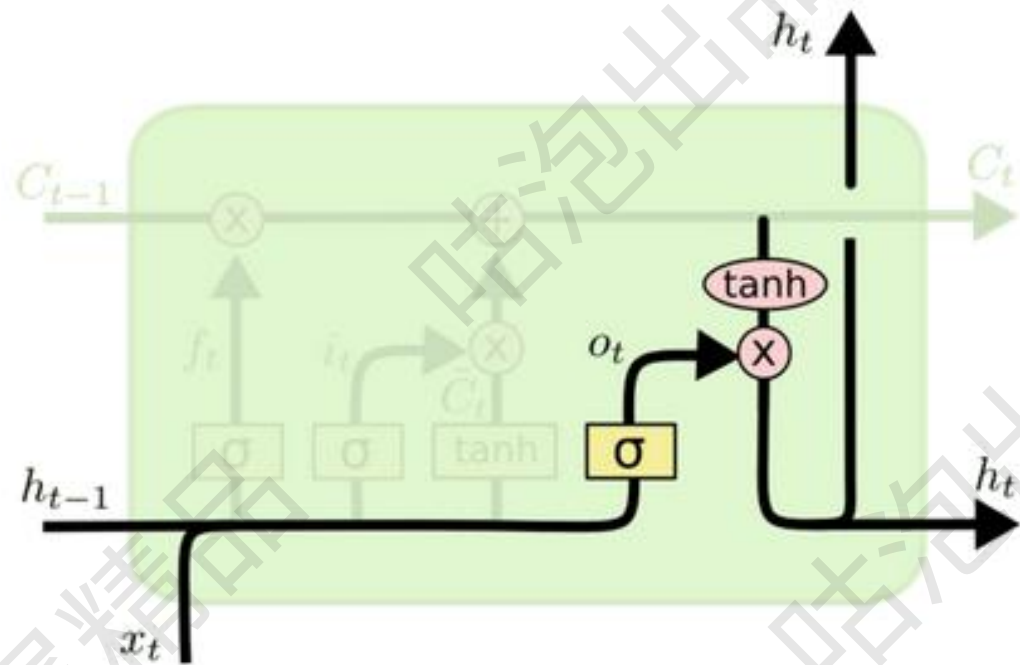
LSTM



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

更新细胞状态

LSTM



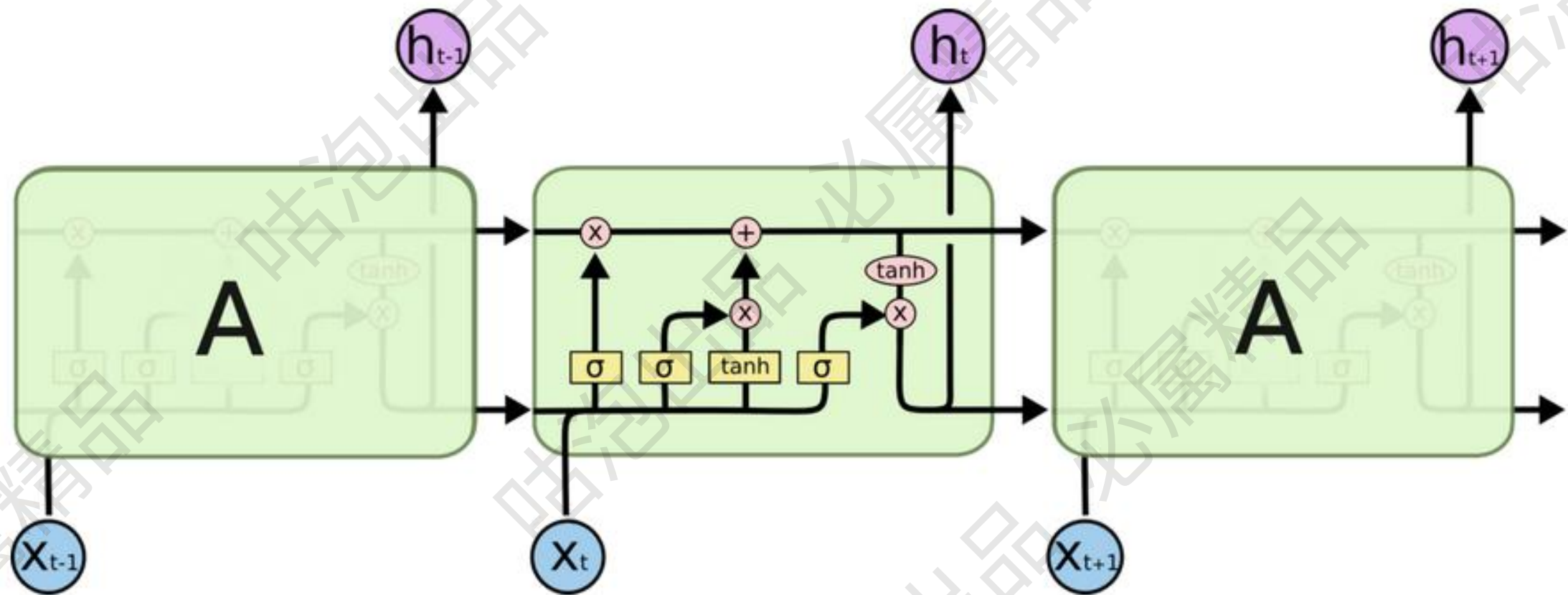
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

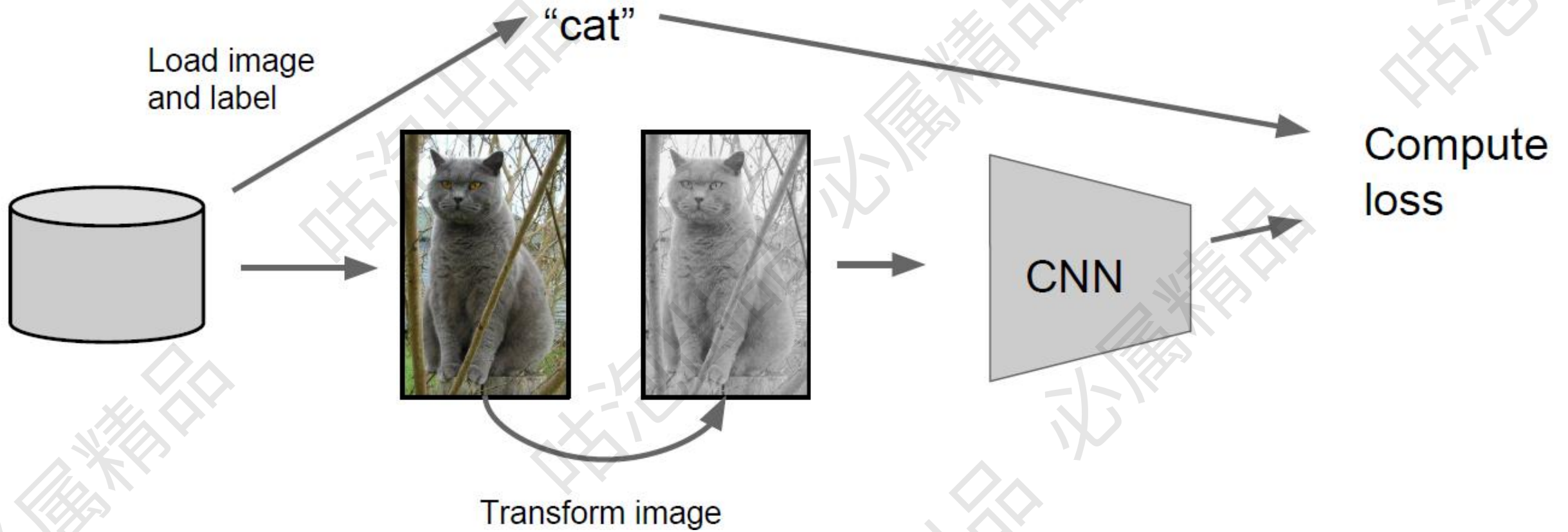
利用新的控制参数产生输出

输出信息

LSTM



Data Augmentation:



Data Augmentation:

- Change the pixels without changing the label
- Train on transformed data
- VERY widely used



08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	00	55
40	40	99	40	17	81	18	57	60	87	17	40	98	43	69	45	94	56	62	00
81	49	31	73	55	70	14	20	93	71	40	67	53	48	30	03	49	13	36	65
52	70	95	23	04	60	11	42	50	37	68	56	01	32	56	71	37	02	36	91
22	31	16	71	51	67	33	59	41	92	36	54	22	40	40	28	66	33	13	80
24	47	32	40	05	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	04	39	63	08	40	91	66	40	94	21
24	55	38	05	66	73	00	26	97	17	78	78	96	83	14	88	34	80	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	39	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	08	83	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
17	36	68	87	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	69
04	42	16	73	39	05	39	11	24	54	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	00	89	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	49	04	61	16	23	57	05	54
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	89	19	67	48

What the computer sees

Data Augmentation:

1. Horizontal flips



Data Augmentation:

2. Random crops/scales

Training: sample random crops / scales

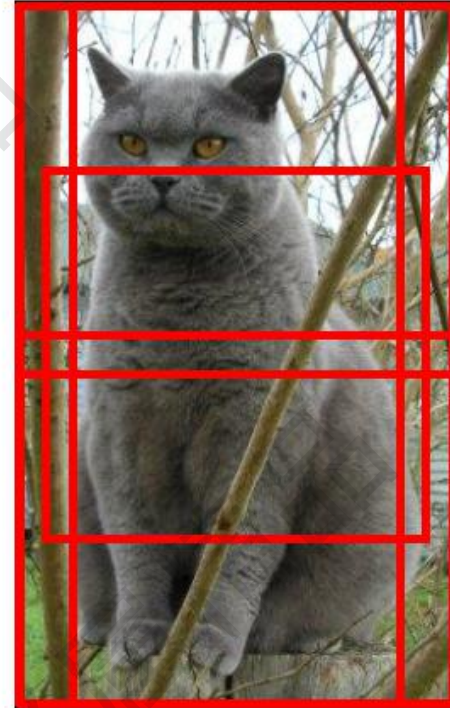
ResNet:

1. Pick random L in range $[256, 480]$
2. Resize training image, short side = L
3. Sample random 224×224 patch

Testing: average a fixed set of crops

ResNet:

1. Resize image at 5 scales: $\{224, 256, 384, 480, 640\}$
2. For each size, use 10 224×224 crops: 4 corners + center, + flips



Data Augmentation:

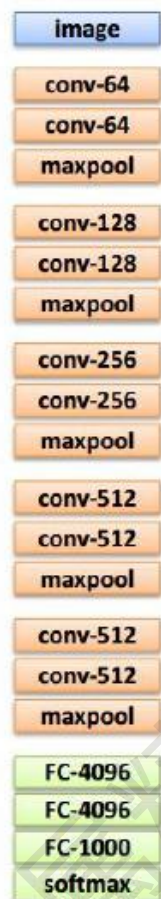
Random mix/combinations of :

- translation
- rotation
- stretching
- shearing,
- lens distortions, ... (go crazy)

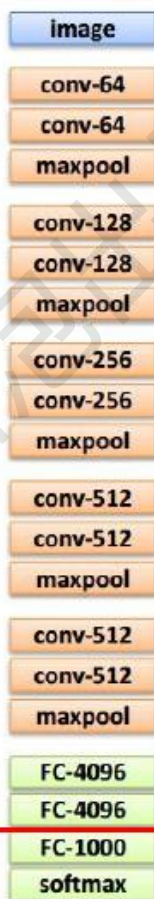
Transfer Learning:

“You need a lot of a data if you want to train/use CNNs”

Transfer Learning:



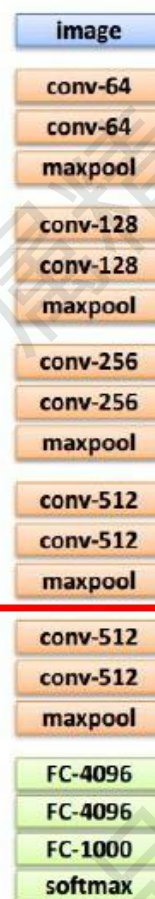
1. Train on
Imagenet



2. Small dataset:
feature extractor

Freeze these

Train this



3. Medium dataset:
finetuning

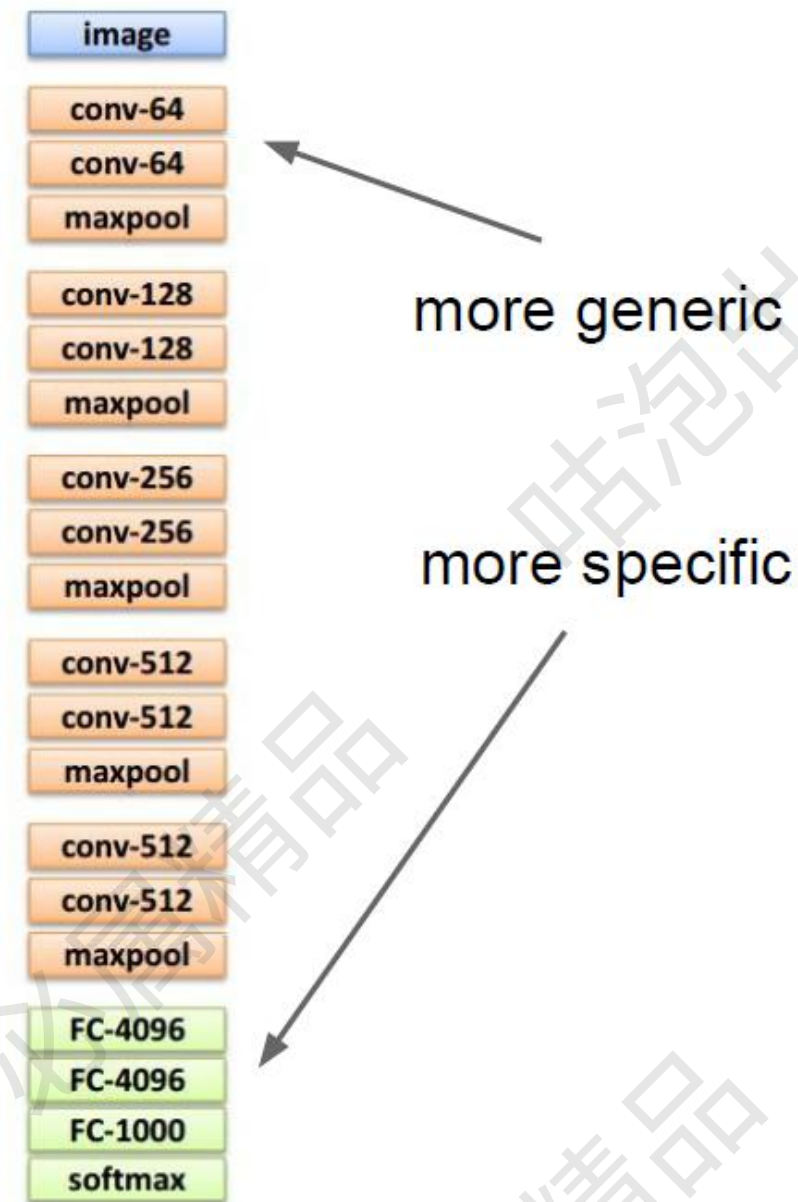
more data = retrain more of
the network (or all of it)

Freeze these

tip: use only $\sim 1/10$ th of
the original learning rate
in finetuning top layer,
and $\sim 1/100$ th on
intermediate layers

Train this

Transfer Learning:



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Transfer Learning:

Takeaway for your projects/beyond:

Have some dataset of interest but it has $< \sim 1\text{M}$ images?

1. Find a very large dataset that has similar data, train a big ConvNet there.
2. Transfer learn to your dataset

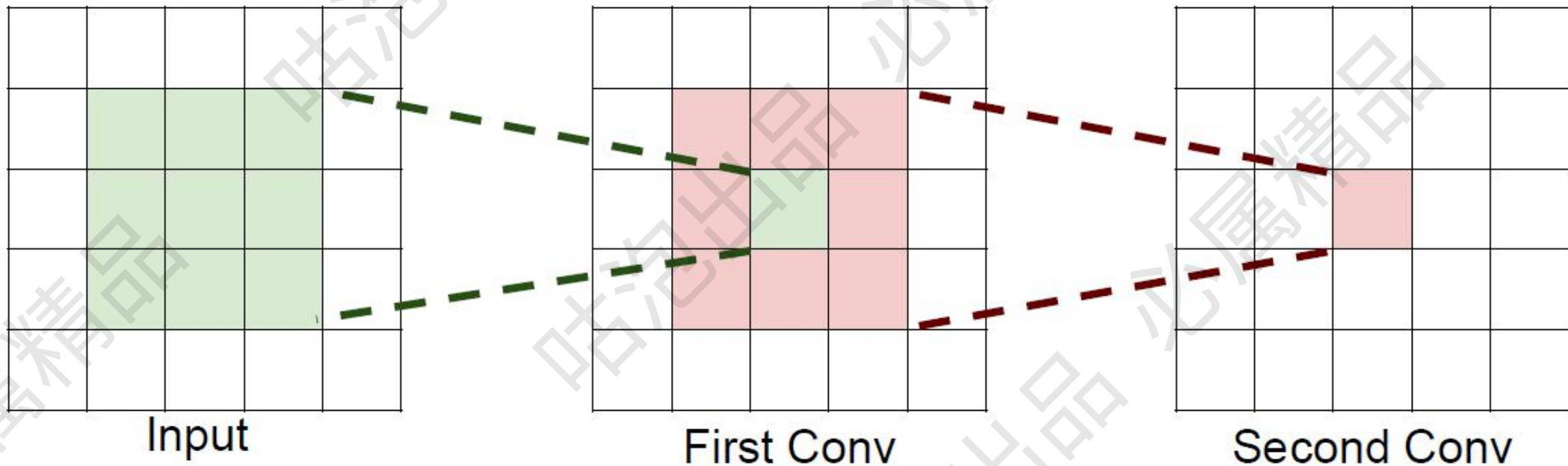
Caffe ConvNet library has a “**Model Zoo**” of pretrained models:

<https://github.com/BVLC/caffe/wiki/Model-Zoo>

Design Conv Network:

Suppose we stack two 3x3 conv layers (stride 1)

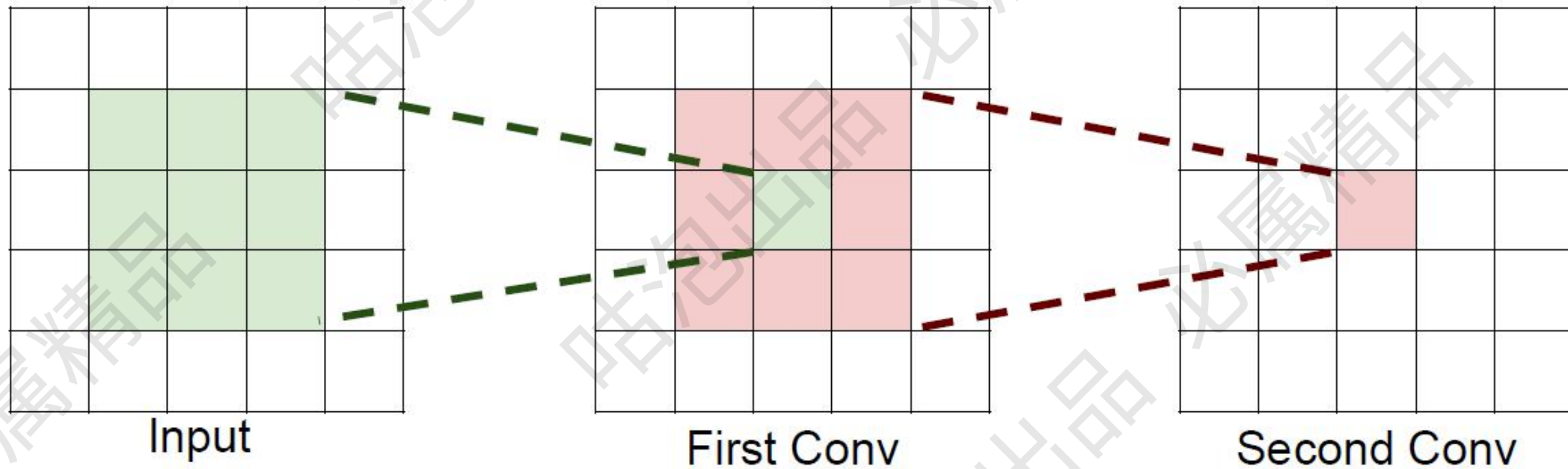
Each neuron sees 3x3 region of previous activation map



Design Conv Network:

Question: How big of a region in the input does a neuron on the second conv layer see?

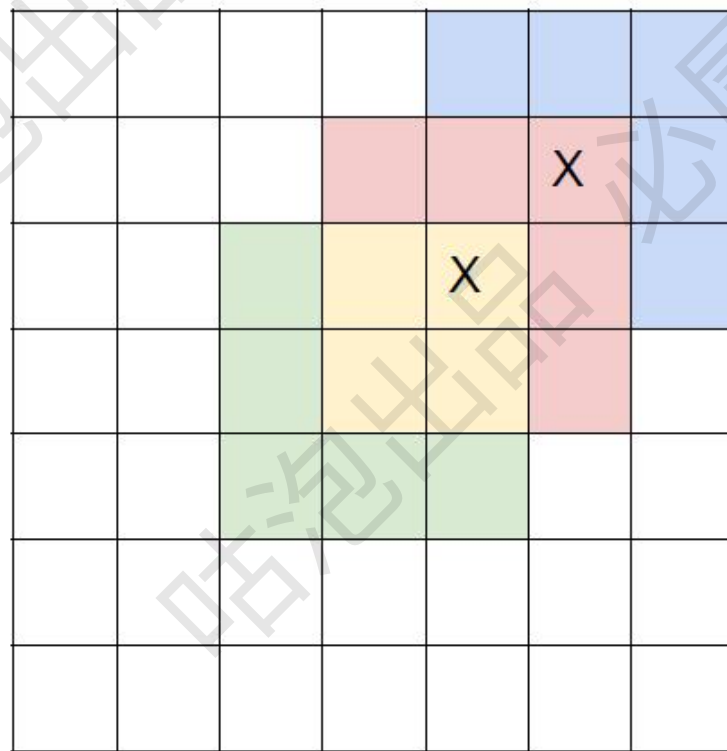
Answer: 5×5



Design Conv Network:

Question: If we stack **three** 3x3 conv layers, how big of an input region does a neuron in the third layer see?

Answer: 7 x 7



Three 3 x 3 conv
gives similar
representational
power as a single
7 x 7 convolution

Design Conv Network:

Suppose input is $H \times W \times C$ and we use convolutions with C filters to preserve depth (stride 1, padding to preserve H, W)

one CONV with 7×7 filters

Number of weights:

$$= C \times (7 \times 7 \times C) = 49 C^2$$

three CONV with 3×3 filters

Number of weights:

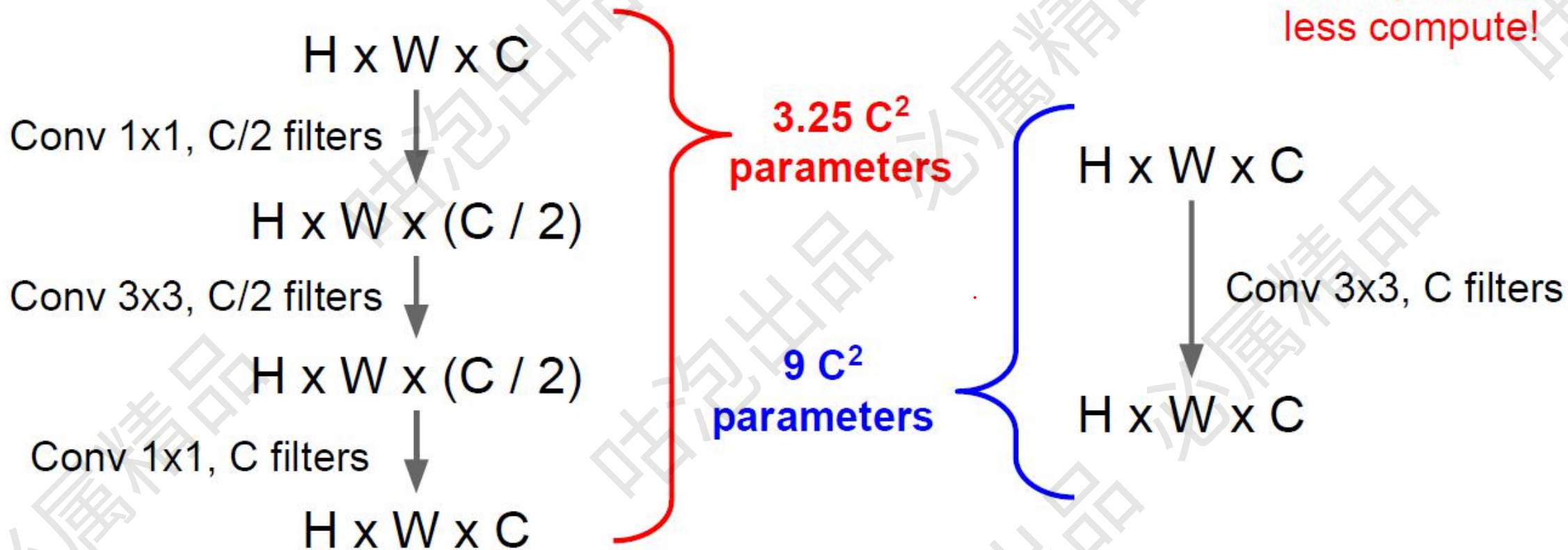
$$= 3 \times C \times (3 \times 3 \times C) = 27 C^2$$

Fewer parameters, more nonlinearity = GOOD

Design Conv Network:

Why stop at 3 x 3 filters? Why not try 1 x 1?

More nonlinearity,
fewer params,
less compute!



Design Conv Network:

- Replace large convolutions (5×5 , 7×7) with stacks of 3×3 convolutions
- 1×1 “bottleneck” convolutions are very efficient
- Can factor $N \times N$ convolutions into $1 \times N$ and $N \times 1$
- All of the above give fewer parameters, less compute, more nonlinearity

GPU VS CPU:

Spot the CPU!
“central processing unit”



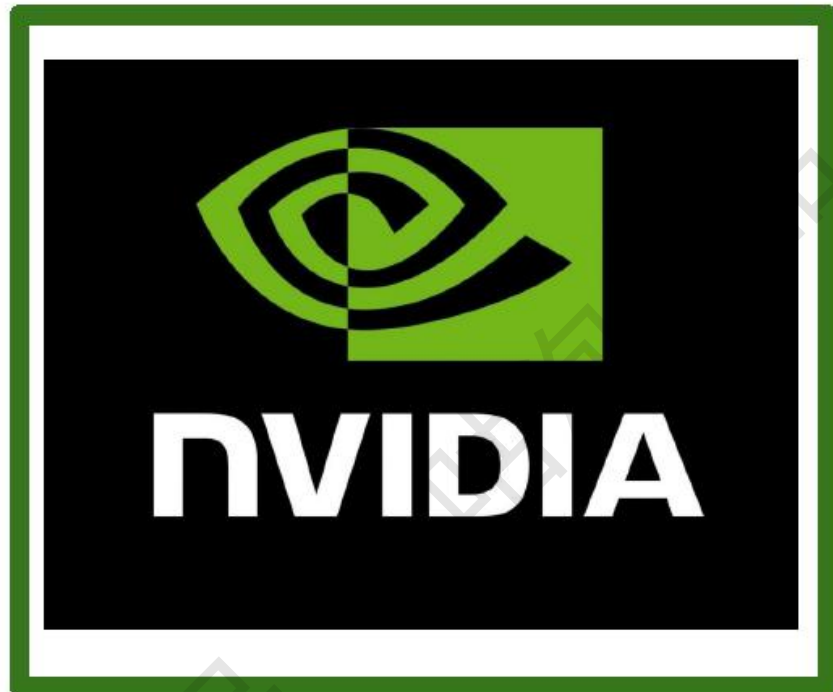
GPU VS CPU:

Spot the GPU!

“graphics processing unit”



GPU VS CPU:



VS



NVIDIA is much more
common for deep learning

GPU VS CPU:

CPU

Few, fast cores (1 - 16)

Good at sequential processing



GPU

Many, slower cores (thousands)

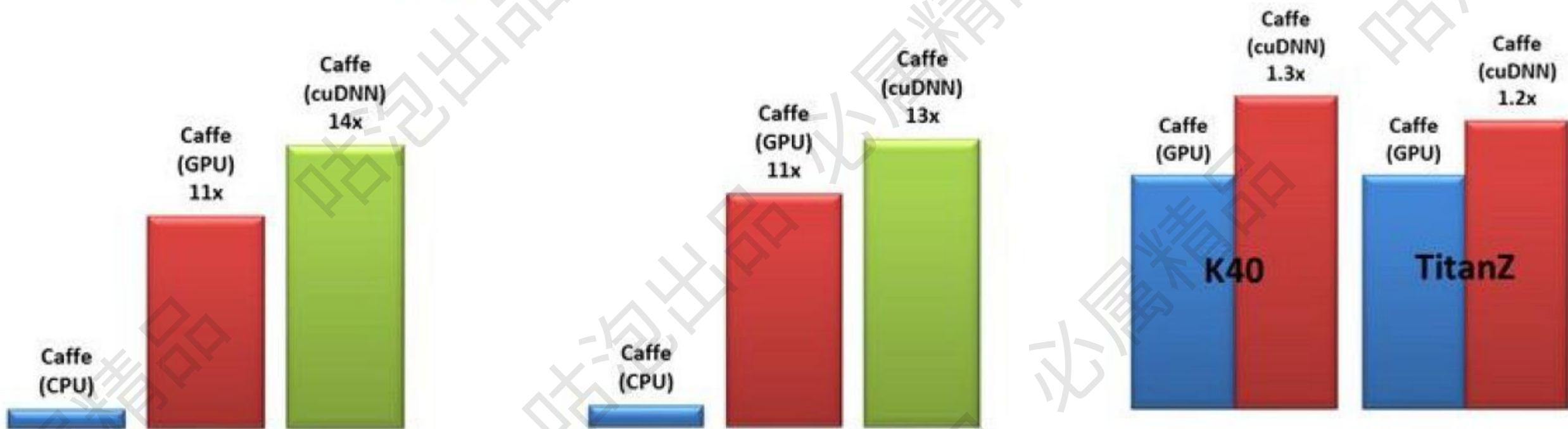
Originally for graphics

Good at parallel computation



GPU VS CPU:

GPUs are really good at convolution (cuDNN):



Caffe:

Caffe

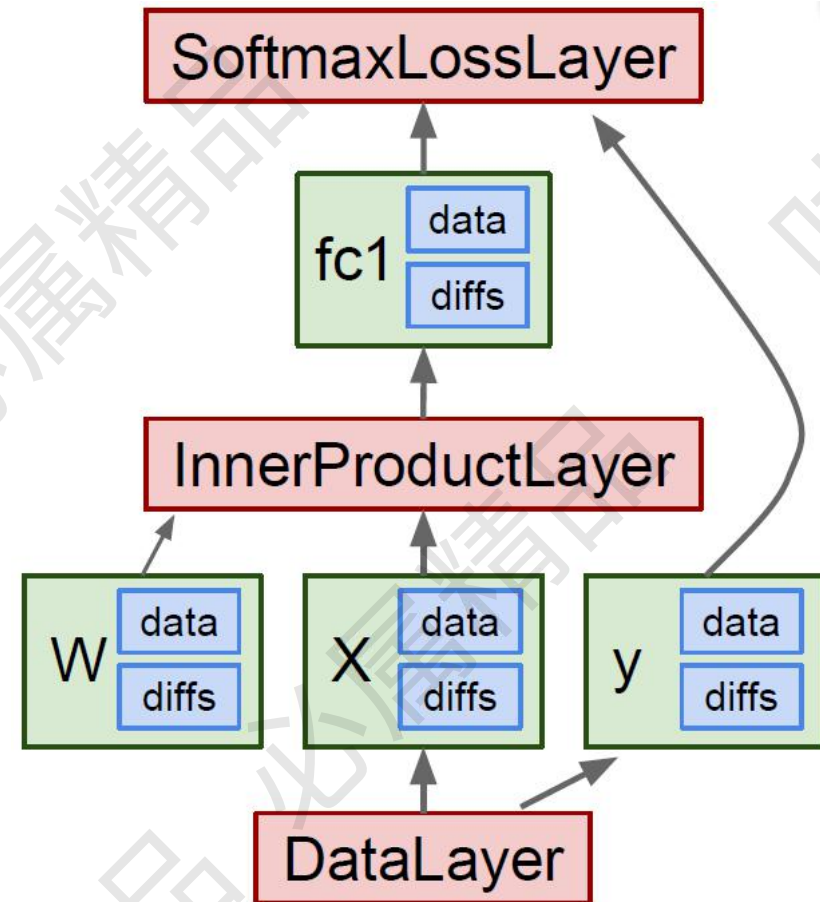
<http://caffe.berkeleyvision.org>

Caffe Overview:

- From U.C. Berkeley
- Written in C++
- Has Python and MATLAB bindings
- Good for training or finetuning feedforward models

Caffe Main Classes:

- **Blob**: Stores data and derivatives ([header source](#))
- **Layer**: Transforms bottom blobs to top blobs ([header + source](#))
- **Net**: Many layers; computes gradients via forward / backward ([header source](#))
- **Solver**: Uses gradients to update weights ([header source](#))



Caffe Training:

No need to write code!

1. Convert data (run a script)
2. Define net (edit prototxt)
3. Define solver (edit prototxt)
4. Train (with pretrained weights) (run a script)

Caffe Step 1: Convert Data

- DataLayer reading from LMDB is the easiest
- Create LMDB using [convert_imageset](#)
- Need text file where each line is
 - “[path/to/image.jpeg] [label]”
- Create HDF5 file yourself using h5py

Caffe Step 2: Define Net

```
name: "LogisticRegressionNet"
layers {
  top: "data"
  top: "label"
  name: "data"
  type: HDF5_DATA
  hdf5_data_param {
    source: "examples/hdf5_classification/data/train.txt"
    batch_size: 10
  }
  include {
    phase: TRAIN
  }
}
layers {
  bottom: "data"
  top: "fc1"
  name: "fc1"
  type: INNER_PRODUCT
  blobs_lr: 1
  blobs_lr: 2
  weight_decay: 1
  weight_decay: 0
```

```
inner_product_param {
  num_output: 2
  weight_filler {
    type: "gaussian"
    std: 0.01
  }
  bias_filler {
    type: "constant"
    value: 0
  }
}
layers {
  bottom: "fc1"
  bottom: "label"
  top: "loss"
  name: "loss"
  type: SOFTMAX_LOSS
}
```

Caffe Step 3: Define Solver

- Write a prototxt file defining a [SolverParameter](#)
- If finetuning, copy existing solver. prototxt file
 - Change net to be your net
 - Change snapshot_prefix to your output
 - Reduce base learning rate (divide by 100)
 - Maybe change max_iter and snapshot

```
1 net: "models/bvlc_alexnet/train_val.prototxt"
2 test_iter: 1000
3 test_interval: 1000
4 base_lr: 0.01
5 lr_policy: "step"
6 gamma: 0.1
7 stepsize: 100000
8 display: 20
9 max_iter: 450000
10 momentum: 0.9
11 weight_decay: 0.0005
12 snapshot: 10000
13 snapshot_prefix: "models/bvlc_alexnet/caffe_alexnet_train"
14 solver_mode: GPU
```

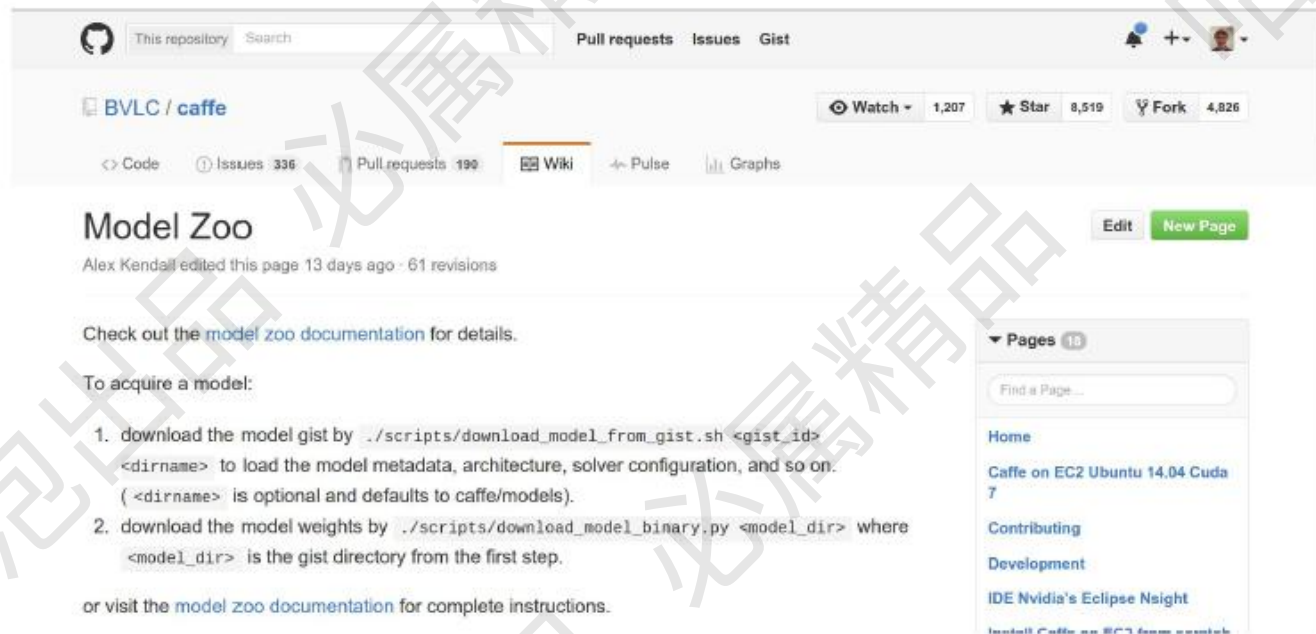
Caffe Step 4: Train!

```
./build/tools/caffe train \  
-gpu 0 \  
-model path/to/trainval.prototxt \  
-solver path/to/solver.prototxt \  
-weights path/to/pretrained_weights.caffemodel
```

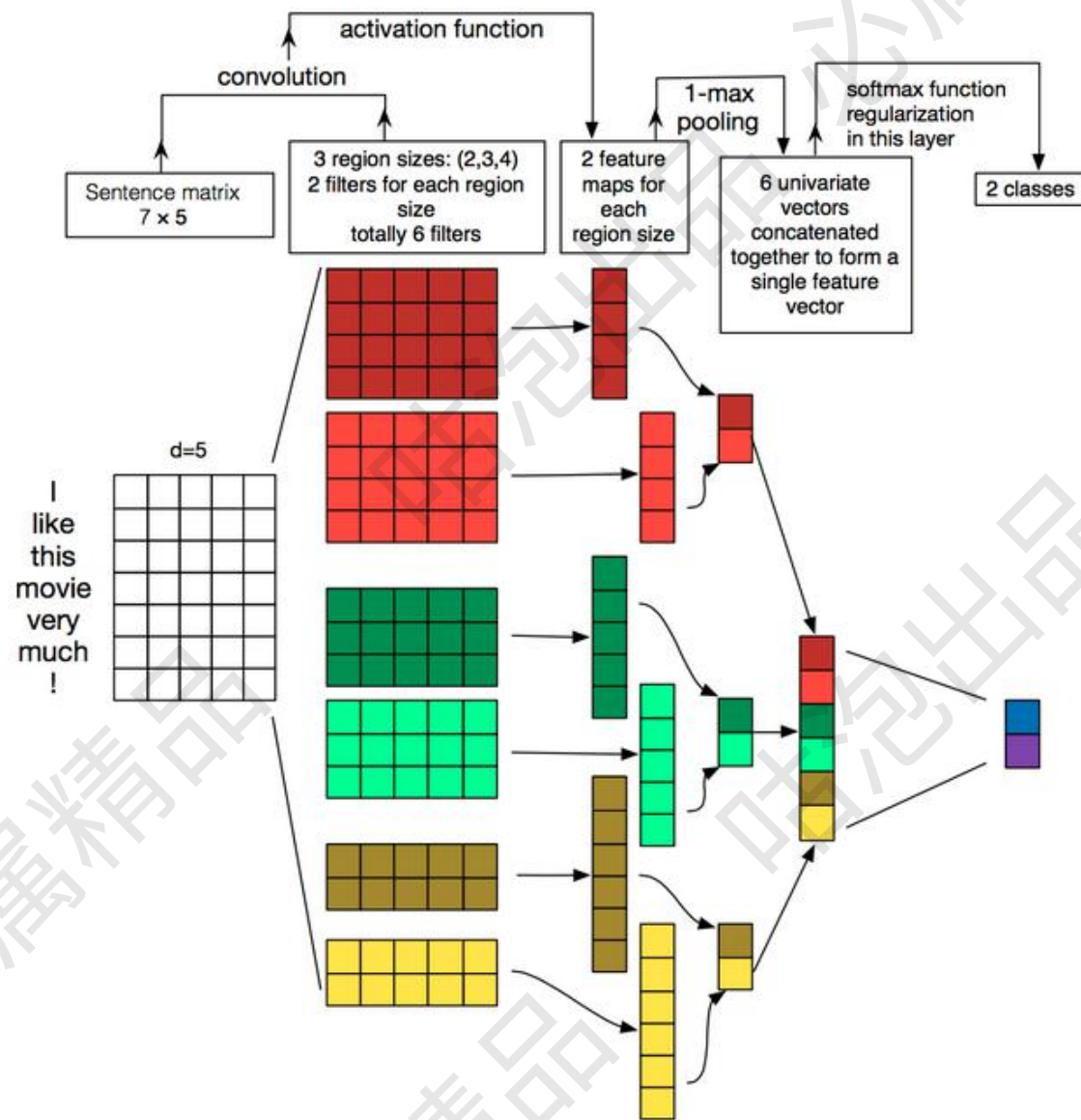
Caffe:

Caffe: Model Zoo

AlexNet, VGG,
GoogLeNet, ResNet,
plus others



CNN文本分类



Seq2Seq

✓ 机器翻译的历史

✎ 最早期的逐字翻译

Quiero ir a la playa más bonita.

I want to go to the beach more pretty.

Seq2Seq

✓ 机器翻译的历史

✎ 统计学的机器翻译

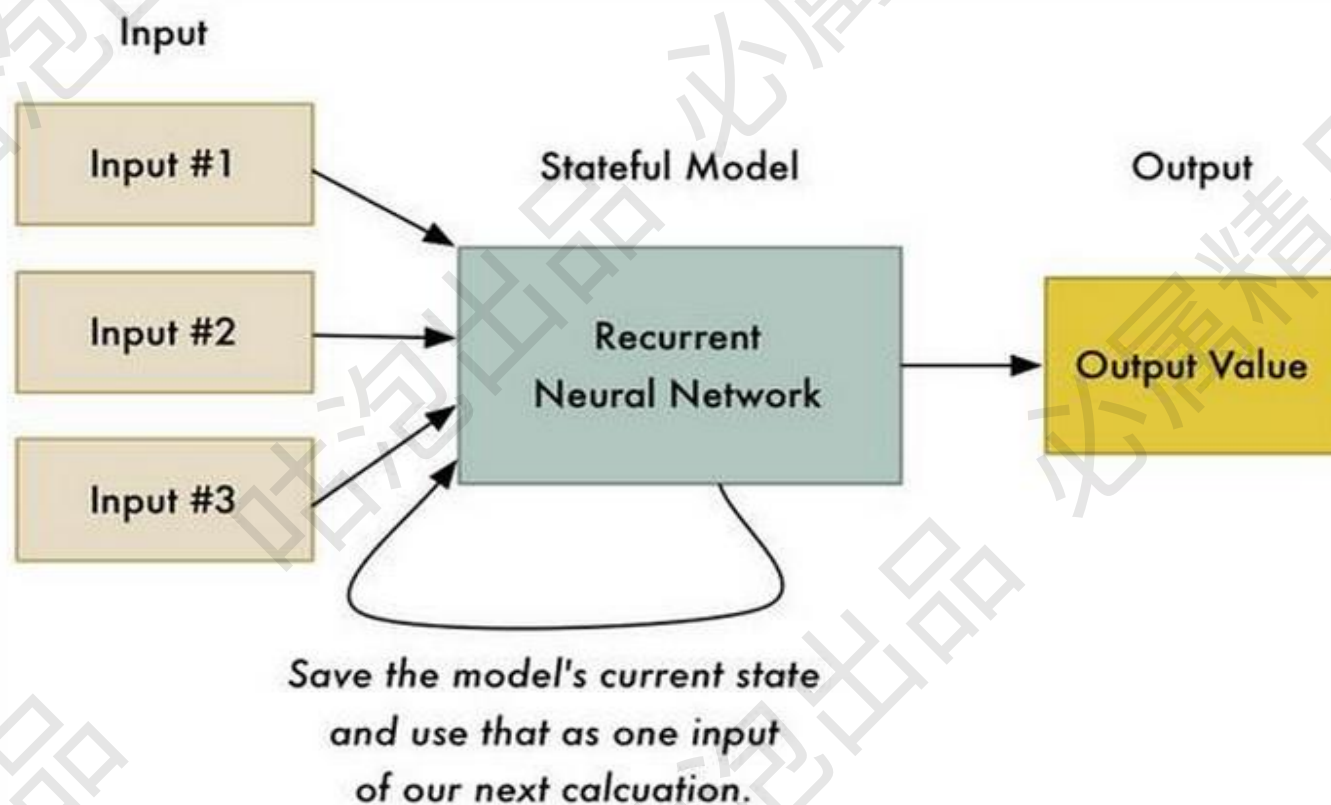
Quiero ir a la playa más bonita.

-
- | | | | | |
|----------|----------------|-------|------------------|---------------|
| - I want | - to go | - to | - the beach | - more pretty |
| - I love | - to work | - at | - the seaside | - most pretty |
| - I like | - to run | - per | - the open space | - more lovely |
| - I try | - to appear | | | - most lovely |
| - I mean | - to be on | | | - more tidy |
| | - to be | | | - most tidy |
| | - to leave | | | |
| | - to pass away | | | |
| | - to forget | | | |

Seq2Seq

✓ 机器翻译的历史

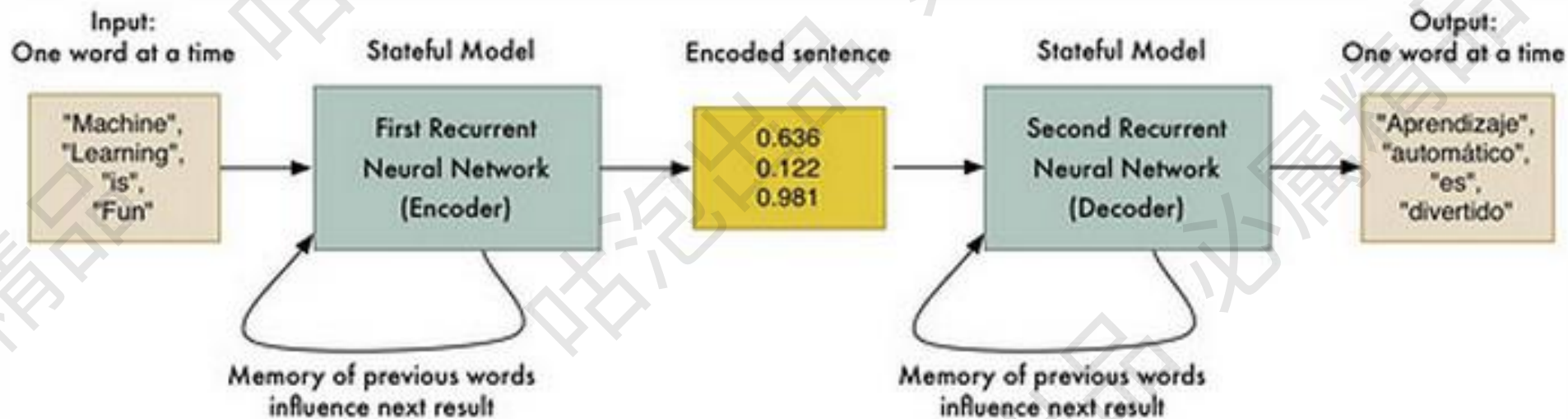
📎 循环网络和编码



Seq2Seq

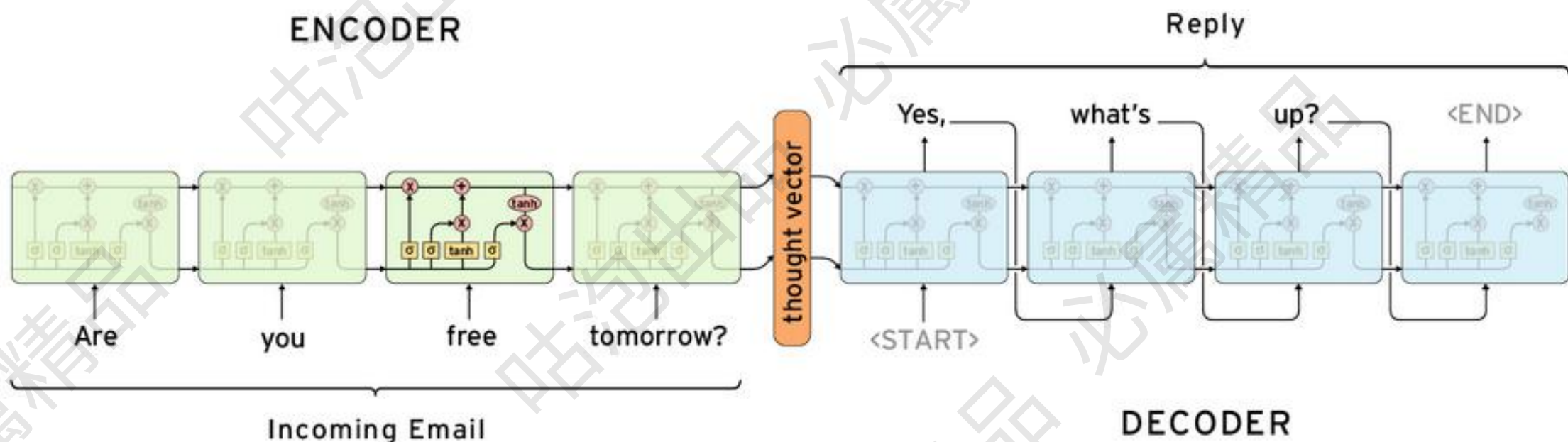
✓ 机器翻译的历史

✎ 解码与翻译



Seq2Seq

✓ Seq2Seq网络架构 (sequence-to-sequence)



Seq2Seq

✓ Seq2Seq应用

 机器翻译

English Spanish French Detect language ▾

I'm definitely not using Google Translate to do
my I



Spanish English Romanian ▾

Translate

Definitivamente no estoy usando Google
Translate para



Seq2Seq

✓ Seq2Seq应用

✎ 文本摘要

Good quality summary output

S: a man charged with the murder last year of a british backpacker confessed to the slaying on the night he was charged with her killing , according to police evidence presented at a court hearing tuesday . ian douglas previte , ## , is charged with murdering caroline stuttle , ## , of yorkshire , england

T: man charged with british backpacker 's death confessed to crime police officer claims

O: man charged with murdering british backpacker confessed to murder

S: following are the leading scorers in the english premier league after saturday 's matches : ## - alan shearer -lrb- newcastle united -rrb- , james beattie .

T: leading scorers in english premier league

O: english premier league leading scorers

S: volume of transactions at the nigerian stock exchange has continued its decline since last week , a nse official said thursday . the latest statistics showed that a total of ##.### million shares valued at ###.### million naira -lrb- about #.### million us dollars -rrb- were traded on wednesday in #,### deals .

T: transactions dip at nigerian stock exchange

O: transactions at nigerian stock exchange down

Seq2Seq

✓ Seq2Seq应用

✎ 情感对话生成

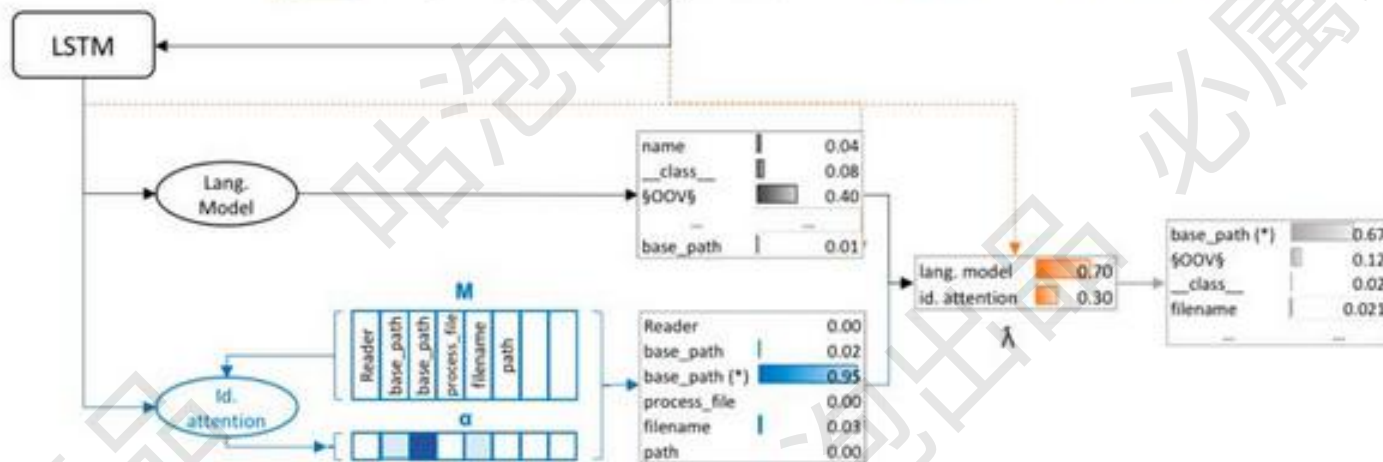
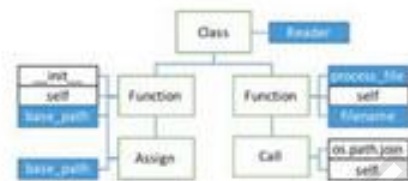
Post	Model		Response(Original)	Response(Translated)
比情人节更庸俗的事就是， 对情人节的接待规格待遇提出 要求。 What is more vulgar than Valentine's day is to make specific demands on the reception of the Valentine's day.	Seq2Seq		这才是真正的情人节。	This is a real Valentine's day.
	ECM	Like	情人节快乐！	Happy Valentine's Day!
		Happiness	哈哈，这也太浪漫了吧！	Aha, this is too romantic!
		Sadness	我也想要这样的情人节。	I also want this kind of Valentine's day, sigh.
		Disgust	这就是所谓的情人节！	This is the so-called Valentine's day!
		Anger	这是赤裸裸的炫耀啊！	This is shameless show-off!
知道真相的我眼泪掉下来。 Knowing the truth, I laughed myself to tears.	Seq2Seq		知道真相的我眼泪掉下来。	After knowing the truth, I burst into tears.
	ECM	Like	太可爱了，太可爱了！	Too lovely, it's too lovely.
		Happiness	笑得我眼泪都出来了。	I can't help laugh myself to tears.
		Sadness	知道真相的我眼泪掉下来。	After knowing the truth, I burst into tears.
		Disgust	这是真的还是假的？	Is this true?
		Anger	什么意思？没看懂。	What do you mean? I don't understand.

Seq2Seq

✓ Seq2Seq应用

📎 代码补全

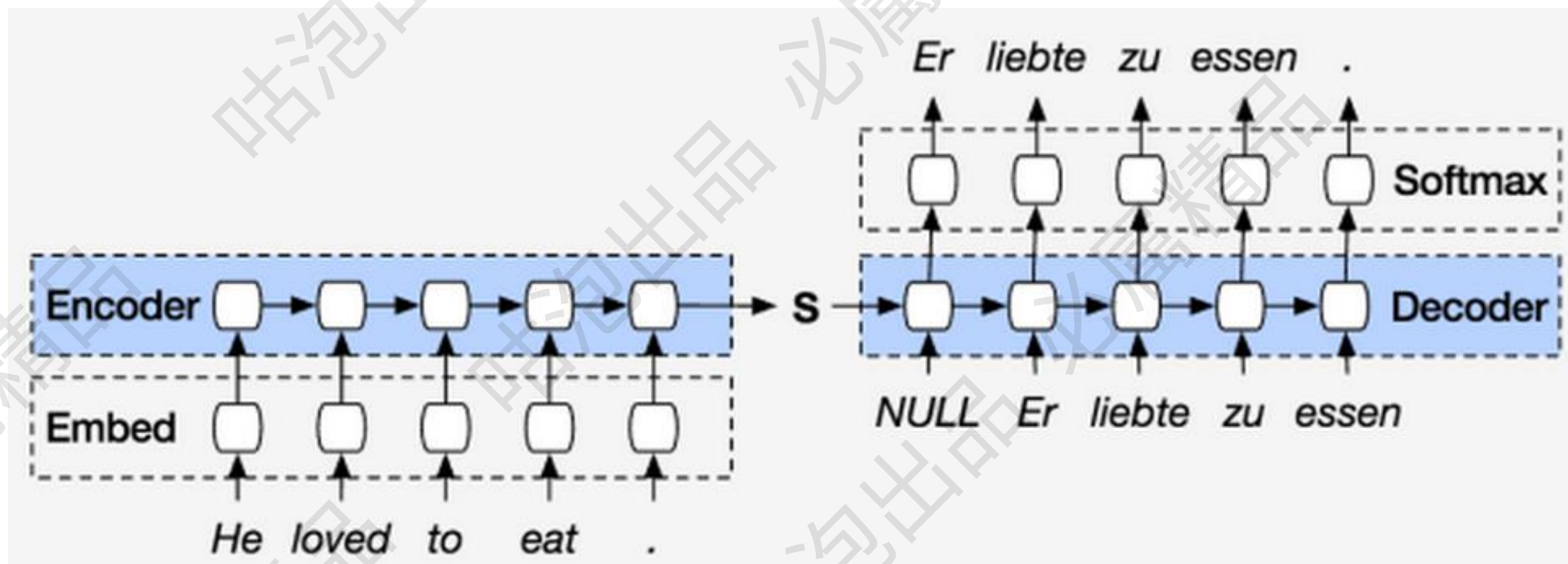
```
import os
class Reader:
    def __init__(self, base_path):
        self.base_path = base_path
    def process_file(self, filename):
        path = os.path.join(self. ?
```



Seq2Seq

✓ Seq2Seq存在的问题

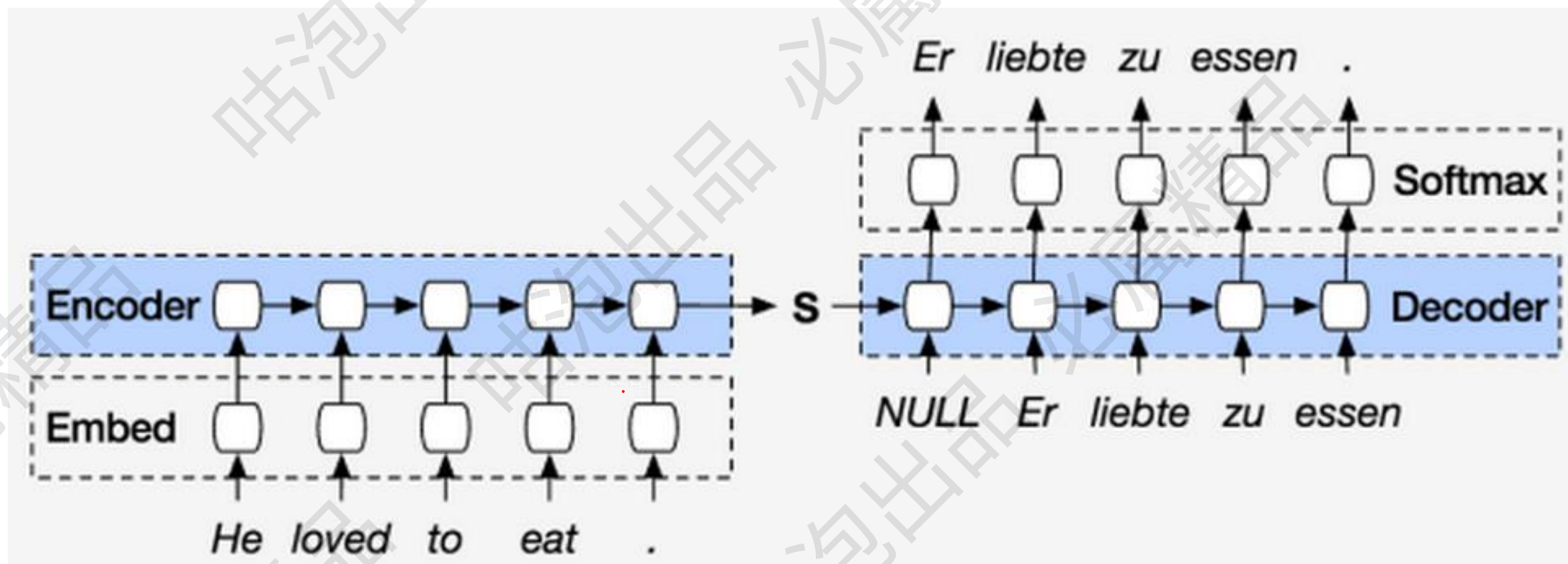
✎ 压缩损失了信息



Seq2Seq

✓ Seq2Seq存在的问题

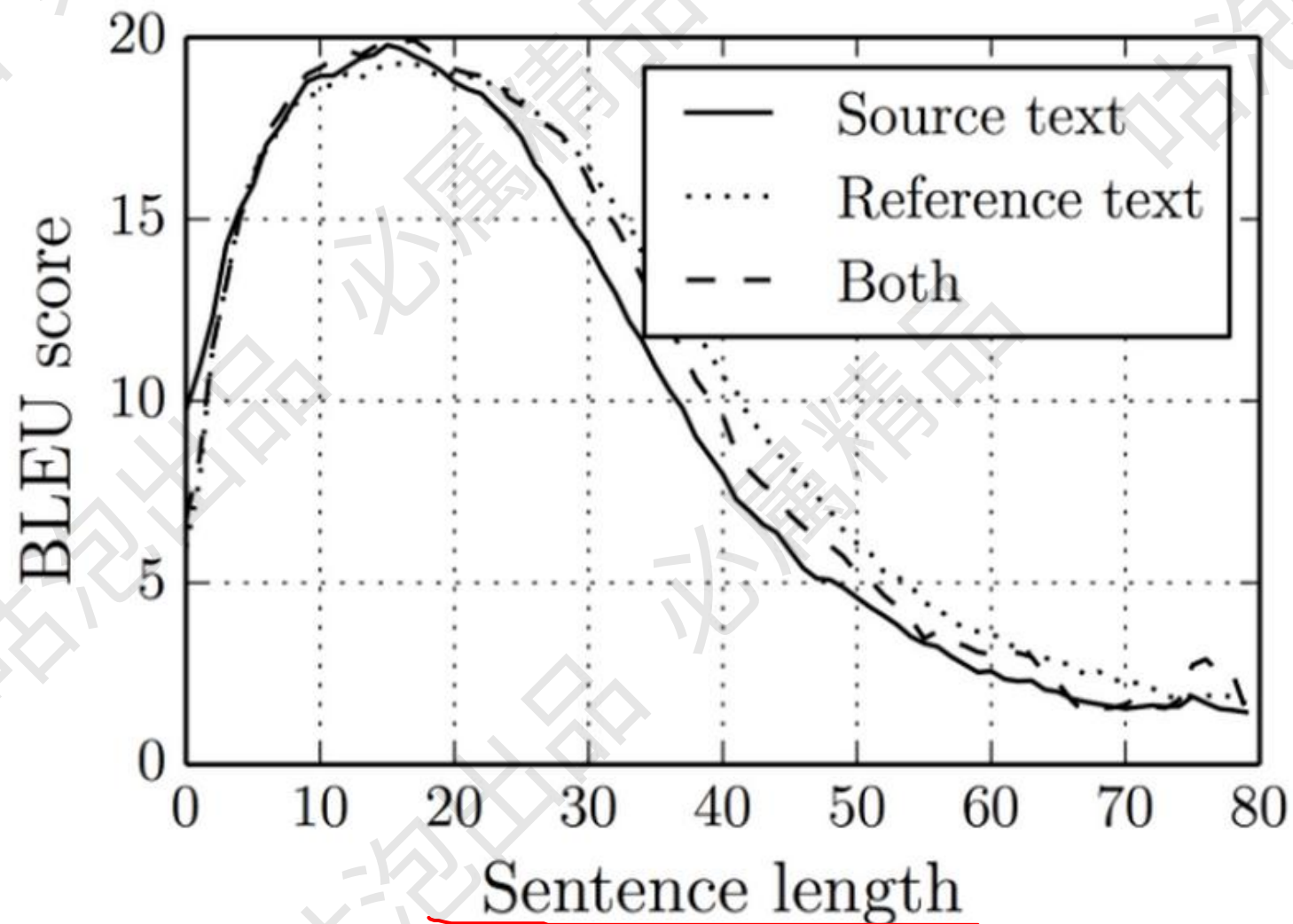
📎 压缩损失了信息



Seq2Seq

✓ Seq2Seq存在的问题

✎ 长度限制



Seq2Seq

✓ Attention机制

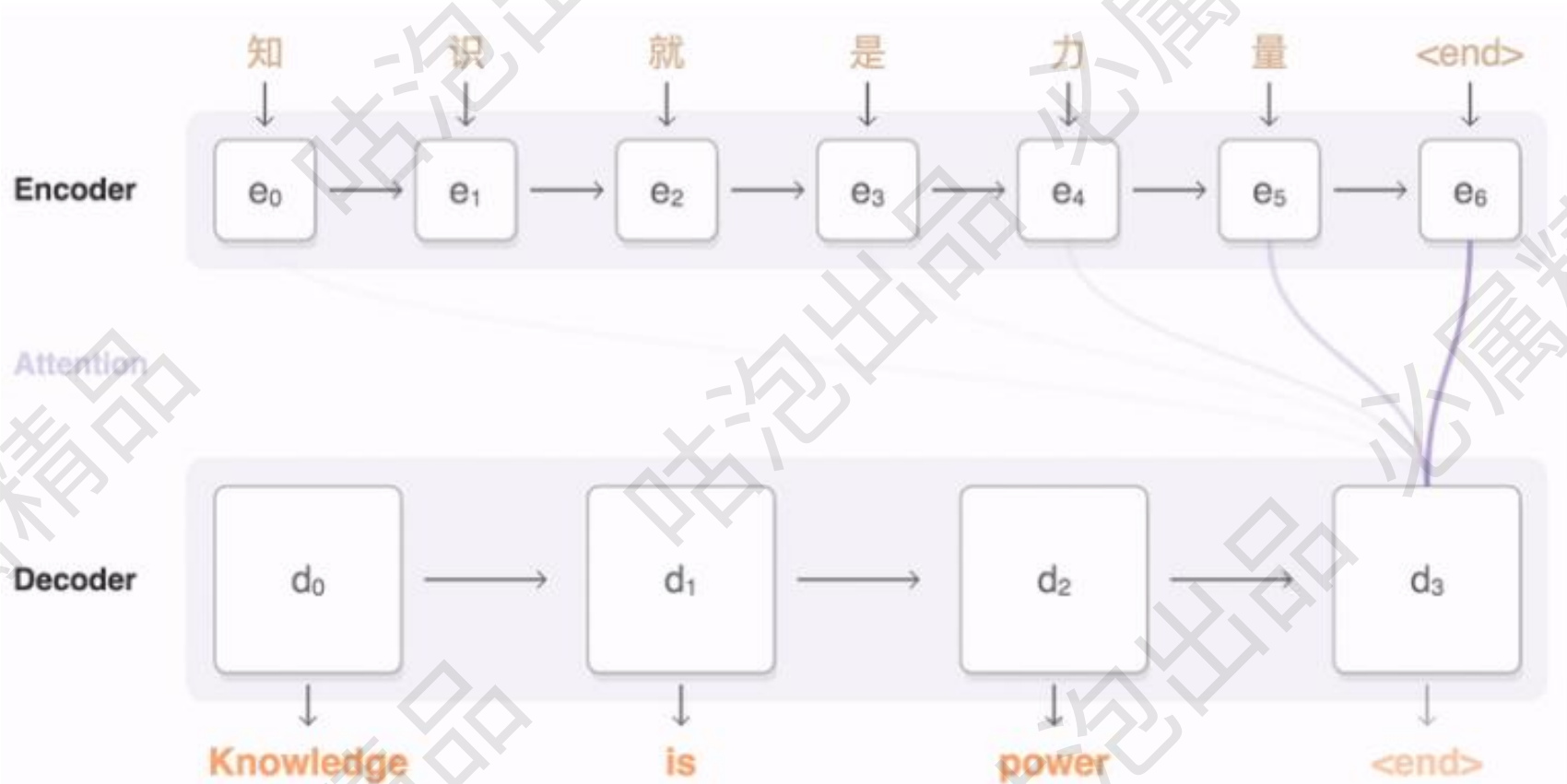
✎ “高分辨率” 聚焦在图片的某个特定区域并以 “低分辨率” 感知图像的周边区域的模式

✎ 通过大量实验证明，将attention机制应用在机器翻译，摘要生成，阅读理解等问题上，取得的成效显著

Seq2Seq

✓ Attention机制

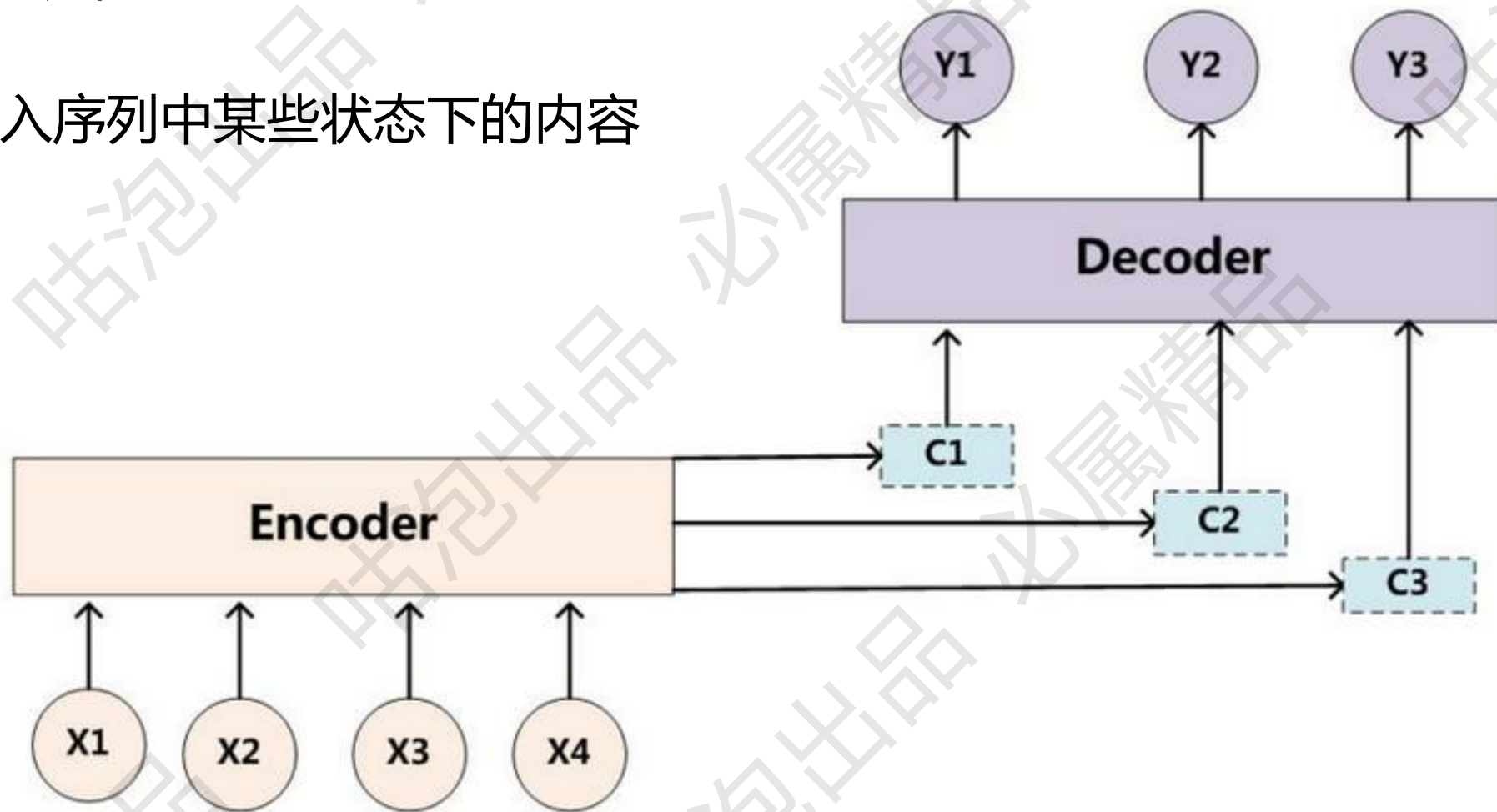
📌 关注输入序列中某些状态下的内容



Seq2Seq

✓ Attention机制

✎ 关注输入序列中某些状态下的内容



Seq2Seq

✓ 机器翻译: Cats eat mice $y_1=f(C_1), y_2=f(C_2, y_1), y_3=f(C_3, y_1, y_2)$

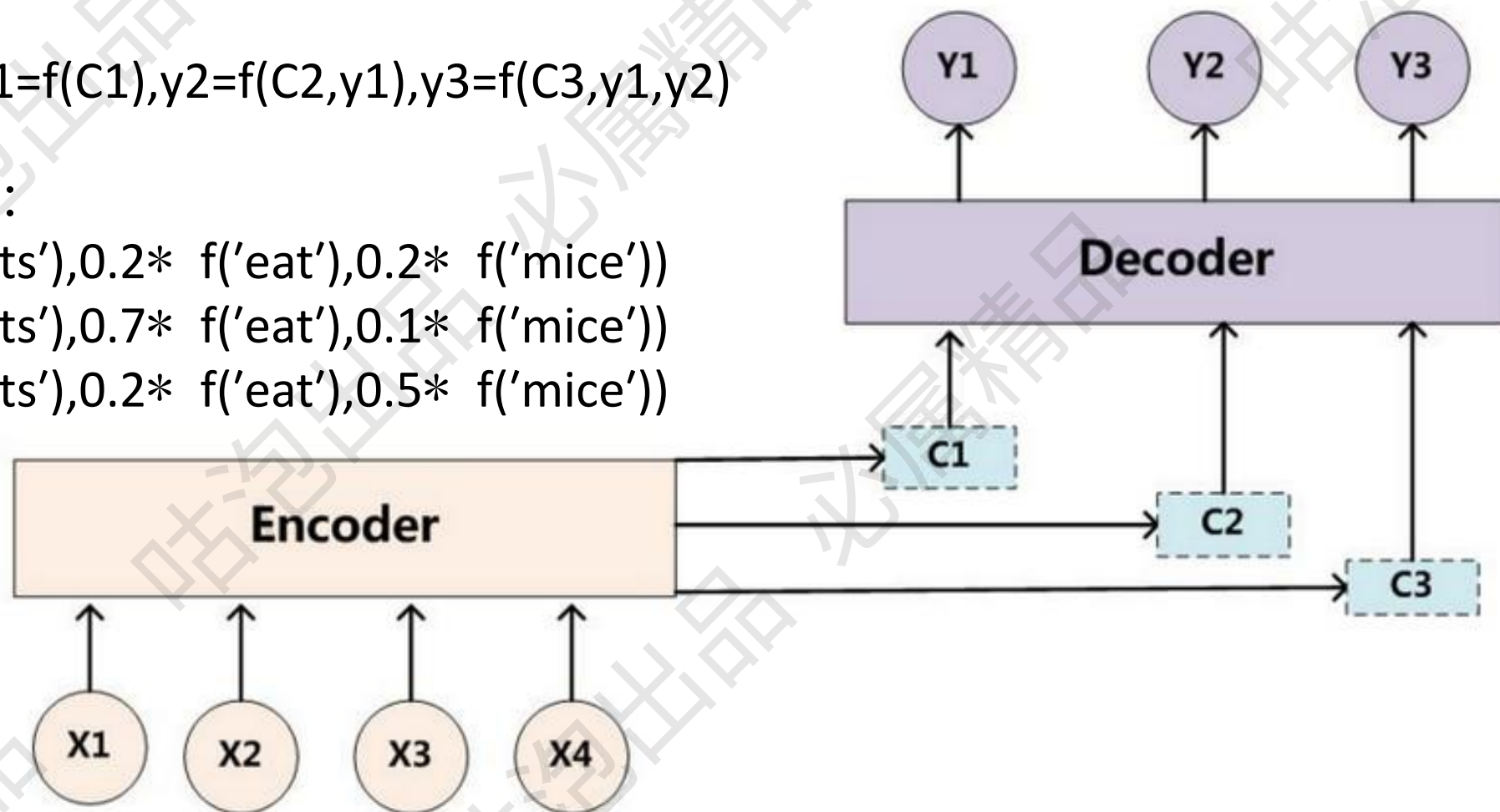
✎ 无Attention: $y_1=f(C_1), y_2=f(C_2, y_1), y_3=f(C_3, y_1, y_2)$

加入Attention:

$C_1 = g(0.6 * f('Cats'), 0.2 * f('eat'), 0.2 * f('mice'))$

$C_2 = g(0.2 * f('Cats'), 0.7 * f('eat'), 0.1 * f('mice'))$

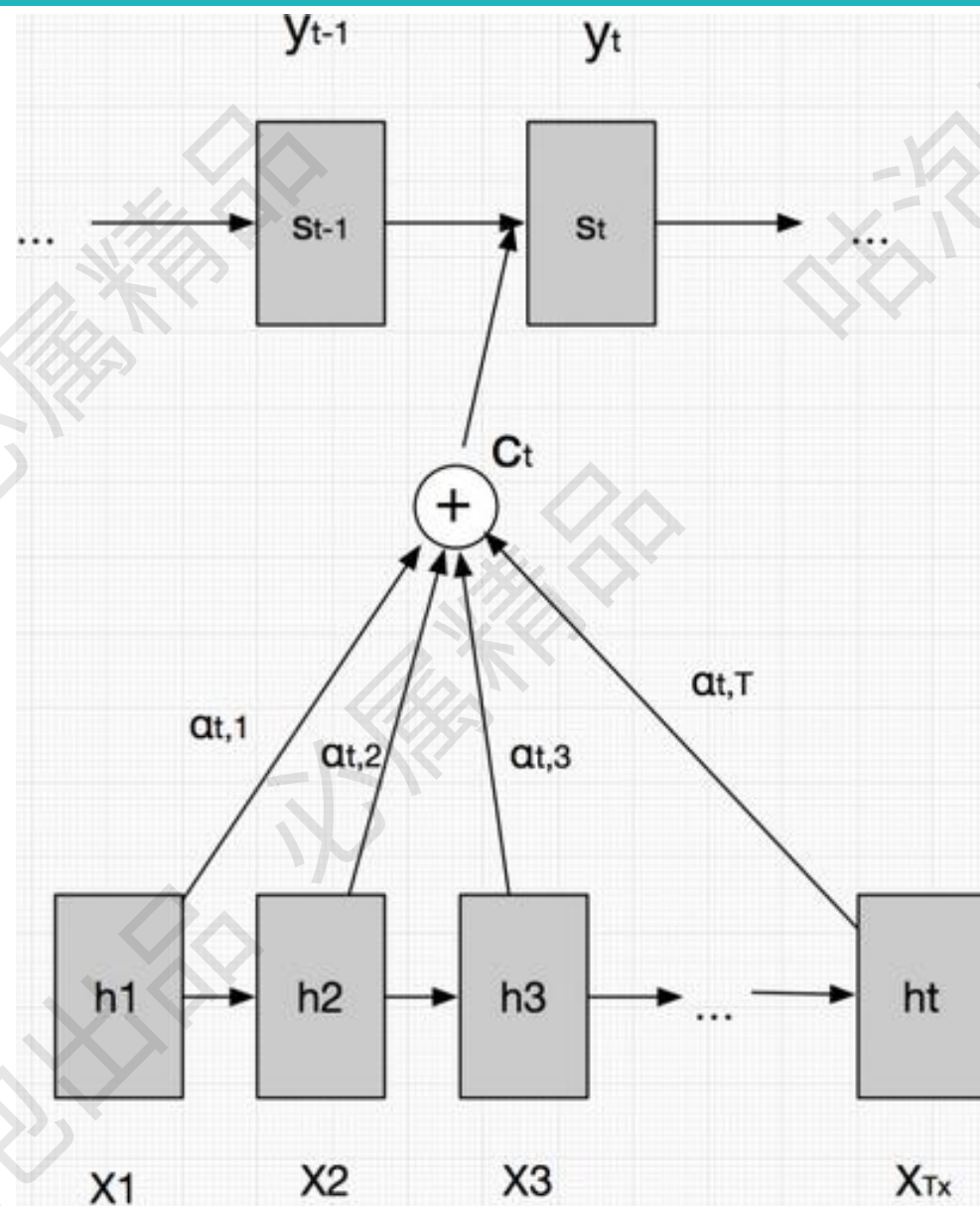
$C_3 = g(0.3 * f('Cats'), 0.2 * f('eat'), 0.5 * f('mice'))$



Seq2Seq

✓ Attention机制

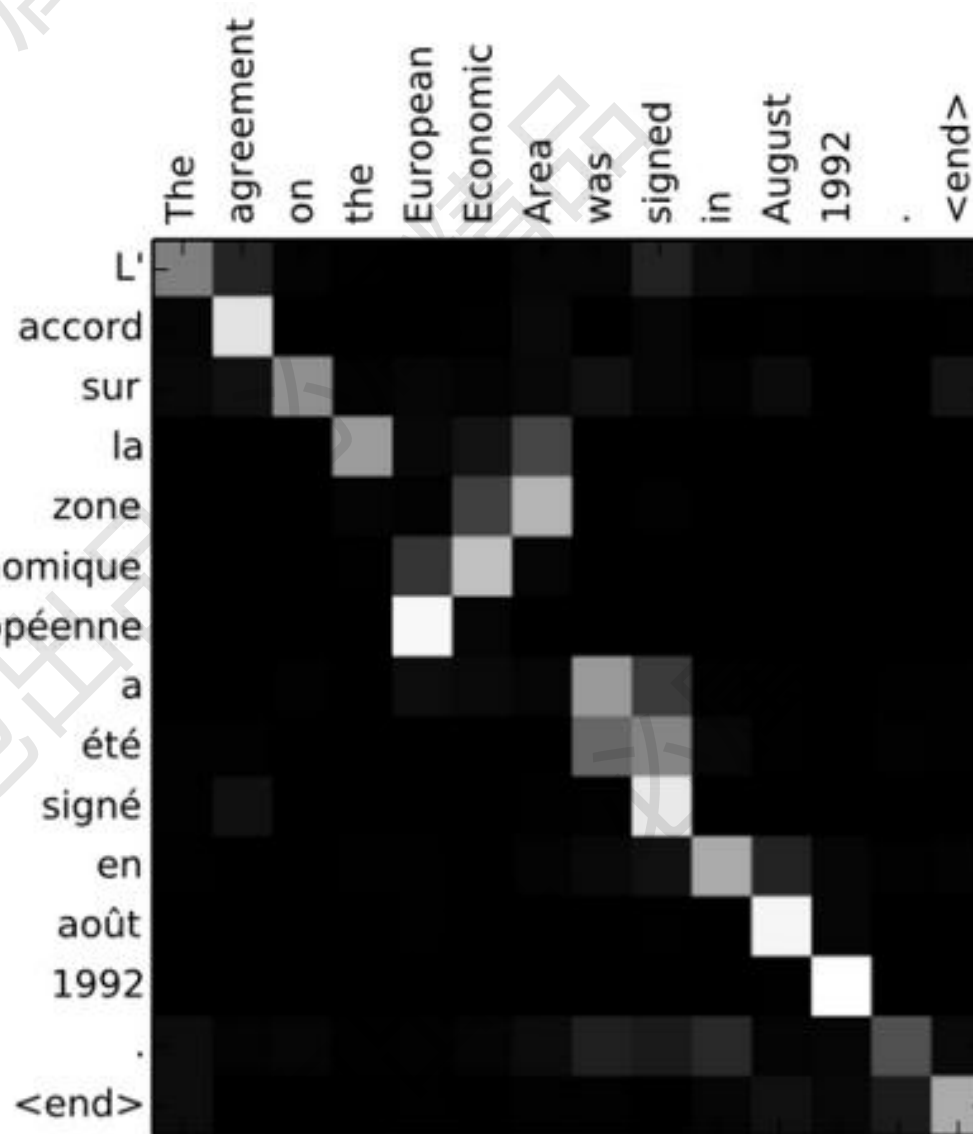
✎ 对Encoder层状态的加权
从而掌握输入语句中的所有细节信息



[illegible]

✓ Attention机制

加权效果



语言模型

✓ 语言模型实例：

我 今 天 下 午 打 篮 球

$$\begin{aligned} p(S) &= p(w_1, w_2, w_3, w_4, w_5, \dots, w_n) \\ &= p(w_1) p(w_2|w_1) p(w_3|w_1, w_2) \dots p(w_n|w_1, w_2, \dots, w_{n-1}) \end{aligned}$$

$p(S)$ 被称为语言模型，即用来计算一个句子概率的模型

语言模型

✓ 语言模型存在哪些问题呢?

$$p(w_i|w_1, w_2, \dots, w_{i-1}) = p(w_1, w_2, \dots, w_{i-1}, w_i) / p(w_1, w_2, \dots, w_{i-1})$$



1.数据过于稀疏

2.参数空间太大

语言模型

假设下一个词的出现依赖它前面的一个词：

$$\begin{aligned} p(S) &= p(w_1)p(w_2|w_1)p(w_3|w_1,w_2)\dots p(w_n|w_1,w_2,\dots, \\ &\quad w_{n-1}) \\ &= p(w_1)p(w_2|w_1)p(w_3|w_2)\dots p(w_n|w_{n-1}) \end{aligned}$$

假设下一个词的出现依赖它前面的两个词：

$$\begin{aligned} p(S) &= p(w_1)p(w_2|w_1)p(w_3|w_1,w_2)\dots p(w_n|w_1,w_2,\dots,w_{n-1}) \\ &= p(w_1)p(w_2|w_1)p(w_3|w_1,w_2)\dots p(w_n|w_{n-1},w_{n-2}) \end{aligned}$$

c(wi)如下:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

c(wi-1,wi)如下:

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

I want english food

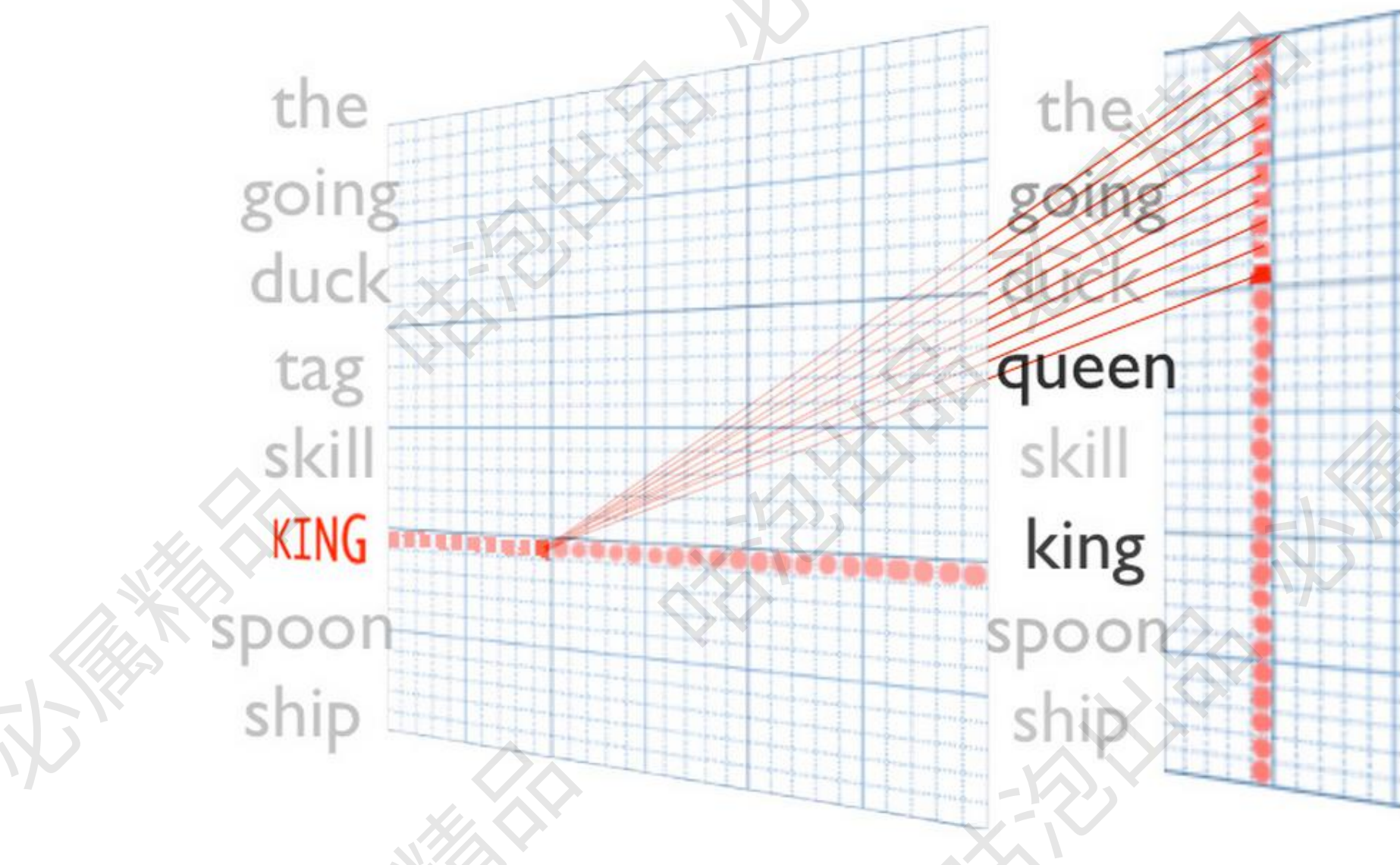
$$p(I \text{ want chinese food}) = P(\text{want}|I) \\ \times P(\text{chinese}|\text{want}) \\ \times P(\text{food}|\text{chinese})$$

语言模型

n	模型参数数量
1 (unigram)	2×10^5
2 (bigram)	4×10^{10}
3 (trigram)	8×10^{15}
4 (4-gram)	16×10^{20}

假设词典的大小是 N 则模型参数的量级是 $(O(N^n))$

词向量



词向量

expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$


词向量

Nearest words to **frog**:

1. frogs
2. toad
3. litoria
4. leptodactylidae
5. rana
6. lizard
7. eleutherodactylus



litoria



leptodactylidae

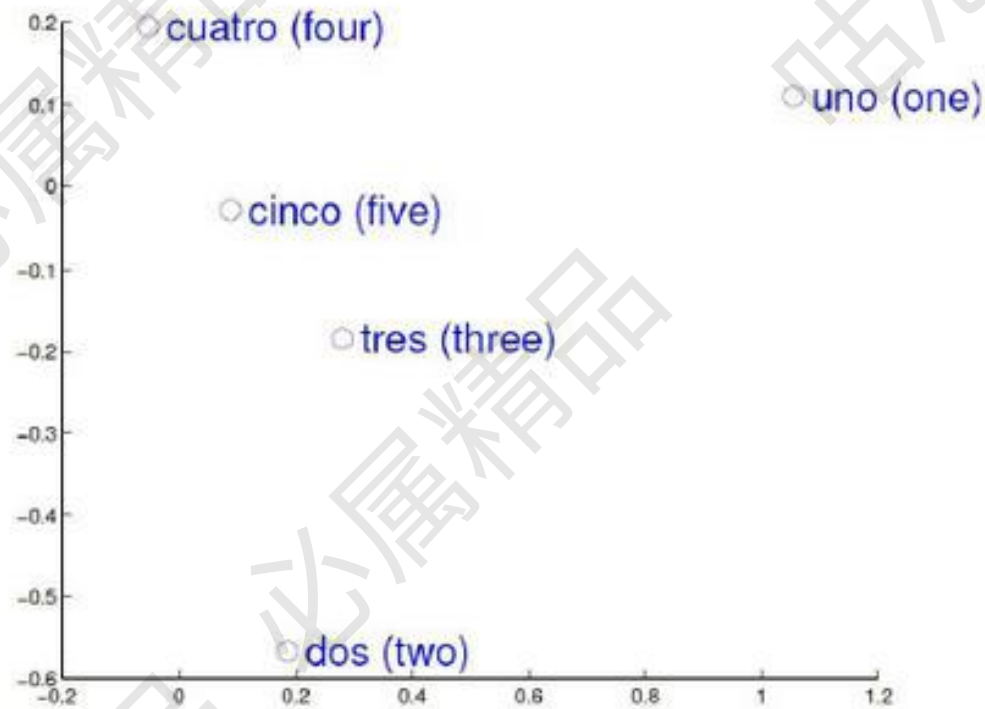


rana

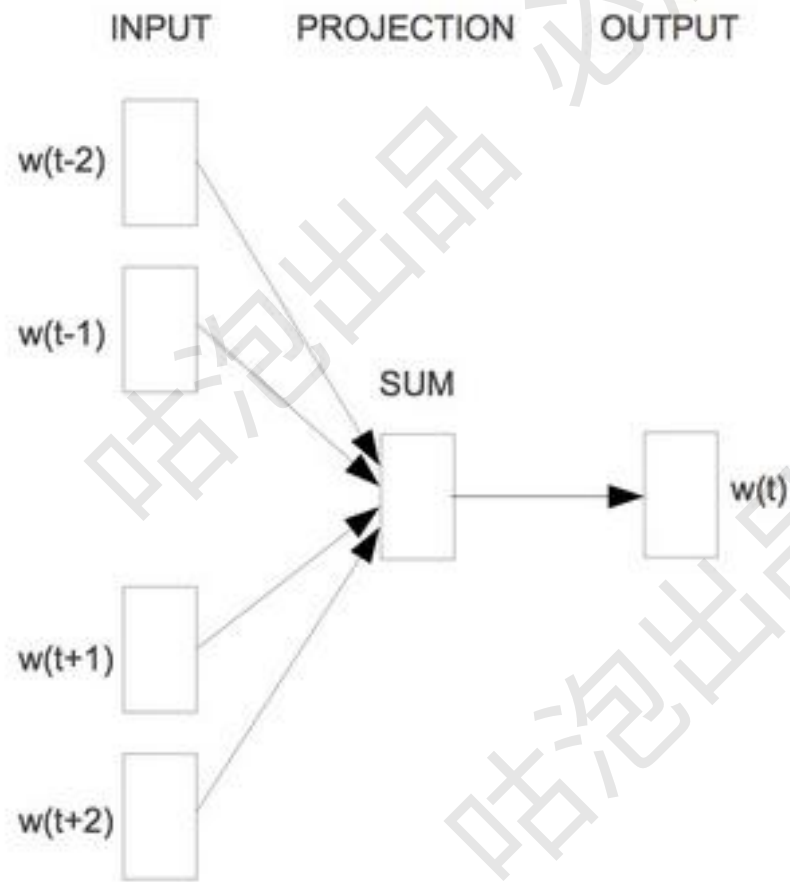


eleutherodactylus

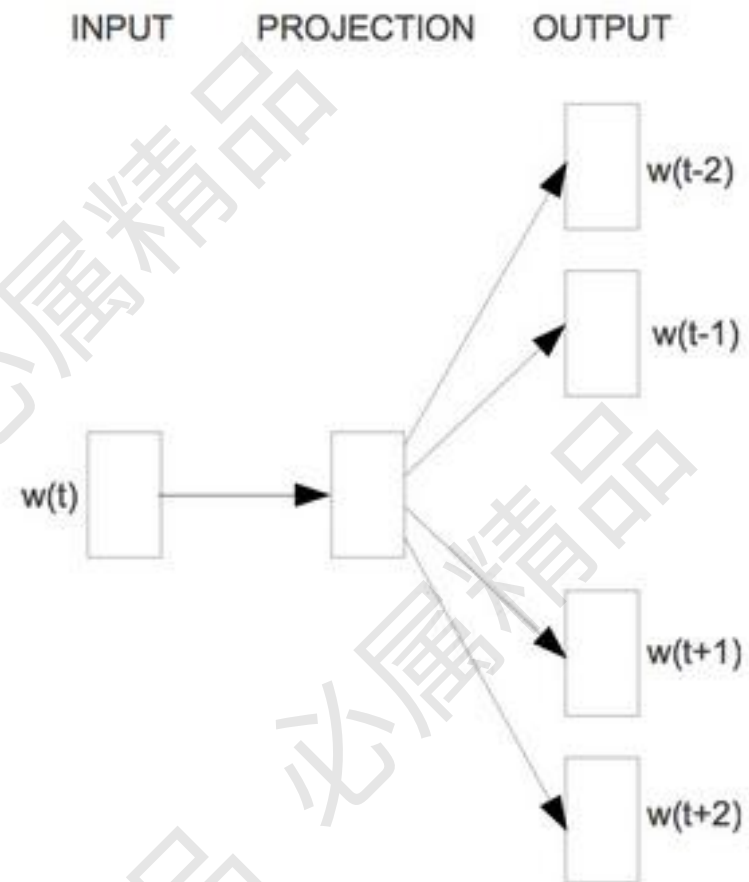
词向量



Hierarchical Softmax



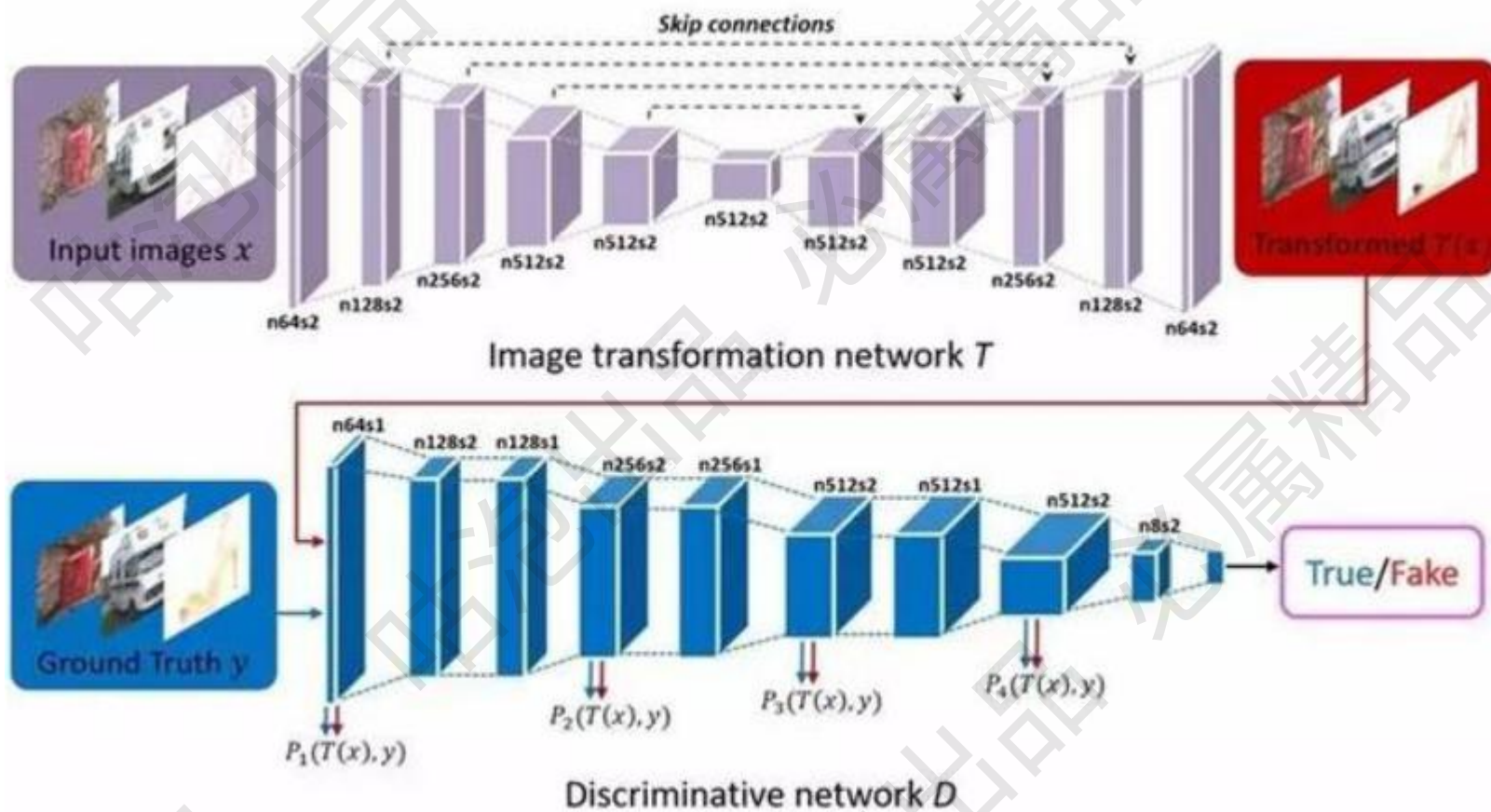
CBOW



Skip-gram

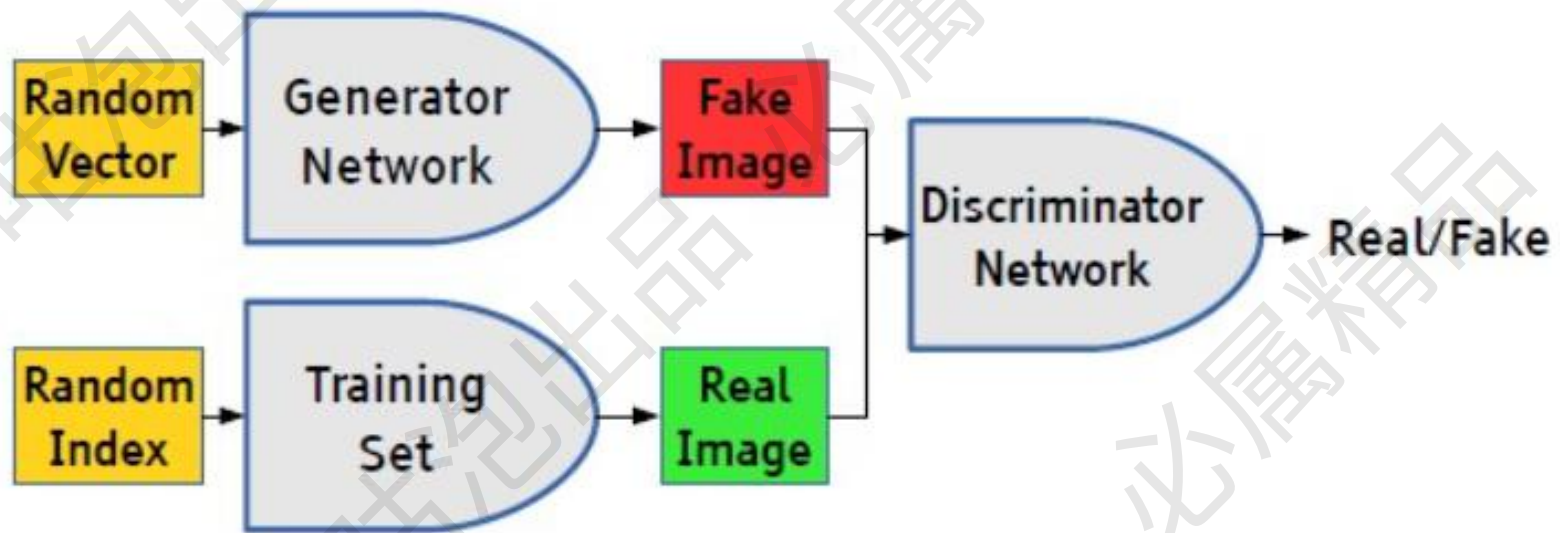
GAN

✓ 对抗生成网络



GAN

✓ 对抗生成网络



GAN

✓ 对抗生成网络

